CHAPTER 2

Solution of Linear Algebraic Equations

2.0 Introduction

The most basic task in linear algebra, and perhaps in all of scientific computing, is to solve for the unknowns in a set of linear algebraic equations. In general, a set of linear algebraic equations looks like this:

$$\begin{aligned} a_{00}x_0 + a_{01}x_1 + a_{02}x_2 + \cdots + a_{0,N-1}x_{N-1} &= b_0 \\ a_{10}x_0 + a_{11}x_1 + a_{12}x_2 + \cdots + a_{1,N-1}x_{N-1} &= b_1 \\ a_{20}x_0 + a_{21}x_1 + a_{22}x_2 + \cdots + a_{2,N-1}x_{N-1} &= b_2 \\ \cdots \qquad\qquad & \cdots \\ a_{M-1,0}x_0 + a_{M-1,1}x_1 + \cdots + a_{M-1,N-1}x_{N-1} &= b_{M-1} \end{aligned} \tag{2.0.1}$$

Here the N unknowns x_j, $j = 0, 1, \ldots, N-1$ are related by M equations. The coefficients a_{ij} with $i = 0, 1, \ldots, M-1$ and $j = 0, 1, \ldots, N-1$ are known numbers, as are the *right-hand side* quantities b_i, $i = 0, 1, \ldots, M-1$.

If $N = M$, then there are as many equations as unknowns, and there is a good chance of solving for a unique solution set of x_j's. Otherwise, if $N \neq M$, things are even more interesting; we'll have more to say about this below.

If we write the coefficients a_{ij} as a matrix, and the right-hand sides b_i as a column vector,

$$\mathbf{A} = \begin{bmatrix} a_{00} & a_{01} & \ldots & a_{0,N-1} \\ a_{10} & a_{11} & \ldots & a_{1,N-1} \\ & \cdots & & \\ a_{M-1,0} & a_{M-1,1} & \ldots & a_{M-1,N-1} \end{bmatrix} \qquad \mathbf{b} = \begin{bmatrix} b_0 \\ b_1 \\ \cdots \\ b_{M-1} \end{bmatrix} \tag{2.0.2}$$

then equation (2.0.1) can be written in matrix form as

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \tag{2.0.3}$$

Here, and throughout the book, we use a raised dot to denote matrix multiplication, *or* the multiplication of a matrix and a vector, *or* the dot product of two vectors.

This usage is nonstandard, but we think it adds clarity: the dot is, in all of these cases, a *contraction* operator that represents the sum over a pair of indices, for example

$$\begin{aligned} \mathbf{C} = \mathbf{A} \cdot \mathbf{B} \quad &\Longleftrightarrow \quad c_{ik} = \sum_j a_{ij} b_{jk} \\ \mathbf{b} = \mathbf{A} \cdot \mathbf{x} \quad &\Longleftrightarrow \quad b_i = \sum_j a_{ij} x_j \\ \mathbf{d} = \mathbf{x} \cdot \mathbf{A} \quad &\Longleftrightarrow \quad d_j = \sum_i x_i a_{ij} \\ q = \mathbf{x} \cdot \mathbf{y} \quad &\Longleftrightarrow \quad q = \sum_i x_i y_i \end{aligned} \tag{2.0.4}$$

In matrices, by convention, the first index on an element a_{ij} denotes its row and the second index its column. For most purposes you don't need to know how a matrix is stored in a computer's physical memory; you just reference matrix elements by their two-dimensional addresses, e.g., a_{34} = `a[3][4]`. This C++ notation can in fact hide a variety of subtle and versatile physical storage schemes, see §1.4 and §1.5.

2.0.1 Nonsingular versus Singular Sets of Equations

You might be wondering why, above, and for the case $M = N$, we credited only a "good" chance of solving for the unknowns. Analytically, there can fail to be a solution (or a unique solution) if one or more of the M equations is a linear combination of the others, a condition called *row degeneracy*, or if all equations contain certain variables only in exactly the same linear combination, called *column degeneracy*. It turns out that, for square matrices, row degeneracy implies column degeneracy, and vice versa. A set of equations that is degenerate is called *singular*. We will consider singular matrices in some detail in §2.6.

Numerically, at least two additional things prevent us from getting a good solution:

- While not exact linear combinations of each other, some of the equations may be so close to linearly dependent that roundoff errors in the machine render them linearly dependent at some stage in the solution process. In this case your numerical procedure will fail, and it can tell you that it has failed.
- Accumulated roundoff errors in the solution process can swamp the true solution. This problem particularly emerges if N is too large. The numerical procedure does not fail algorithmically. However, it returns a set of x's that are wrong, as can be discovered by direct substitution back into the original equations. The closer a set of equations is to being singular, the more likely this is to happen, since increasingly close cancellations will occur during the solution. In fact, the preceding item can be viewed as the special case in which the loss of significance is unfortunately total.

Much of the sophistication of well-written "linear equation-solving packages" is devoted to the detection and/or correction of these two pathologies. It is difficult to give any firm guidelines for when such sophistication is needed, since there is no such thing as a "typical" linear problem. But here is a rough idea: Linear sets with N no larger than 20 or 50 are routine if they are not close to singular; they rarely

require more than the most straightforward methods, even in only single (that is, `float`) precision. With `double` precision, this number can readily be extended to N as large as perhaps 1000, after which point the limiting factor anyway soon becomes machine time, not accuracy.

Even larger linear sets, N in the thousands or millions, can be solved when the coefficients are sparse (that is, mostly zero), by methods that take advantage of the sparseness. We discuss this further in §2.7.

Unfortunately, one seems just as often to encounter linear problems that, by their underlying nature, are close to singular. In this case, you *might* need to resort to sophisticated methods even for the case of $N = 10$ (though rarely for $N = 5$). Singular value decomposition (§2.6) is a technique that can sometimes turn singular problems into nonsingular ones, in which case additional sophistication becomes unnecessary.

2.0.2 Tasks of Computational Linear Algebra

There is much more to linear algebra than just solving a single set of equations with a single right-hand side. Here, we list the major topics treated in this chapter. (Chapter 11 continues the subject with discussion of eigenvalue/eigenvector problems.)

When $M = N$:

- Solution of the matrix equation $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ for an unknown vector $\mathbf{x}$ (§2.1 – §2.10).
- Solution of more than one matrix equation $\mathbf{A} \cdot \mathbf{x}_j = \mathbf{b}_j$, for a set of vectors $\mathbf{x}_j$, $j = 0, 1, \ldots$, each corresponding to a different, known right-hand side vector $\mathbf{b}_j$. In this task the key simplification is that the matrix $\mathbf{A}$ is held constant, while the right-hand sides, the $\mathbf{b}$'s, are changed (§2.1 – §2.10).
- Calculation of the matrix $\mathbf{A}^{-1}$ that is the matrix inverse of a square matrix $\mathbf{A}$, i.e., $\mathbf{A} \cdot \mathbf{A}^{-1} = \mathbf{A}^{-1} \cdot \mathbf{A} = \mathbf{1}$, where $\mathbf{1}$ is the identity matrix (all zeros except for ones on the diagonal). This task is equivalent, for an $N \times N$ matrix $\mathbf{A}$, to the previous task with N different $\mathbf{b}_j$'s ($j = 0, 1, \ldots, N-1$), namely the unit vectors ($\mathbf{b}_j$ = all zero elements except for 1 in the jth component). The corresponding $\mathbf{x}$'s are then the columns of the matrix inverse of $\mathbf{A}$ (§2.1 and §2.3).
- Calculation of the determinant of a square matrix $\mathbf{A}$ (§2.3).

If $M < N$, or if $M = N$ but the equations are degenerate, then there are effectively fewer equations than unknowns. In this case there can be either no solution, or else more than one solution vector $\mathbf{x}$. In the latter event, the solution space consists of a particular solution $\mathbf{x}_p$ added to any linear combination of (typically) $N - M$ vectors (which are said to be in the nullspace of the matrix $\mathbf{A}$). The task of finding the solution space of $\mathbf{A}$ involves

- Singular value decomposition of a matrix $\mathbf{A}$ (§2.6).

If there are more equations than unknowns, $M > N$, there is in general no solution vector $\mathbf{x}$ to equation (2.0.1), and the set of equations is said to be *overdetermined*. It happens frequently, however, that the best "compromise" solution is sought, the one that comes closest to satisfying all equations simultaneously. If closeness is defined in the least-squares sense, i.e., that the sum of the squares of the differences between the left- and right-hand sides of equation (2.0.1) be mini-

mized, then the overdetermined linear problem reduces to a (usually) solvable linear problem, called the

- Linear least-squares problem.

The reduced set of equations to be solved can be written as the $N \times N$ set of equations

$$(\mathbf{A}^T \cdot \mathbf{A}) \cdot \mathbf{x} = (\mathbf{A}^T \cdot \mathbf{b}) \tag{2.0.5}$$

where $\mathbf{A}^T$ denotes the transpose of the matrix $\mathbf{A}$. Equations (2.0.5) are called the *normal equations* of the linear least-squares problem. There is a close connection between singular value decomposition and the linear least-squares problem, and the latter is also discussed in §2.6. You should be warned that direct solution of the normal equations (2.0.5) is not generally the best way to find least-squares solutions.

Some other topics in this chapter include

- Iterative improvement of a solution (§2.5)
- Various special forms: symmetric positive-definite (§2.9), tridiagonal (§2.4), band-diagonal (§2.4), Toeplitz (§2.8), Vandermonde (§2.8), sparse (§2.7)
- Strassen's "fast matrix inversion" (§2.11).

2.0.3 Software for Linear Algebra

Going beyond what we can include in this book, several good software packages for linear algebra are available, though not always in C++. LAPACK, a successor to the venerable LINPACK, was developed at Argonne National Laboratories and deserves particular mention because it is published, documented, and available for free use. ScaLAPACK is a version available for parallel architectures. Packages available commercially include those in the IMSL and NAG libraries.

Sophisticated packages are designed with very large linear systems in mind. They therefore go to great effort to minimize not only the number of operations, but also the required storage. Routines for the various tasks are usually provided in several versions, corresponding to several possible simplifications in the form of the input coefficient matrix: symmetric, triangular, banded, positive-definite, etc. If you have a large matrix in one of these forms, you should certainly take advantage of the increased efficiency provided by these different routines, and not just use the form provided for general matrices.

There is also a great watershed dividing routines that are *direct* (i.e., execute in a predictable number of operations) from routines that are *iterative* (i.e., attempt to converge to the desired answer in however many steps are necessary). Iterative methods become preferable when the battle against loss of significance is in danger of being lost, either due to large N or because the problem is close to singular. We will treat iterative methods only incompletely in this book, in §2.7 and in Chapters 19 and 20. These methods are important but mostly beyond our scope. We will, however, discuss in detail a technique that is on the borderline between direct and iterative methods, namely the iterative improvement of a solution that has been obtained by direct methods (§2.5).

CITED REFERENCES AND FURTHER READING:

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press).

Gill, P.E., Murray, W., and Wright, M.H. 1991, *Numerical Linear Algebra and Optimization*, vol. 1 (Redwood City, CA: Addison-Wesley).

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 4.

Ueberhuber, C.W. 1997, *Numerical Computation: Methods, Software, and Analysis*, 2 vols. (Berlin: Springer), Chapter 13.

Coleman, T.F., and Van Loan, C. 1988, *Handbook for Matrix Computations* (Philadelphia: S.I.A.M.).

Forsythe, G.E., and Moler, C.B. 1967, *Computer Solution of Linear Algebraic Systems* (Englewood Cliffs, NJ: Prentice-Hall).

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).

Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).

Johnson, L.W., and Riess, R.D. 1982, *Numerical Analysis*, 2nd ed. (Reading, MA: Addison-Wesley), Chapter 2.

Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), Chapter 9.

2.1 Gauss-Jordan Elimination

Gauss-Jordan elimination is probably the way you learned to solve linear equations in high school. (You may have learned it as "Gaussian elimination," but, strictly speaking, that term refers to the somewhat different technique discussed in §2.2.) The basic idea is to add or subtract linear combinations of the given equations until each equation contains only one of the unknowns, thus giving an immediate solution. You might also have learned to use the same technique for calculating the inverse of a matrix.

For solving sets of linear equations, Gauss-Jordan elimination produces *both* the solution of the equations for one or more right-hand side vectors $\mathbf{b}$, and also the matrix inverse $\mathbf{A}^{-1}$. However, its principal deficiencies are (i) that it requires all the right-hand sides to be stored and manipulated at the same time, and (ii) that when the inverse matrix is *not* desired, Gauss-Jordan is three times slower than the best alternative technique for solving a single linear set (§2.3). The method's principal strength is that it is as stable as any other direct method, perhaps even a bit more stable when full pivoting is used (see §2.1.2).

For inverting a matrix, Gauss-Jordan elimination is about as efficient as any other direct method. We know of no reason not to use it in this application if you are sure that the matrix inverse is what you really want.

You might wonder about deficiency (i) above: If we are getting the matrix inverse anyway, can't we later let it multiply a new right-hand side to get an additional solution? This does work, but it gives an answer that is very susceptible to roundoff error and not nearly as good as if the new vector had been included with the set of right-hand side vectors in the first instance.

Thus, Gauss-Jordan elimination should not be your method of first choice for solving linear equations. The decomposition methods in §2.3 are better. Why do we discuss Gauss-Jordan at all? Because it is straightforward, solid as a rock, and a good place for us to introduce the important concept of *pivoting* which will also

be important for the methods described later. The actual sequence of operations performed in Gauss-Jordan elimination is very closely related to that performed by the routines in the next two sections.

2.1.1 Elimination on Column-Augmented Matrices

For clarity, and to avoid writing endless ellipses ($\cdots$) we will write out equations only for the case of four equations and four unknowns, and with three different right-hand side vectors that are known in advance. You can write bigger matrices and extend the equations to the case of $N \times N$ matrices, with M sets of right-hand side vectors, in completely analogous fashion. The routine implemented below in §2.1.2 is, of course, general.

Consider the linear matrix equation

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} \cdot \left[\begin{pmatrix} x_{00} \\ x_{10} \\ x_{20} \\ x_{30} \end{pmatrix} \sqcup \begin{pmatrix} x_{01} \\ x_{11} \\ x_{21} \\ x_{31} \end{pmatrix} \sqcup \begin{pmatrix} x_{02} \\ x_{12} \\ x_{22} \\ x_{32} \end{pmatrix} \sqcup \begin{pmatrix} y_{00} & y_{01} & y_{02} & y_{03} \\ y_{10} & y_{11} & y_{12} & y_{13} \\ y_{20} & y_{21} & y_{22} & y_{23} \\ y_{30} & y_{31} & y_{32} & y_{33} \end{pmatrix} \right]$$
$$= \left[\begin{pmatrix} b_{00} \\ b_{10} \\ b_{20} \\ b_{30} \end{pmatrix} \sqcup \begin{pmatrix} b_{01} \\ b_{11} \\ b_{21} \\ b_{31} \end{pmatrix} \sqcup \begin{pmatrix} b_{02} \\ b_{12} \\ b_{22} \\ b_{32} \end{pmatrix} \sqcup \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \right] \tag{2.1.1}$$

Here the raised dot ($\cdot$) signifies matrix multiplication, while the operator $\sqcup$ just signifies column augmentation, that is, removing the abutting parentheses and making a wider matrix out of the operands of the $\sqcup$ operator.

It should not take you long to write out equation (2.1.1) and to see that it simply states that x_{ij} is the ith component ($i = 0, 1, 2, 3$) of the vector solution of the jth right-hand side ($j = 0, 1, 2$), the one whose coefficients are $b_{ij}, i = 0, 1, 2, 3$; and that the matrix of unknown coefficients y_{ij} is the inverse matrix of a_{ij}. In other words, the matrix solution of

$$[\mathbf{A}] \cdot [\mathbf{x}_0 \sqcup \mathbf{x}_1 \sqcup \mathbf{x}_2 \sqcup \mathbf{Y}] = [\mathbf{b}_0 \sqcup \mathbf{b}_1 \sqcup \mathbf{b}_2 \sqcup \mathbf{1}] \tag{2.1.2}$$

where $\mathbf{A}$ and $\mathbf{Y}$ are square matrices, the $\mathbf{b}_i$'s and $\mathbf{x}_i$'s are column vectors, and $\mathbf{1}$ is the identity matrix, simultaneously solves the linear sets

$$\mathbf{A} \cdot \mathbf{x}_0 = \mathbf{b}_0 \qquad \mathbf{A} \cdot \mathbf{x}_1 = \mathbf{b}_1 \qquad \mathbf{A} \cdot \mathbf{x}_2 = \mathbf{b}_2 \tag{2.1.3}$$

and

$$\mathbf{A} \cdot \mathbf{Y} = \mathbf{1} \tag{2.1.4}$$

Now it is also elementary to verify the following facts about (2.1.1):

- Interchanging any two *rows* of $\mathbf{A}$ and the corresponding *rows* of the $\mathbf{b}$'s and of $\mathbf{1}$ does not change (or scramble in any way) the solution $\mathbf{x}$'s and $\mathbf{Y}$. Rather, it just corresponds to writing the same set of linear equations in a different order.
- Likewise, the solution set is unchanged and in no way scrambled if we replace any row in $\mathbf{A}$ by a linear combination of itself and any other row, as long as we do the same linear combination of the rows of the $\mathbf{b}$'s and $\mathbf{1}$ (which then is no longer the identity matrix, of course).

- Interchanging any two *columns* of **A** gives the same solution set only if we simultaneously interchange corresponding *rows* of the **x**'s and of **Y**. In other words, this interchange scrambles the order of the rows in the solution. If we do this, we will need to unscramble the solution by restoring the rows to their original order.

Gauss-Jordan elimination uses one or more of the above operations to reduce the matrix **A** to the identity matrix. When this is accomplished, the right-hand side becomes the solution set, as one sees instantly from (2.1.2).

2.1.2 Pivoting

In "Gauss-Jordan elimination with no pivoting," only the second operation in the above list is used. The zeroth row is divided by the element a_{00} (this being a trivial linear combination of the zeroth row with any other row — zero coefficient for the other row). Then the right amount of the zeroth row is subtracted from each other row to make all the remaining a_{i0}'s zero. The zeroth column of **A** now agrees with the identity matrix. We move to column 1 and divide row 1 by a_{11}, then subtract the right amount of row 1 from rows 0, 2, and 3, so as to make their entries in column 1 zero. Column 1 is now reduced to the identity form. And so on for columns 2 and 3. As we do these operations to **A**, we of course also do the corresponding operations to the **b**'s and to **1** (which by now no longer resembles the identity matrix in any way!).

Obviously we will run into trouble if we ever encounter a zero element on the (then current) diagonal when we are going to divide by the diagonal element. (The element that we divide by, incidentally, is called the *pivot element* or *pivot*.) Not so obvious, but true, is the fact that Gauss-Jordan elimination with no pivoting (no use of the first or third procedures in the above list) is numerically unstable in the presence of any roundoff error, even when a zero pivot is not encountered. You must *never* do Gauss-Jordan elimination (or Gaussian elimination; see below) without pivoting!

So what *is* this magic pivoting? Nothing more than interchanging rows (*partial pivoting*) or rows and columns (*full pivoting*), so as to put a particularly desirable element in the diagonal position from which the pivot is about to be selected. Since we don't want to mess up the part of the identity matrix that we have already built up, we can choose among elements that are both (i) on rows below (or on) the one that is about to be normalized, and also (ii) on columns to the right (or on) the column we are about to eliminate. Partial pivoting is easier than full pivoting, because we don't have to keep track of the permutation of the solution vector. Partial pivoting makes available as pivots only the elements already in the correct column. It turns out that partial pivoting is "almost" as good as full pivoting, in a sense that can be made mathematically precise, but which need not concern us here (for discussion and references, see [1]). To show you both variants, we do full pivoting in the routine in this section and partial pivoting in §2.3.

We have to state how to recognize a particularly desirable pivot when we see one. The answer to this is not completely known theoretically. It is known, both theoretically and in practice, that simply picking the largest (in magnitude) available element as the pivot is a very good choice. A curiosity of this procedure, however, is that the choice of pivot will depend on the original scaling of the equations. If we take the third linear equation in our original set and multiply it by a factor of a million, it is almost guaranteed that it will contribute the first pivot; yet the underlying solution

of the equations is not changed by this multiplication! One therefore sometimes sees routines which choose as pivot that element which *would* have been largest if the original equations had all been scaled to have their largest coefficient normalized to unity. This is called *implicit pivoting*. There is some extra bookkeeping to keep track of the scale factors by which the rows would have been multiplied. (The routines in §2.3 include implicit pivoting, but the routine in this section does not.)

Finally, let us consider the storage requirements of the method. With a little reflection you will see that at every stage of the algorithm, *either* an element of **A** is predictably a one or zero (if it is already in a part of the matrix that has been reduced to identity form) *or else* the exactly corresponding element of the matrix that started as **1** is predictably a one or zero (if its mate in **A** has not been reduced to the identity form). Therefore the matrix **1** does not have to exist as separate storage: The matrix inverse of **A** is gradually built up in **A** as the original **A** is destroyed. Likewise, the solution vectors **x** can gradually replace the right-hand side vectors **b** and share the same storage, since after each column in **A** is reduced, the corresponding row entry in the **b**'s is never again used.

Here is a routine that does Gauss-Jordan elimination with full pivoting, replacing its input matrices by the desired answers. Immediately following is an overloaded version for use when there are no right-hand sides, i.e., when you want only the matrix inverse.

gaussj.h

```
void gaussj(MatDoub_IO &a, MatDoub_IO &b)
```

Linear equation solution by Gauss-Jordan elimination, equation (2.1.1) above. The input matrix is `a[0..n-1][0..n-1]`. `b[0..n-1][0..m-1]` is input containing the `m` right-hand side vectors. On output, `a` is replaced by its matrix inverse, and `b` is replaced by the corresponding set of solution vectors.

```
{
    Int i,icol,irow,j,k,l,ll,n=a.nrows(),m=b.ncols();
    Doub big,dum,pivinv;
    VecInt indxc(n),indxr(n),ipiv(n);
    for (j=0;j<n;j++) ipiv[j]=0;
    for (i=0;i<n;i++) {
        big=0.0;
        for (j=0;j<n;j++)
            if (ipiv[j] != 1)
                for (k=0;k<n;k++) {
                    if (ipiv[k] == 0) {
                        if (abs(a[j][k]) >= big) {
                            big=abs(a[j][k]);
                            irow=j;
                            icol=k;
                        }
                    }
                }
        ++(ipiv[icol]);
```

These integer arrays are used for bookkeeping on the pivoting.

This is the main loop over the columns to be reduced.

This is the outer loop of the search for a pivot element.

We now have the pivot element, so we interchange rows, if needed, to put the pivot element on the diagonal. The columns are not physically interchanged, only relabeled: `indxc[i]`, the column of the $(i+1)$th pivot element, is the $(i+1)$th column that is reduced, while `indxr[i]` is the row in which that pivot element was originally located. If `indxr[i]` $\neq$ `indxc[i]`, there is an implied column interchange. With this form of bookkeeping, the solution b's will end up in the correct order, and the inverse matrix will be scrambled by columns.

```
        if (irow != icol) {
            for (l=0;l<n;l++) SWAP(a[irow][l],a[icol][l]);
            for (l=0;l<m;l++) SWAP(b[irow][l],b[icol][l]);
        }
        indxr[i]=irow;
```

We are now ready to divide the pivot row by the pivot element, located at `irow` and `icol`.

```
        indxc[i]=icol;
        if (a[icol][icol] == 0.0) throw("gaussj: Singular Matrix");
        pivinv=1.0/a[icol][icol];
        a[icol][icol]=1.0;
        for (l=0;l<n;l++) a[icol][l] *= pivinv;
        for (l=0;l<m;l++) b[icol][l] *= pivinv;
        for (ll=0;ll<n;ll++)                Next, we reduce the rows...
            if (ll != icol) {               ...except for the pivot one, of course.
                dum=a[ll][icol];
                a[ll][icol]=0.0;
                for (l=0;l<n;l++) a[ll][l] -= a[icol][l]*dum;
                for (l=0;l<m;l++) b[ll][l] -= b[icol][l]*dum;
            }
    }
    This is the end of the main loop over columns of the reduction. It only remains to unscramble the solution in view of the column interchanges. We do this by interchanging pairs of columns in the reverse order that the permutation was built up.
    for (l=n-1;l>=0;l--) {
        if (indxr[l] != indxc[l])
            for (k=0;k<n;k++)
                SWAP(a[k][indxr[l]],a[k][indxc[l]]);
    }                                       And we are done.
}

void gaussj(MatDoub_IO &a)
Overloaded version with no right-hand sides. Replaces a by its inverse.
{
    MatDoub b(a.nrows(),0);          Dummy vector with zero columns.
    gaussj(a,b);
}
```

2.1.3 Row versus Column Elimination Strategies

The above discussion can be amplified by a modest amount of formalism. Row operations on a matrix $\mathbf{A}$ correspond to pre- (that is, left-) multiplication by some simple matrix $\mathbf{R}$. For example, the matrix $\mathbf{R}$ with components

$$R_{ij} = \begin{cases} 1 & \text{if } i = j \text{ and } i \neq 2, 4 \\ 1 & \text{if } i = 2,\ j = 4 \\ 1 & \text{if } i = 4,\ j = 2 \\ 0 & \text{otherwise} \end{cases} \tag{2.1.5}$$

effects the interchange of rows 2 and 4. Gauss-Jordan elimination by row operations alone (including the possibility of *partial* pivoting) consists of a series of such left-multiplications, yielding successively

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} &= \mathbf{b} \\ (\cdots \mathbf{R}_2 \cdot \mathbf{R}_1 \cdot \mathbf{R}_0 \cdot \mathbf{A}) \cdot \mathbf{x} &= \cdots \mathbf{R}_2 \cdot \mathbf{R}_1 \cdot \mathbf{R}_0 \cdot \mathbf{b} \\ (\mathbf{1}) \cdot \mathbf{x} &= \cdots \mathbf{R}_2 \cdot \mathbf{R}_1 \cdot \mathbf{R}_0 \cdot \mathbf{b} \\ \mathbf{x} &= \cdots \mathbf{R}_2 \cdot \mathbf{R}_1 \cdot \mathbf{R}_0 \cdot \mathbf{b} \end{aligned} \tag{2.1.6}$$

The key point is that since the $\mathbf{R}$'s build from right to left, the right-hand side is simply transformed at each stage from one vector to another.

Column operations, on the other hand, correspond to post-, or right-, multiplications by simple matrices, call them $\mathbf{C}$. The matrix in equation (2.1.5), if right-multiplied onto a matrix $\mathbf{A}$, will interchange *columns* 2 and 4 of $\mathbf{A}$. Elimination by column operations involves (conceptually) inserting a column operator, *and also its inverse,* between the matrix $\mathbf{A}$ and the

unknown vector $\mathbf{x}$:

$$\begin{aligned}
\mathbf{A}\cdot\mathbf{x} &= \mathbf{b} \\
\mathbf{A}\cdot\mathbf{C}_0\cdot\mathbf{C}_0^{-1}\cdot\mathbf{x} &= \mathbf{b} \\
\mathbf{A}\cdot\mathbf{C}_0\cdot\mathbf{C}_1\cdot\mathbf{C}_1^{-1}\cdot\mathbf{C}_0^{-1}\cdot\mathbf{x} &= \mathbf{b} \\
(\mathbf{A}\cdot\mathbf{C}_0\cdot\mathbf{C}_1\cdot\mathbf{C}_2\cdots)\cdots\mathbf{C}_2^{-1}\cdot\mathbf{C}_1^{-1}\cdot\mathbf{C}_0^{-1}\cdot\mathbf{x} &= \mathbf{b} \\
(\mathbf{1})\cdots\mathbf{C}_2^{-1}\cdot\mathbf{C}_1^{-1}\cdot\mathbf{C}_0^{-1}\cdot\mathbf{x} &= \mathbf{b}
\end{aligned} \tag{2.1.7}$$

which (peeling off the $\mathbf{C}^{-1}$'s one at a time) implies a solution

$$\mathbf{x} = \mathbf{C}_0\cdot\mathbf{C}_1\cdot\mathbf{C}_2\cdots\mathbf{b} \tag{2.1.8}$$

Notice the essential difference between equation (2.1.8) and equation (2.1.6). In the latter case, the $\mathbf{C}$'s must be applied to $\mathbf{b}$ in the *reverse order* from that in which they become known. That is, they must all be stored along the way. This requirement greatly reduces the usefulness of column operations, generally restricting them to simple permutations, for example in support of full pivoting.

CITED REFERENCES AND FURTHER READING:

Wilkinson, J.H. 1965, *The Algebraic Eigenvalue Problem* (New York: Oxford University Press).[1]

Carnahan, B., Luther, H.A., and Wilkes, J.O. 1969, *Applied Numerical Methods* (New York: Wiley), Example 5.2, p. 282.

Bevington, P.R., and Robinson, D.K. 2002, *Data Reduction and Error Analysis for the Physical Sciences*, 3rd ed. (New York: McGraw-Hill), p. 247.

Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).

Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §9.3–1.

2.2 Gaussian Elimination with Backsubstitution

Any discussion of *Gaussian elimination with backsubstitution* is primarily pedagogical. The method stands between full elimination schemes such as Gauss-Jordan, and triangular decomposition schemes such as will be discussed in the next section. Gaussian elimination reduces a matrix not all the way to the identity matrix, but only halfway, to a matrix whose components on the diagonal and above (say) remain nontrivial. Let us now see what advantages accrue.

Suppose that in doing Gauss-Jordan elimination, as described in §2.1, we at each stage subtract away rows only *below* the then-current pivot element. When a_{11} is the pivot element, for example, we divide the row 1 by its value (as before), but now use the pivot row to zero only a_{21} and a_{31}, not a_{01} (see equation 2.1.1). Suppose, also, that we do only partial pivoting, never interchanging columns, so that the order of the unknowns never needs to be modified.

Then, when we have done this for all the pivots, we will be left with a reduced equation that looks like this (in the case of a single right-hand side vector):

$$\begin{bmatrix} a'_{00} & a'_{01} & a'_{02} & a'_{03} \\ 0 & a'_{11} & a'_{12} & a'_{13} \\ 0 & 0 & a'_{22} & a'_{23} \\ 0 & 0 & 0 & a'_{33} \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b'_0 \\ b'_1 \\ b'_2 \\ b'_3 \end{bmatrix} \tag{2.2.1}$$

Here the primes signify that the a's and b's do not have their original numerical values, but have been modified by all the row operations in the elimination to this point. The procedure up to this point is termed *Gaussian elimination*.

2.2.1 Backsubstitution

But how do we solve for the x's? The last x (x_3 in this example) is already isolated, namely

$$x_3 = b'_3/a'_{33} \tag{2.2.2}$$

With the last x known we can move to the penultimate x,

$$x_2 = \frac{1}{a'_{22}}[b'_2 - x_3 a'_{23}] \tag{2.2.3}$$

and then proceed with the x before that one. The typical step is

$$x_i = \frac{1}{a'_{ii}}\left[b'_i - \sum_{j=i+1}^{N-1} a'_{ij} x_j\right] \tag{2.2.4}$$

The procedure defined by equation (2.2.4) is called *backsubstitution*. The combination of Gaussian elimination and backsubstitution yields a solution to the set of equations.

The advantage of Gaussian elimination and backsubstitution over Gauss-Jordan elimination is simply that the former is faster in raw operations count: The innermost loops of Gauss-Jordan elimination, each containing one subtraction and one multiplication, are executed N^3 and $N^2 m$ times (where there are N equations and unknowns, and m different right-hand sides). The corresponding loops in Gaussian elimination are executed only $\frac{1}{3}N^3$ times (only half the matrix is reduced, and the increasing numbers of predictable zeros reduce the count to one-third), and $\frac{1}{2}N^2 m$ times, respectively. Each backsubstitution of a right-hand side is $\frac{1}{2}N^2$ executions of a similar loop (one multiplication plus one subtraction). For $m \ll N$ (only a few right-hand sides) Gaussian elimination thus has about a factor three advantage over Gauss-Jordan. (We could reduce this advantage to a factor 1.5 by *not* computing the inverse matrix as part of the Gauss-Jordan scheme.)

For computing the inverse matrix (which we can view as the case of $m = N$ right-hand sides, namely the N unit vectors that are the columns of the identity matrix), Gaussian elimination and backsubstitution at first glance require $\frac{1}{3}N^3$ (matrix reduction) $+\frac{1}{2}N^3$ (right-hand side manipulations) $+\frac{1}{2}N^3$ (N backsubstitutions) $= \frac{4}{3}N^3$ loop executions, which is more than the N^3 for Gauss-Jordan. However, the unit vectors are quite special in containing all zeros except for one element. If this

is taken into account, the right-side manipulations can be reduced to only $\frac{1}{6}N^3$ loop executions, and, for matrix inversion, the two methods have identical efficiencies.

Both Gaussian elimination and Gauss-Jordan elimination share the disadvantage that all right-hand sides must be known in advance. The LU decomposition method in the next section does not share that deficiency, and also has an equally small operations count, both for solution with any number of right-hand sides and for matrix inversion.

CITED REFERENCES AND FURTHER READING:

Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §9.3–1.

Isaacson, E., and Keller, H.B. 1966, *Analysis of Numerical Methods*; reprinted 1994 (New York: Dover), §2.1.

Johnson, L.W., and Riess, R.D. 1982, *Numerical Analysis*, 2nd ed. (Reading, MA: Addison-Wesley), §2.2.1.

Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).

2.3 LU Decomposition and Its Applications

Suppose we are able to write the matrix $\mathbf{A}$ as a product of two matrices,

$$\mathbf{L} \cdot \mathbf{U} = \mathbf{A} \tag{2.3.1}$$

where $\mathbf{L}$ is *lower triangular* (has elements only on the diagonal and below) and $\mathbf{U}$ is *upper triangular* (has elements only on the diagonal and above). For the case of a 4×4 matrix $\mathbf{A}$, for example, equation (2.3.1) would look like this:

$$\begin{bmatrix} \alpha_{00} & 0 & 0 & 0 \\ \alpha_{10} & \alpha_{11} & 0 & 0 \\ \alpha_{20} & \alpha_{21} & \alpha_{22} & 0 \\ \alpha_{30} & \alpha_{31} & \alpha_{32} & \alpha_{33} \end{bmatrix} \cdot \begin{bmatrix} \beta_{00} & \beta_{01} & \beta_{02} & \beta_{03} \\ 0 & \beta_{11} & \beta_{12} & \beta_{13} \\ 0 & 0 & \beta_{22} & \beta_{23} \\ 0 & 0 & 0 & \beta_{33} \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} \tag{2.3.2}$$

We can use a decomposition such as (2.3.1) to solve the linear set

$$\mathbf{A} \cdot \mathbf{x} = (\mathbf{L} \cdot \mathbf{U}) \cdot \mathbf{x} = \mathbf{L} \cdot (\mathbf{U} \cdot \mathbf{x}) = \mathbf{b} \tag{2.3.3}$$

by first solving for the vector $\mathbf{y}$ such that

$$\mathbf{L} \cdot \mathbf{y} = \mathbf{b} \tag{2.3.4}$$

and then solving

$$\mathbf{U} \cdot \mathbf{x} = \mathbf{y} \tag{2.3.5}$$

What is the advantage of breaking up one linear set into two successive ones? The advantage is that the solution of a triangular set of equations is quite trivial, as

we have already seen in §2.2 (equation 2.2.4). Thus, equation (2.3.4) can be solved by *forward substitution* as follows:

$$\begin{aligned} y_0 &= \frac{b_0}{\alpha_{00}} \\ y_i &= \frac{1}{\alpha_{ii}}\left[b_i - \sum_{j=0}^{i-1} \alpha_{ij} y_j\right] \qquad i = 1, 2, \ldots, N-1 \end{aligned} \tag{2.3.6}$$

while (2.3.5) can then be solved by *backsubstitution* exactly as in equations (2.2.2) – (2.2.4),

$$\begin{aligned} x_{N-1} &= \frac{y_{N-1}}{\beta_{N-1,N-1}} \\ x_i &= \frac{1}{\beta_{ii}}\left[y_i - \sum_{j=i+1}^{N-1} \beta_{ij} x_j\right] \qquad i = N-2, N-3, \ldots, 0 \end{aligned} \tag{2.3.7}$$

Equations (2.3.6) and (2.3.7) total (for each right-hand side **b**) N^2 executions of an inner loop containing one multiply and one add. If we have N right-hand sides that are the unit column vectors (which is the case when we are inverting a matrix), then taking into account the leading zeros reduces the total execution count of (2.3.6) from $\frac{1}{2}N^3$ to $\frac{1}{6}N^3$, while (2.3.7) is unchanged at $\frac{1}{2}N^3$.

Notice that, once we have the LU decomposition of **A**, we can solve with as many right-hand sides as we then care to, one at a time. This is a distinct advantage over the methods of §2.1 and §2.2.

2.3.1 Performing the LU Decomposition

How then can we solve for **L** and **U**, given **A**? First, we write out the i,jth component of equation (2.3.1) or (2.3.2). That component always is a sum beginning with

$$\alpha_{i0}\beta_{0j} + \cdots = a_{ij}$$

The number of terms in the sum depends, however, on whether i or j is the smaller number. We have, in fact, the three cases,

$$i < j: \qquad \alpha_{i0}\beta_{0j} + \alpha_{i1}\beta_{1j} + \cdots + \alpha_{ii}\beta_{ij} = a_{ij} \tag{2.3.8}$$

$$i = j: \qquad \alpha_{i0}\beta_{0j} + \alpha_{i1}\beta_{1j} + \cdots + \alpha_{ii}\beta_{jj} = a_{ij} \tag{2.3.9}$$

$$i > j: \qquad \alpha_{i0}\beta_{0j} + \alpha_{i1}\beta_{1j} + \cdots + \alpha_{ij}\beta_{jj} = a_{ij} \tag{2.3.10}$$

Equations (2.3.8) – (2.3.10) total N^2 equations for the $N^2 + N$ unknown α's and β's (the diagonal being represented twice). Since the number of unknowns is greater than the number of equations, we are invited to specify N of the unknowns arbitrarily and then try to solve for the others. In fact, as we shall see, it is always possible to take

$$\alpha_{ii} \equiv 1 \qquad i = 0, \ldots, N-1 \tag{2.3.11}$$

A surprising procedure, now, is *Crout's algorithm*, which quite trivially solves the set of $N^2 + N$ equations (2.3.8) – (2.3.11) for all the α's and β's by just arranging the equations in a certain order! That order is as follows:

- Set $\alpha_{ii} = 1, i = 0, \ldots, N-1$ (equation 2.3.11).
- For each $j = 0, 1, 2, \ldots, N-1$ do these two procedures: First, for $i = 0, 1, \ldots, j$, use (2.3.8), (2.3.9), and (2.3.11) to solve for β_{ij}, namely

$$\beta_{ij} = a_{ij} - \sum_{k=0}^{i-1} \alpha_{ik}\beta_{kj} \tag{2.3.12}$$

(When $i = 0$ in 2.3.12 the summation term is taken to mean zero.) Second, for $i = j+1, j+2, \ldots, N-1$ use (2.3.10) to solve for α_{ij}, namely

$$\alpha_{ij} = \frac{1}{\beta_{jj}} \left(a_{ij} - \sum_{k=0}^{j-1} \alpha_{ik}\beta_{kj} \right) \tag{2.3.13}$$

Be sure to do both procedures before going on to the next j.

If you work through a few iterations of the above procedure, you will see that the α's and β's that occur on the right-hand side of equations (2.3.12) and (2.3.13) are already determined by the time they are needed. You will also see that every a_{ij} is used only once and never again. This means that the corresponding α_{ij} or β_{ij} can be stored in the location that the a used to occupy: the decomposition is "in place." [The diagonal unity elements α_{ii} (equation 2.3.11) are not stored at all.] In brief, Crout's method fills in the combined matrix of α's and β's,

$$\begin{bmatrix} \beta_{00} & \beta_{01} & \beta_{02} & \beta_{03} \\ \alpha_{10} & \beta_{11} & \beta_{12} & \beta_{13} \\ \alpha_{20} & \alpha_{21} & \beta_{22} & \beta_{23} \\ \alpha_{30} & \alpha_{31} & \alpha_{32} & \beta_{33} \end{bmatrix} \tag{2.3.14}$$

by columns from left to right, and within each column from top to bottom (see Figure 2.3.1).

What about pivoting? Pivoting (i.e., selection of a salubrious pivot element for the division in equation 2.3.13) is absolutely essential for the stability of Crout's method. Only partial pivoting (interchange of rows) can be implemented efficiently. However this is enough to make the method stable. This means, incidentally, that we don't actually decompose the matrix $\mathbf{A}$ into LU form, but rather we decompose a rowwise permutation of $\mathbf{A}$. (If we keep track of what that permutation is, this decomposition is just as useful as the original one would have been.)

Pivoting is slightly subtle in Crout's algorithm. The key point to notice is that equation (2.3.12) in the case of $i = j$ (its final application) is *exactly the same* as equation (2.3.13) except for the division in the latter equation; in both cases the upper limit of the sum is $k = j-1$ $(= i-1)$. This means that we don't have to commit ourselves as to whether the diagonal element β_{jj} is the one that happens to fall on the diagonal in the first instance, or whether one of the (undivided) α_{ij}'s below it in the column, $i = j+1, \ldots, N-1$, is to be "promoted" to become the diagonal β. This can be decided after all the candidates in the column are in hand. As you should be able to guess by now, we will choose the largest one as the diagonal β (pivot

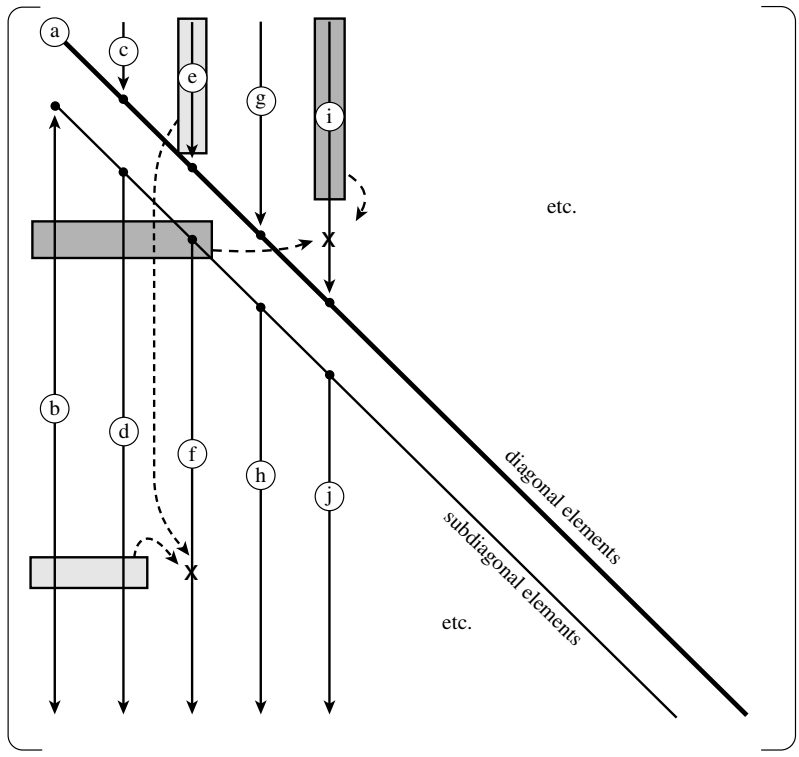


Figure 2.3.1. Crout's algorithm for LU decomposition of a matrix. Elements of the original matrix are modified in the order indicated by lowercase letters: a, b, c, etc. Shaded boxes show the previously modified elements that are used in modifying two typical elements, each indicated by an "x".

element), and then do all the divisions by that element *en masse*. This is *Crout's method with partial pivoting*. Our implementation has one additional wrinkle: It initially finds the largest element in each row, and subsequently (when it is looking for the maximal pivot element) scales the comparison *as if* we had initially scaled all the equations to make their maximum coefficient equal to unity; this is the *implicit pivoting* mentioned in §2.1.

The inner loop of the LU decomposition, equations (2.3.12) and (2.3.13), resembles the inner loop of matrix multiplication. There is a triple loop over the indices i, j, and k. There are six permutations of the order in which these loops can be done. The straightforward implementation of Crout's algorithm corresponds to the jik permutation, where the order of the indices is the order of the loops from outermost to innermost. On modern processors with a hierarchy of cache memory, and when matrices are stored by rows, the fastest execution time is usually the kij or ikj ordering. Pivoting is easier with kij ordering, so that is the implementation we use. This is called "outer product Gaussian elimination" by Golub and Van Loan [1].

LU decomposition is well suited for implementation as an object (a `class` or `struct`). The constructor performs the decomposition, and the object itself stores the result. Then, a method for forward- and backsubstitution can be called once, or many times, to solve for one or more right-hand sides. Methods for additional functionality are also easy to include. The object's declaration looks like this:

ludcmp.h
```
struct LUdcmp
```
Object for solving linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ using LU decomposition, and related functions.
```
{
    Int n;
    MatDoub lu;                                  Stores the decomposition.
    VecInt indx;                                 Stores the permutation.
    Doub d;                                      Used by det.
    LUdcmp(MatDoub_I &a);                        Constructor. Argument is the matrix A.
    void solve(VecDoub_I &b, VecDoub_O &x);      Solve for a single right-hand side.
    void solve(MatDoub_I &b, MatDoub_O &x);      Solve for multiple right-hand sides.
    void inverse(MatDoub_O &ainv);               Calculate matrix inverse A^-1.
    Doub det();                                  Return determinant of A.
    void mprove(VecDoub_I &b, VecDoub_IO &x);          Discussed in §2.5.
    MatDoub_I &aref;                             Used only by mprove.
};
```

Here is the implementation of the constructor, whose argument is the input matrix that is to be LU decomposed. The input matrix is not altered; a copy is made, on which outer product Gaussian elimination is then done in-place.

ludcmp.h
```
LUdcmp::LUdcmp(MatDoub_I &a) : n(a.nrows()), lu(a), aref(a), indx(n) {
```
Given a matrix `a[0..n-1][0..n-1]`, this routine replaces it by the LU decomposition of a rowwise permutation of itself. `a` is input. On output, it is arranged as in equation (2.3.14) above; `indx[0..n-1]` is an output vector that records the row permutation effected by the partial pivoting; `d` is output as ± 1 depending on whether the number of row interchanges was even or odd, respectively. This routine is used in combination with `solve` to solve linear equations or invert a matrix.
```
    const Doub TINY=1.0e-40;             A small number.
    Int i,imax,j,k;
    Doub big,temp;
    VecDoub vv(n);                       vv stores the implicit scaling of each row.
    d=1.0;                               No row interchanges yet.
    for (i=0;i<n;i++) {                  Loop over rows to get the implicit scaling infor-
        big=0.0;                             mation.
        for (j=0;j<n;j++)
            if ((temp=abs(lu[i][j])) > big) big=temp;
        if (big == 0.0) throw("Singular matrix in LUdcmp");
        No nonzero largest element.
        vv[i]=1.0/big;                   Save the scaling.
    }
    for (k=0;k<n;k++) {                  This is the outermost kij loop.
        big=0.0;                         Initialize for the search for largest pivot element.
        for (i=k;i<n;i++) {
            temp=vv[i]*abs(lu[i][k]);
            if (temp > big) {            Is the figure of merit for the pivot better than
                big=temp;                    the best so far?
                imax=i;
            }
        }
        if (k != imax) {                 Do we need to interchange rows?
            for (j=0;j<n;j++) {          Yes, do so...
                temp=lu[imax][j];
                lu[imax][j]=lu[k][j];
                lu[k][j]=temp;
            }
            d = -d;                      ...and change the parity of d.
            vv[imax]=vv[k];              Also interchange the scale factor.
        }
        indx[k]=imax;
        if (lu[k][k] == 0.0) lu[k][k]=TINY;
        If the pivot element is zero, the matrix is singular (at least to the precision of the
        algorithm). For some applications on singular matrices, it is desirable to substitute
```

```
        TINY for zero.
        for (i=k+1;i<n;i++) {
            temp=lu[i][k] /= lu[k][k];  Divide by the pivot element.
            for (j=k+1;j<n;j++)         Innermost loop: reduce remaining submatrix.
                lu[i][j] -= temp*lu[k][j];
        }
    }
}
```

Once the LUdcmp object is constructed, two functions implementing equations (2.3.6) and (2.3.7) are available for solving linear equations. The first solves a single right-hand side vector $\mathbf{b}$ for a solution vector $\mathbf{x}$. The second simultaneously solves multiple right-hand vectors, arranged as the columns of a matrix $\mathbf{B}$. In other words, it calculates the matrix $\mathbf{A}^{-1} \cdot \mathbf{B}$.

```
void LUdcmp::solve(VecDoub_I &b, VecDoub_O &x)                    ludcmp.h
```

Solves the set of n linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ using the stored LU decomposition of $\mathbf{A}$. b[0..n-1] is input as the right-hand side vector $\mathbf{b}$, while x returns the solution vector $\mathbf{x}$; b and x may reference the same vector, in which case the solution overwrites the input. This routine takes into account the possibility that b will begin with many zero elements, so it is efficient for use in matrix inversion.

```
{
    Int i,ii=0,ip,j;
    Doub sum;
    if (b.size() != n || x.size() != n)
        throw("LUdcmp::solve bad sizes");
    for (i=0;i<n;i++) x[i] = b[i];
    for (i=0;i<n;i++) {          When ii is set to a positive value, it will become the
        ip=indx[i];                  index of the first nonvanishing element of b. We now
        sum=x[ip];                   do the forward substitution, equation (2.3.6). The
        x[ip]=x[i];                  only new wrinkle is to unscramble the permutation
        if (ii != 0)                 as we go.
            for (j=ii-1;j<i;j++) sum -= lu[i][j]*x[j];
        else if (sum != 0.0)     A nonzero element was encountered, so from now on we
            ii=i+1;                  will have to do the sums in the loop above.
        x[i]=sum;
    }
    for (i=n-1;i>=0;i--) {       Now we do the backsubstitution, equation (2.3.7).
        sum=x[i];
        for (j=i+1;j<n;j++) sum -= lu[i][j]*x[j];
        x[i]=sum/lu[i][i];       Store a component of the solution vector X.
    }                            All done!
}
```

```
void LUdcmp::solve(MatDoub_I &b, MatDoub_O &x)
```

Solves m sets of n linear equations $\mathbf{A} \cdot \mathbf{X} = \mathbf{B}$ using the stored LU decomposition of $\mathbf{A}$. The matrix b[0..n-1][0..m-1] inputs the right-hand sides, while x[0..n-1][0..m-1] returns the solution $\mathbf{A}^{-1} \cdot \mathbf{B}$. b and x may reference the same matrix, in which case the solution overwrites the input.

```
{
    int i,j,m=b.ncols();
    if (b.nrows() != n || x.nrows() != n || b.ncols() != x.ncols())
        throw("LUdcmp::solve bad sizes");
    VecDoub xx(n);
    for (j=0;j<m;j++) {              Copy and solve each column in turn.
        for (i=0;i<n;i++) xx[i] = b[i][j];
        solve(xx,xx);
        for (i=0;i<n;i++) x[i][j] = xx[i];
    }
}
```

The LU decomposition in `LUdcmp` requires about $\frac{1}{3}N^3$ executions of the inner loops (each with one multiply and one add). This is thus the operation count for solving one (or a few) right-hand sides, and is a factor of 3 better than the Gauss-Jordan routine `gaussj` that was given in §2.1, and a factor of 1.5 better than a Gauss-Jordan routine (not given) that does not compute the inverse matrix. For inverting a matrix, the total count (including the forward- and backsubstitution as discussed following equation 2.3.7 above) is $(\frac{1}{3} + \frac{1}{6} + \frac{1}{2})N^3 = N^3$, the same as `gaussj`.

To summarize, this is the preferred way to solve the linear set of equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$:

```
const Int n = ...
MatDoub a(n,n);
VecDoub b(n),x(n);
...
LUdcmp alu(a);
alu.solve(b,x);
```

The answer will be given back in `x`. Your original matrix `a` and vector `b` are not altered. If you need to recover the storage in the object `alu`, then start a temporary scope with "`{`" before `alu` is declared, and end that scope with "`}`" when you want `alu` to be destroyed.

If you subsequently want to solve a set of equations with the same $\mathbf{A}$ but a different right-hand side $\mathbf{b}$, you repeat *only*

```
alu.solve(b,x);
```

2.3.2 Inverse of a Matrix

`LUdcmp` has a member function that gives the inverse of the matrix $\mathbf{A}$. Simply, it creates an identity matrix and then invokes the appropriate `solve` method.

ludcmp.h

```
void LUdcmp::inverse(MatDoub_O &ainv)
Using the stored LU decomposition, return in ainv the matrix inverse A^-1.
{
    Int i,j;
    ainv.resize(n,n);
    for (i=0;i<n;i++) {
        for (j=0;j<n;j++) ainv[i][j] = 0.;
        ainv[i][i] = 1.;
    }
    solve(ainv,ainv);
}
```

The matrix `ainv` will now contain the inverse of the original matrix `a`. Alternatively, there is nothing wrong with using a Gauss-Jordan routine like `gaussj` (§2.1) to invert a matrix in place, destroying the original. Both methods have practically the same operations count.

2.3.3 Determinant of a Matrix

The determinant of an LU decomposed matrix is just the product of the diagonal elements,

$$\det = \prod_{j=0}^{N-1} \beta_{jj} \tag{2.3.15}$$

We don't, recall, compute the decomposition of the original matrix, but rather a decomposition of a rowwise permutation of it. Luckily, we have kept track of whether the number of row interchanges was even or odd, so we just preface the product by the corresponding sign. (You now finally know the purpose of `d` in the `LUdcmp` structure.)

ludcmp.h

```
Doub LUdcmp::det()
Using the stored LU decomposition, return the determinant of the matrix A.
{
    Doub dd = d;
    for (Int i=0;i<n;i++) dd *= lu[i][i];
    return dd;
}
```

For a matrix of any substantial size, it is quite likely that the determinant will overflow or underflow your computer's floating-point dynamic range. In such a case you can easily add another member function that, e.g., divides by powers of ten, to keep track of the scale separately, or, e.g., accumulates the sum of logarithms of the absolute values of the factors and the sign separately.

2.3.4 Complex Systems of Equations

If your matrix $\mathbf{A}$ is real, but the right-hand side vector is complex, say $\mathbf{b} + i\,\mathbf{d}$, then (i) LU decompose $\mathbf{A}$ in the usual way, (ii) backsubstitute $\mathbf{b}$ to get the real part of the solution vector, and (iii) backsubstitute $\mathbf{d}$ to get the imaginary part of the solution vector.

If the matrix itself is complex, so that you want to solve the system

$$(\mathbf{A} + i\,\mathbf{C}) \cdot (\mathbf{x} + i\,\mathbf{y}) = (\mathbf{b} + i\,\mathbf{d}) \tag{2.3.16}$$

then there are two possible ways to proceed. The best way is to rewrite `LUdcmp` with complex routines. Complex modulus substitutes for absolute value in the construction of the scaling vector `vv` and in the search for the largest pivot elements. Everything else goes through in the obvious way, with complex arithmetic used as needed.

A quick-and-dirty way to solve complex systems is to take the real and imaginary parts of (2.3.16), giving

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} - \mathbf{C} \cdot \mathbf{y} &= \mathbf{b} \\ \mathbf{C} \cdot \mathbf{x} + \mathbf{A} \cdot \mathbf{y} &= \mathbf{d} \end{aligned} \tag{2.3.17}$$

which can be written as a $2N \times 2N$ set of *real* equations,

$$\begin{pmatrix} \mathbf{A} & -\mathbf{C} \\ \mathbf{C} & \mathbf{A} \end{pmatrix} \cdot \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix} = \begin{pmatrix} \mathbf{b} \\ \mathbf{d} \end{pmatrix} \tag{2.3.18}$$

and then solved with `LUdcmp`'s routines in their present forms. This scheme is a factor of 2 inefficient in storage, since $\mathbf{A}$ and $\mathbf{C}$ are stored twice. It is also a factor of 2 inefficient in time, since the complex multiplies in a complexified version of the routines would each use 4 real multiplies, while the solution of a $2N \times 2N$ problem involves 8 times the work of an $N \times N$ one. If you can tolerate these factor-of-two inefficiencies, then equation (2.3.18) is an easy way to proceed.

CITED REFERENCES AND FURTHER READING:

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), Chapter 4.[1]

Anderson, E., et al. 1999, LAPACK User's Guide, 3rd ed. (Philadelphia: S.I.A.M.). Online with software at 2007+, `http://www.netlib.org/lapack`.

Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §3.3, and p. 50.

Forsythe, G.E., and Moler, C.B. 1967, *Computer Solution of Linear Algebraic Systems* (Englewood Cliffs, NJ: Prentice-Hall), Chapters 9, 16, and 18.

Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §4.1.

Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §9.11.

Horn, R.A., and Johnson, C.R. 1985, *Matrix Analysis* (Cambridge: Cambridge University Press).

2.4 Tridiagonal and Band-Diagonal Systems of Equations

The special case of a system of linear equations that is *tridiagonal*, that is, has nonzero elements only on the diagonal plus or minus one column, is one that occurs frequently. Also common are systems that are *band-diagonal*, with nonzero elements only along a few diagonal lines adjacent to the main diagonal (above and below).

For tridiagonal sets, the procedures of LU decomposition, forward- and back-substitution each take only $O(N)$ operations, and the whole solution can be encoded very concisely. The resulting routine `tridag` is one that we will use in later chapters.

Naturally, one does not reserve storage for the full $N \times N$ matrix, but only for the nonzero components, stored as three vectors. The set of equations to be solved is

$$\begin{bmatrix} b_0 & c_0 & 0 & \cdots & & & \\ a_1 & b_1 & c_1 & \cdots & & & \\ & & & \cdots & & & \\ & & & \cdots & a_{N-2} & b_{N-2} & c_{N-2} \\ & & & \cdots & 0 & a_{N-1} & b_{N-1} \end{bmatrix} \cdot \begin{bmatrix} u_0 \\ u_1 \\ \cdots \\ u_{N-2} \\ u_{N-1} \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ \cdots \\ r_{N-2} \\ r_{N-1} \end{bmatrix} \tag{2.4.1}$$

Notice that a_0 and c_{N-1} are undefined and are not referenced by the routine that follows.

tridag.h

```
void tridag(VecDoub_I &a, VecDoub_I &b, VecDoub_I &c, VecDoub_I &r, VecDoub_O &u)
```

Solves for a vector `u[0..n-1]` the tridiagonal linear set given by equation (2.4.1). `a[0..n-1]`, `b[0..n-1]`, `c[0..n-1]`, and `r[0..n-1]` are input vectors and are not modified.

```
{
    Int j,n=a.size();
    Doub bet;
    VecDoub gam(n);                    One vector of workspace, gam, is needed.
    if (b[0] == 0.0) throw("Error 1 in tridag");
    If this happens, then you should rewrite your equations as a set of order N − 1, with u1
    trivially eliminated.
    u[0]=r[0]/(bet=b[0]);
    for (j=1;j<n;j++) {                Decomposition and forward substitution.
        gam[j]=c[j-1]/bet;
        bet=b[j]-a[j]*gam[j];
```

```
        if (bet == 0.0) throw("Error 2 in tridag");      Algorithm fails; see below.
        u[j]=(r[j]-a[j]*u[j-1])/bet;
    }
    for (j=(n-2);j>=0;j--)
        u[j] -= gam[j+1]*u[j+1];                         Backsubstitution.
}
```

There is no pivoting in `tridag`. It is for this reason that `tridag` can fail even when the underlying matrix is nonsingular: A zero pivot can be encountered even for a nonsingular matrix. In practice, this is not something to lose sleep about. The kinds of problems that lead to tridiagonal linear sets usually have additional properties which guarantee that the algorithm in `tridag` will succeed. For example, if

$$|b_j| > |a_j| + |c_j| \qquad j = 0, \ldots, N-1 \tag{2.4.2}$$

(called *diagonal dominance*), then it can be shown that the algorithm cannot encounter a zero pivot.

It is possible to construct special examples in which the lack of pivoting in the algorithm causes numerical instability. In practice, however, such instability is almost never encountered — unlike the general matrix problem where pivoting is essential.

The tridiagonal algorithm is the rare case of an algorithm that, in practice, is more robust than theory says it should be. Of course, should you ever encounter a problem for which `tridag` fails, you can instead use the more general method for band-diagonal systems, described below (the `Bandec` object).

Some other matrix forms consisting of tridiagonal with a small number of additional elements (e.g., upper right and lower left corners) also allow rapid solution; see §2.7.

2.4.1 Parallel Solution of Tridiagonal Systems

It is possible to solve tridiagonal systems doing many of the operations in parallel. We illustrate by the special case with $N = 7$:

$$\begin{bmatrix} b_0 & c_0 & & & & & & \\ a_1 & b_1 & c_1 & & & & & \\ & a_2 & b_2 & c_2 & & & & \\ & & a_3 & b_3 & c_3 & & & \\ & & & a_4 & b_4 & c_4 & & \\ & & & & a_5 & b_5 & c_5 & \\ & & & & & a_6 & b_6 & \end{bmatrix} \cdot \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ r_2 \\ r_3 \\ r_4 \\ r_5 \\ r_6 \end{bmatrix} \tag{2.4.3}$$

The basic idea is to partition the problem into even and odd elements, recurse to solve the latter, and then solve the former in parallel. Specifically, we first rewrite equation (2.4.3) by permuting its rows and columns,

$$\begin{bmatrix} b_0 & & & & c_0 & & & \\ & b_2 & & & a_2 & c_2 & & \\ & & b_4 & & & a_4 & c_4 & \\ & & & b_6 & & & a_6 & \\ a_1 & c_1 & & & b_1 & & & \\ & a_3 & c_3 & & & b_3 & & \\ & & a_5 & c_5 & & & b_5 & \end{bmatrix} \cdot \begin{bmatrix} u_0 \\ u_2 \\ u_4 \\ u_6 \\ u_1 \\ u_3 \\ u_5 \end{bmatrix} = \begin{bmatrix} r_0 \\ r_2 \\ r_4 \\ r_6 \\ r_1 \\ r_3 \\ r_5 \end{bmatrix} \tag{2.4.4}$$

Now observe that, by row operations that subtract multiples of the first four rows from each of the last three rows, we can eliminate all nonzero elements in the lower-left quadrant. The price we pay is bringing some new elements into the lower-right quadrant, whose

nonzero elements we now call x's, y's, and z's. We call the modified right-hand sides q. The transformed problem is now

$$\begin{bmatrix} b_0 & & & & c_0 & & \\ & b_2 & & & a_2 & c_2 & \\ & & b_4 & & & a_4 & c_4 \\ & & & b_6 & & & a_6 \\ & & & & y_0 & z_0 & \\ & & & & x_1 & y_1 & z_1 \\ & & & & & x_2 & y_2 \end{bmatrix} \cdot \begin{bmatrix} u_0 \\ u_2 \\ u_4 \\ u_6 \\ u_1 \\ u_3 \\ u_5 \end{bmatrix} = \begin{bmatrix} r_0 \\ r_2 \\ r_4 \\ r_6 \\ q_0 \\ q_1 \\ q_2 \end{bmatrix} \tag{2.4.5}$$

Notice that the last three rows form a new, smaller, tridiagonal problem, which we can solve simply by recursing. Once its solution is known, the first four rows can be solved by a simple, parallelizable, substitution. For discussion of this and related methods for parallelizing tridiagonal systems, and references to the literature, see [2].

2.4.2 Band-Diagonal Systems

Where tridiagonal systems have nonzero elements only on the diagonal plus or minus one, band-diagonal systems are slightly more general and have (say) $m_1 \geq 0$ nonzero elements immediately to the left of (below) the diagonal and $m_2 \geq 0$ nonzero elements immediately to its right (above it). Of course, this is only a useful classification if m_1 and m_2 are both $\ll N$. In that case, the solution of the linear system by LU decomposition can be accomplished much faster, and in much less storage, than for the general $N \times N$ case.

The precise definition of a band-diagonal matrix with elements a_{ij} is that

$$a_{ij} = 0 \quad \text{when} \quad j > i + m_2 \quad \text{or} \quad i > j + m_1 \tag{2.4.6}$$

Band-diagonal matrices are stored and manipulated in a so-called compact form, which results if the matrix is tilted 45° clockwise, so that its nonzero elements lie in a long, narrow matrix with $m_1 + 1 + m_2$ columns and N rows. This is best illustrated by an example: The band-diagonal matrix

$$\begin{pmatrix} 3 & 1 & 0 & 0 & 0 & 0 & 0 \\ 4 & 1 & 5 & 0 & 0 & 0 & 0 \\ 9 & 2 & 6 & 5 & 0 & 0 & 0 \\ 0 & 3 & 5 & 8 & 9 & 0 & 0 \\ 0 & 0 & 7 & 9 & 3 & 2 & 0 \\ 0 & 0 & 0 & 3 & 8 & 4 & 6 \\ 0 & 0 & 0 & 0 & 2 & 4 & 4 \end{pmatrix} \tag{2.4.7}$$

which has $N = 7$, $m_1 = 2$, and $m_2 = 1$, is stored compactly as the 7×4 matrix,

$$\begin{pmatrix} x & x & 3 & 1 \\ x & 4 & 1 & 5 \\ 9 & 2 & 6 & 5 \\ 3 & 5 & 8 & 9 \\ 7 & 9 & 3 & 2 \\ 3 & 8 & 4 & 6 \\ 2 & 4 & 4 & x \end{pmatrix} \tag{2.4.8}$$

Here x denotes elements that are wasted space in the compact format; these will not be referenced by any manipulations and can have arbitrary values. Notice that the diagonal of the original matrix appears in column m_1, with subdiagonal elements to its left and superdiagonal elements to its right.

The simplest manipulation of a band-diagonal matrix, stored compactly, is to multiply it by a vector to its right. Although this is algorithmically trivial, you might want to study the following routine as an example of how to pull nonzero elements a_{ij} out of the compact storage format in an orderly fashion.

```
void banmul(MatDoub_I &a, const Int m1, const Int m2, VecDoub_I &x,
    VecDoub_O &b)
```
banded.h

Matrix multiply $\mathbf{b} = \mathbf{A} \cdot \mathbf{x}$, where $\mathbf{A}$ is band-diagonal with `m1` rows below the diagonal and `m2` rows above. The input vector is `x[0..n-1]` and the output vector is `b[0..n-1]`. The array `a[0..n-1][0..m1+m2]` stores $\mathbf{A}$ as follows: The diagonal elements are in `a[0..n-1][m1]`. Subdiagonal elements are in `a[`j`..n-1][0..m1-1]` with $j > 0$ appropriate to the number of elements on each subdiagonal. Superdiagonal elements are in `a[0..`j`][m1+1..m1+m2]` with $j <$ `n-1` appropriate to the number of elements on each superdiagonal.

```
{
    Int i,j,k,tmploop,n=a.nrows();
    for (i=0;i<n;i++) {
        k=i-m1;
        tmploop=MIN(m1+m2+1,Int(n-k));
        b[i]=0.0;
        for (j=MAX(0,-k);j<tmploop;j++) b[i] += a[i][j]*x[j+k];
    }
}
```

It is not possible to store the LU decomposition of a band-diagonal matrix $\mathbf{A}$ quite as compactly as the compact form of $\mathbf{A}$ itself. The decomposition (essentially by Crout's method; see §2.3) produces additional nonzero "fill-ins." One straightforward storage scheme is to store the upper triangular factor (U) in a space with the same shape as $\mathbf{A}$, and to store the lower triangular factor (L) in a separate compact matrix of size $N \times m_1$. The diagonal elements of U (whose product, times `d` $= \pm 1$, gives the determinant) are in the first column of U.

Here is an object, analogous to `LUdcmp` in §2.3, for solving band-diagonal linear equations:

```
struct Bandec {
```
banded.h

Object for solving linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ for a band-diagonal matrix $\mathbf{A}$, using LU decomposition.

```
    Int n,m1,m2;
    MatDoub au,al;                Upper and lower triangular matrices, stored compactly.
    VecInt indx;
    Doub d;
    Bandec(MatDoub_I &a, const int mm1, const int mm2); Constructor.
    void solve(VecDoub_I &b, VecDoub_O &x);       Solve a right-hand side vector.
    Doub det();                                   Return determinant of A.
};
```

The constructor takes as arguments the compactly stored matrix $\mathbf{A}$, and the integers m_1 and m_2. (One could of course define a "band-diagonal matrix object" to encapsulate these quantities, but in this brief treatment we want to keep things simple.)

```
Bandec::Bandec(MatDoub_I &a, const Int mm1, const Int mm2)
    : n(a.nrows()), au(a), m1(mm1), m2(mm2), al(n,m1), indx(n)
```
banded.h

Constructor. Given an `n`×`n` band-diagonal matrix $\mathbf{A}$ with `m1` subdiagonal rows and `m2` superdiagonal rows, compactly stored in the array `a[0..n-1][0..m1+m2]` as described in the comment for routine `banmul`, an LU decomposition of a rowwise permutation of $\mathbf{A}$ is constructed. The upper and lower triangular matrices are stored in `au` and `al`, respectively. The stored vector `indx[0..n-1]` records the row permutation effected by the partial pivoting; `d` is ± 1 depending on whether the number of row interchanges was even or odd, respectively.

```
{
    const Doub TINY=1.0e-40;
    Int i,j,k,l,mm;
    Doub dum;
    mm=m1+m2+1;
    l=m1;
    for (i=0;i<m1;i++) {                  Rearrange the storage a bit.
        for (j=m1-i;j<mm;j++) au[i][j-l]=au[i][j];
        l--;
        for (j=mm-l-1;j<mm;j++) au[i][j]=0.0;
    }
```

```
    d=1.0;
    l=m1;
    for (k=0;k<n;k++) {                    For each row...
        dum=au[k][0];
        i=k;
        if (l<n) l++;
        for (j=k+1;j<l;j++) {              Find the pivot element.
            if (abs(au[j][0]) > abs(dum)) {
                dum=au[j][0];
                i=j;
            }
        }
        indx[k]=i+1;
        if (dum == 0.0) au[k][0]=TINY;
        Matrix is algorithmically singular, but proceed anyway with TINY pivot (desirable in
        some applications).
        if (i != k) {                      Interchange rows.
            d = -d;
            for (j=0;j<mm;j++) SWAP(au[k][j],au[i][j]);
        }
        for (i=k+1;i<l;i++) {              Do the elimination.
            dum=au[i][0]/au[k][0];
            al[k][i-k-1]=dum;
            for (j=1;j<mm;j++) au[i][j-1]=au[i][j]-dum*au[k][j];
            au[i][mm-1]=0.0;
        }
    }
}
```

Some pivoting is possible within the storage limitations of `bandec`, and the above routine does take advantage of the opportunity. In general, when `TINY` is returned as a diagonal element of U, then the original matrix (perhaps as modified by roundoff error) is in fact singular. In this regard, `bandec` is somewhat more robust than `tridag` above, which can fail algorithmically even for nonsingular matrices; `bandec` is thus also useful (with $m_1 = m_2 = 1$) for some ill-behaved tridiagonal systems.

Once the matrix $\mathbf{A}$ has been decomposed, any number of right-hand sides can be solved in turn by repeated calls to the `solve` method, the forward- and backsubstitution routine analogous to its same-named cousin in §2.3.

banded.h

```
void Bandec::solve(VecDoub_I &b, VecDoub_O &x)
```

Given a right-hand side vector `b[0..n-1]`, solves the band-diagonal linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. The solution vector $\mathbf{x}$ is returned as `x[0..n-1]`.

```
{
    Int i,j,k,l,mm;
    Doub dum;
    mm=m1+m2+1;
    l=m1;
    for (k=0;k<n;k++) x[k] = b[k];
    for (k=0;k<n;k++) {                    Forward substitution, unscrambling the permuted rows
        j=indx[k]-1;                           as we go.
        if (j!=k) SWAP(x[k],x[j]);
        if (l<n) l++;
        for (j=k+1;j<l;j++) x[j] -= al[k][j-k-1]*x[k];
    }
    l=1;
    for (i=n-1;i>=0;i--) {                 Backsubstitution.
        dum=x[i];
        for (k=1;k<l;k++) dum -= au[i][k]*x[k+i];
        x[i]=dum/au[i][0];
        if (l<mm) l++;
    }
}
```

And, finally, a method for getting the determinant:

```
Doub Bandec::det() {                                                    banded.h
Using the stored decomposition, return the determinant of the matrix A.
    Doub dd = d;
    for (int i=0;i<n;i++) dd *= au[i][0];
    return dd;
}
```

The routines in `Bandec` are based on the Handbook routines *bandet1* and *bansol1* in [1].

CITED REFERENCES AND FURTHER READING:

Keller, H.B. 1968, *Numerical Methods for Two-Point Boundary-Value Problems*; reprinted 1991 (New York: Dover), p. 74.

Dahlquist, G., and Bjorck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), Example 5.4.3, p. 166.

Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §9.11.

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I/6.[1]

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §4.3.

Hockney, R.W., and Jesshope, C.R. 1988, *Parallel Computers 2: Architecture, Programming, and Algorithms* (Bristol and Philadelphia: Adam Hilger), §5.4.[2]

2.5 Iterative Improvement of a Solution to Linear Equations

Obviously it is not easy to obtain greater precision for the solution of a linear set than the precision of your computer's floating-point word. Unfortunately, for large sets of linear equations, it is not always easy to obtain precision equal to, or even comparable to, the computer's limit. In direct methods of solution, roundoff errors accumulate, and they are magnified to the extent that your matrix is close to singular. You can easily lose two or three significant figures for matrices that (you thought) were *far* from singular.

If this happens to you, there is a neat trick to restore the full machine precision, called *iterative improvement* of the solution. The theory is straightforward (see Figure 2.5.1): Suppose that a vector $\mathbf{x}$ is the exact solution of the linear set

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \tag{2.5.1}$$

You don't, however, know $\mathbf{x}$. You only know some slightly wrong solution $\mathbf{x} + \delta\mathbf{x}$, where $\delta\mathbf{x}$ is the unknown error. When multiplied by the matrix $\mathbf{A}$, your slightly wrong solution gives a product slightly discrepant from the desired right-hand side $\mathbf{b}$, namely

$$\mathbf{A} \cdot (\mathbf{x} + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b} \tag{2.5.2}$$

Subtracting (2.5.1) from (2.5.2) gives

$$\mathbf{A} \cdot \delta\mathbf{x} = \delta\mathbf{b} \tag{2.5.3}$$

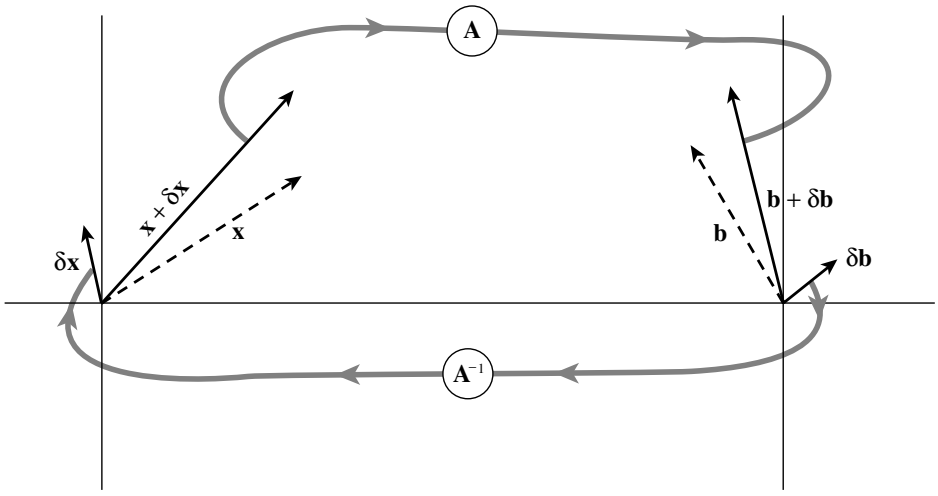


Figure 2.5.1. Iterative improvement of the solution to $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. The first guess $\mathbf{x} + \delta\mathbf{x}$ is multiplied by $\mathbf{A}$ to produce $\mathbf{b} + \delta\mathbf{b}$. The known vector $\mathbf{b}$ is subtracted, giving $\delta\mathbf{b}$. The linear set with this right-hand side is inverted, giving $\delta\mathbf{x}$. This is subtracted from the first guess giving an improved solution $\mathbf{x}$.

But (2.5.2) can also be solved, trivially, for $\delta\mathbf{b}$. Substituting this into (2.5.3) gives

$$\mathbf{A} \cdot \delta\mathbf{x} = \mathbf{A} \cdot (\mathbf{x} + \delta\mathbf{x}) - \mathbf{b} \tag{2.5.4}$$

In this equation, the whole right-hand side is known, since $\mathbf{x} + \delta\mathbf{x}$ is the wrong solution that you want to improve. It is good to calculate the right-hand side in higher precision than the original solution, if you can, since there will be a lot of cancellation in the subtraction of $\mathbf{b}$. Then, we need only solve (2.5.4) for the error $\delta\mathbf{x}$, and then subtract this from the wrong solution to get an improved solution.

An important extra benefit occurs if we obtained the original solution by LU decomposition. In this case we already have the LU decomposed form of $\mathbf{A}$, and all we need do to solve (2.5.4) is compute the right-hand side, forward- and backsubstitute.

Because so much of the necessary machinery is already in `LUdcmp`, we implement iterative improvement as a member function of that class. Since iterative improvement requires the matrix $\mathbf{A}$ (as well as its LU decomposition), we have, with foresight, caused `LUdcmp` to save a reference to the matrix `a` from which it was constructed. If you plan to use iterative improvement, you must not modify `a` or let it go out of scope. (No other method in `LUdcmp` makes use of this reference to `a`.)

ludcmp.h

```
void LUdcmp::mprove(VecDoub_I &b, VecDoub_IO &x)
```

Improves a solution vector `x[0..n-1]` of the linear set of equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. The vectors `b[0..n-1]` and `x[0..n-1]` are input. On output, `x[0..n-1]` is modified, to an improved set of values.

```
{
    Int i,j;
    VecDoub r(n);
    for (i=0;i<n;i++) {
        Ldoub sdp = -b[i];
        for (j=0;j<n;j++)
            sdp += (Ldoub)aref[i][j] * (Ldoub)x[j];
```

Calculate the right-hand side, accumulating the residual in higher precision.

```
        r[i]=sdp;
    }
    solve(r,r);                          Solve for the error term,
    for (i=0;i<n;i++) x[i] -= r[i];      and subtract it from the old solution.
}
```

Iterative improvement is *highly* recommended: It is a process of order only N^2 operations (multiply vector by matrix, forward- and backsubstitute — see discussion following equation 2.3.7); it never hurts; and it can really give you your money's worth if it saves an otherwise ruined solution on which you have already spent of order N^3 operations.

You can call `mprove` several times in succession if you want. Unless you are starting quite far from the true solution, one call is generally enough; but a second call to verify convergence can be reassuring.

If you cannot compute the right-hand side in equation (2.5.4) in higher precision, iterative refinement will still often improve the quality of a solution, although not in all cases as much as if higher precision is available. Many textbooks assert the contrary, but you will find the proof in [1].

2.5.1 More on Iterative Improvement

It is illuminating (and will be useful later in the book) to give a somewhat more solid analytical foundation for equation (2.5.4), and also to give some additional results. Implicit in the previous discussion was the notion that the solution vector $\mathbf{x} + \delta\mathbf{x}$ has an error term; but we neglected the fact that the LU decomposition of $\mathbf{A}$ is itself not exact.

A different analytical approach starts with some matrix $\mathbf{B}_0$ that is assumed to be an *approximate* inverse of the matrix $\mathbf{A}$, so that $\mathbf{B}_0 \cdot \mathbf{A}$ is approximately the identity matrix $\mathbf{1}$. Define the *residual matrix* $\mathbf{R}$ of $\mathbf{B}_0$ as

$$\mathbf{R} \equiv \mathbf{1} - \mathbf{B}_0 \cdot \mathbf{A} \tag{2.5.5}$$

which is supposed to be "small" (we will be more precise below). Note that therefore

$$\mathbf{B}_0 \cdot \mathbf{A} = \mathbf{1} - \mathbf{R} \tag{2.5.6}$$

Next consider the following formal manipulation:

$$\begin{aligned} \mathbf{A}^{-1} &= \mathbf{A}^{-1} \cdot (\mathbf{B}_0^{-1} \cdot \mathbf{B}_0) = (\mathbf{A}^{-1} \cdot \mathbf{B}_0^{-1}) \cdot \mathbf{B}_0 = (\mathbf{B}_0 \cdot \mathbf{A})^{-1} \cdot \mathbf{B}_0 \\ &= (\mathbf{1} - \mathbf{R})^{-1} \cdot \mathbf{B}_0 = (\mathbf{1} + \mathbf{R} + \mathbf{R}^2 + \mathbf{R}^3 + \cdots) \cdot \mathbf{B}_0 \end{aligned} \tag{2.5.7}$$

We can define the nth partial sum of the last expression by

$$\mathbf{B}_n \equiv (\mathbf{1} + \mathbf{R} + \cdots + \mathbf{R}^n) \cdot \mathbf{B}_0 \tag{2.5.8}$$

so that $\mathbf{B}_\infty \to \mathbf{A}^{-1}$, if the limit exists.

It now is straightforward to verify that equation (2.5.8) satisfies some interesting recurrence relations. As regards solving $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$, where $\mathbf{x}$ and $\mathbf{b}$ are vectors, define

$$\mathbf{x}_n \equiv \mathbf{B}_n \cdot \mathbf{b} \tag{2.5.9}$$

Then it is easy to show that

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{B}_0 \cdot (\mathbf{b} - \mathbf{A} \cdot \mathbf{x}_n) \tag{2.5.10}$$

This is immediately recognizable as equation (2.5.4), with $-\delta\mathbf{x} = \mathbf{x}_{n+1} - \mathbf{x}_n$, and with $\mathbf{B}_0$ taking the role of $\mathbf{A}^{-1}$. We see, therefore, that equation (2.5.4) does not require that the LU decomposition of $\mathbf{A}$ be exact, but only that the implied residual $\mathbf{R}$ be small. In rough terms,

if the residual is smaller than the square root of your computer's roundoff error, then after one application of equation (2.5.10) (that is, going from $\mathbf{x}_0 \equiv \mathbf{B}_0 \cdot \mathbf{b}$ to $\mathbf{x}_1$) the first neglected term, of order $\mathbf{R}^2$, will be smaller than the roundoff error. Equation (2.5.10), like equation (2.5.4), moreover, can be applied more than once, since it uses only $\mathbf{B}_0$, and not any of the higher $\mathbf{B}$'s.

A much more surprising recurrence that follows from equation (2.5.8) is one that more than *doubles* the order n at each stage:

$$\mathbf{B}_{2n+1} = 2\mathbf{B}_n - \mathbf{B}_n \cdot \mathbf{A} \cdot \mathbf{B}_n \qquad n = 0, 1, 3, 7, \ldots \tag{2.5.11}$$

Repeated application of equation (2.5.11), from a suitable starting matrix $\mathbf{B}_0$, converges *quadratically* to the unknown inverse matrix $\mathbf{A}^{-1}$ (see §9.4 for the definition of "quadratically"). Equation (2.5.11) goes by various names, including *Schultz's Method* and *Hotelling's Method*; see Pan and Reif [2] for references. In fact, equation (2.5.11) is simply the iterative Newton-Raphson method of root finding (§9.4) applied to matrix inversion.

Before you get too excited about equation (2.5.11), however, you should notice that it involves two full matrix multiplications at each iteration. Each matrix multiplication involves N^3 adds and multiplies. But we already saw in §2.1 – §2.3 that direct inversion of $\mathbf{A}$ requires only N^3 adds and N^3 multiplies *in toto*. Equation (2.5.11) is therefore practical only when special circumstances allow it to be evaluated much more rapidly than is the case for general matrices. We will meet such circumstances later, in §13.10.

In the spirit of delayed gratification, let us nevertheless pursue the two related issues: When does the series in equation (2.5.7) converge; and what is a suitable initial guess $\mathbf{B}_0$ (if, for example, an initial LU decomposition is not feasible)?

We can define the norm of a matrix as the largest amplification of length that it is able to induce on a vector,

$$\|\mathbf{R}\| \equiv \max_{\mathbf{v}\neq\mathbf{0}} \frac{|\mathbf{R}\cdot\mathbf{v}|}{|\mathbf{v}|} \tag{2.5.12}$$

If we let equation (2.5.7) act on some arbitrary right-hand side $\mathbf{b}$, as one wants a matrix inverse to do, it is obvious that a sufficient condition for convergence is

$$\|\mathbf{R}\| < 1 \tag{2.5.13}$$

Pan and Reif [2] point out that a suitable initial guess for $\mathbf{B}_0$ is any sufficiently small constant ϵ times the matrix transpose of $\mathbf{A}$, that is,

$$\mathbf{B}_0 = \epsilon\mathbf{A}^T \qquad \text{or} \qquad \mathbf{R} = \mathbf{1} - \epsilon\mathbf{A}^T \cdot \mathbf{A} \tag{2.5.14}$$

To see why this is so involves concepts from Chapter 11; we give here only the briefest sketch: $\mathbf{A}^T \cdot \mathbf{A}$ is a symmetric, positive-definite matrix, so it has real, positive eigenvalues. In its diagonal representation, $\mathbf{R}$ takes the form

$$\mathbf{R} = \mathrm{diag}(1-\epsilon\lambda_0, 1-\epsilon\lambda_1, \ldots, 1-\epsilon\lambda_{N-1}) \tag{2.5.15}$$

where all the λ_i's are positive. Evidently any ϵ satisfying $0 < \epsilon < 2/(\max_i \lambda_i)$ will give $\|\mathbf{R}\| < 1$. It is not difficult to show that the optimal choice for ϵ, giving the most rapid convergence for equation (2.5.11), is

$$\epsilon = 2/(\max_i \lambda_i + \min_i \lambda_i) \tag{2.5.16}$$

Rarely does one know the eigenvalues of $\mathbf{A}^T \cdot \mathbf{A}$ in equation (2.5.16). Pan and Reif derive several interesting bounds, which are computable directly from $\mathbf{A}$. The following choices guarantee the convergence of $\mathbf{B}_n$ as $n \to \infty$:

$$\epsilon \le 1 \Big/ \sum_{j,k} a_{jk}^2 \qquad \text{or} \qquad \epsilon \le 1 \Big/ \Big(\max_i \sum_j |a_{ij}| \times \max_j \sum_i |a_{ij}| \Big) \tag{2.5.17}$$

The latter expression is truly a remarkable formula, which Pan and Reif derive by noting that the vector norm in equation (2.5.12) need not be the usual L_2 norm, but can instead be either the L_∞ (max) norm, or the L_1 (absolute value) norm. See their work for details.

Another approach, with which we have had some success, is to estimate the largest eigenvalue statistically, by calculating $s_i \equiv |\mathbf{A}\cdot\mathbf{v}_i|^2$ for several unit vector $\mathbf{v}_i$'s with randomly chosen directions in N-space. The largest eigenvalue λ can then be bounded by the maximum of $2\max s_i$ and $2N\,\mathrm{Var}(s_i)/\mu(s_i)$, where Var and μ denote the sample variance and mean, respectively.

CITED REFERENCES AND FURTHER READING:

Johnson, L.W., and Riess, R.D. 1982, *Numerical Analysis*, 2nd ed. (Reading, MA: Addison-Wesley), §2.3.4, p. 55.

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §3.5.3.

Dahlquist, G., and Bjorck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §5.5.6, p. 183.

Forsythe, G.E., and Moler, C.B. 1967, *Computer Solution of Linear Algebraic Systems* (Englewood Cliffs, NJ: Prentice-Hall), Chapter 13.

Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §9.5, p. 437.

Higham, N.J. 1997, "Iterative Refinement for Linear Systems and LAPACK," *IMA Journal of Numerical Analysis*, vol. 17, pp. 495–509.[1]

Pan, V., and Reif, J. 1985, "Efficient Parallel Solution of Linear Systems," in *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing* (New York: Association for Computing Machinery).[2]

2.6 Singular Value Decomposition

There exists a very powerful set of techniques for dealing with sets of equations or matrices that are either singular or else numerically very close to singular. In many cases where Gaussian elimination and LU decomposition fail to give satisfactory results, this set of techniques, known as *singular value decomposition*, or *SVD*, will diagnose for you precisely what the problem is. In some cases, SVD will not only diagnose the problem, it will also solve it, in the sense of giving you a useful numerical answer, although, as we shall see, not necessarily "the" answer that you thought you should get.

SVD is also the method of choice for solving most *linear least-squares* problems. We will outline the relevant theory in this section, but defer detailed discussion of the use of SVD in this application to Chapter 15, whose subject is the parametric modeling of data.

SVD methods are based on the following theorem of linear algebra, whose proof is beyond our scope: Any $M \times N$ matrix $\mathbf{A}$ can be written as the product of an $M \times N$ column-orthogonal matrix $\mathbf{U}$, an $N \times N$ diagonal matrix $\mathbf{W}$ with positive or zero elements (the *singular values*), and the transpose of an $N \times N$ orthogonal matrix $\mathbf{V}$. The various shapes of these matrices are clearer when shown as tableaus. If $M > N$ (which corresponds to the *overdetermined* situation of more equations than

unknowns), the decomposition looks like this:

$$\begin{pmatrix} & & \\ & \mathbf{A} & \\ & & \end{pmatrix} = \begin{pmatrix} & & \\ & \mathbf{U} & \\ & & \end{pmatrix} \cdot \begin{pmatrix} w_0 & & & \\ & w_1 & & \\ & & \cdots & \\ & & \cdots & \\ & & & w_{N-1} \end{pmatrix} \cdot \begin{pmatrix} & & \\ & \mathbf{V}^T & \\ & & \end{pmatrix} \tag{2.6.1}$$

If $M < N$ (the *undetermined* situation of fewer equations than unknowns), it looks like this:

$$\begin{pmatrix} & & \\ & \mathbf{A} & \\ & & \end{pmatrix} = \begin{pmatrix} & & \\ & \mathbf{U} & \\ & & \end{pmatrix} \cdot \begin{pmatrix} w_0 & & & \\ & w_1 & & \\ & & \cdots & \\ & & \cdots & \\ & & & w_{N-1} \end{pmatrix} \cdot \begin{pmatrix} & & \\ & \mathbf{V}^T & \\ & & \end{pmatrix} \tag{2.6.2}$$

The matrix $\mathbf{V}$ is orthogonal in the sense that its columns are orthonormal,

$$\sum_{j=0}^{N-1} V_{jk} V_{jn} = \delta_{kn} \qquad \begin{matrix} 0 \le k \le N-1 \\ 0 \le n \le N-1 \end{matrix} \tag{2.6.3}$$

that is, $\mathbf{V}^T \cdot \mathbf{V} = \mathbf{1}$. Since $\mathbf{V}$ is square, it is also row-orthonormal, $\mathbf{V} \cdot \mathbf{V}^T = 1$. When $M \ge N$, the matrix $\mathbf{U}$ is also column-orthogonal,

$$\sum_{i=0}^{M-1} U_{ik} U_{in} = \delta_{kn} \qquad \begin{matrix} 0 \le k \le N-1 \\ 0 \le n \le N-1 \end{matrix} \tag{2.6.4}$$

that is, $\mathbf{U}^T \cdot \mathbf{U} = \mathbf{1}$. In the case $M < N$, however, two things happen: (i) The singular values w_j for $j = M, \ldots, N-1$ are all zero, and (ii) the corresponding columns of $\mathbf{U}$ are also zero. Equation (2.6.4) then holds only for $k, n \le M-1$.

The decomposition (2.6.1) or (2.6.2) can always be done, no matter how singular the matrix is, and it is "almost" unique. That is to say, it is unique up to (i) making the same permutation of the columns of $\mathbf{U}$, elements of $\mathbf{W}$, and columns of $\mathbf{V}$ (or rows of $\mathbf{V}^T$); or (ii) performing an orthogonal rotation on any set of columns of $\mathbf{U}$ and $\mathbf{V}$ whose corresponding elements of $\mathbf{W}$ happen to be exactly equal. (A special case is multiplying any column of $\mathbf{U}$, and the corresponding column of $\mathbf{V}$ by -1.) A consequence of the permutation freedom is that for the case $M < N$, a numerical algorithm for the decomposition need not return zero w_j's in the canonical positions $j = M, \ldots, N-1$; the $N - M$ zero singular values can be scattered among all positions $j = 0, 1, \ldots, N-1$, and one needs to perform a sort to get the canonical order. In any case, it is conventional to sort *all* the singular values into descending order.

A Webnote [1] gives the details of the routine that actually performs SVD on an arbitrary matrix $\mathbf{A}$, yielding $\mathbf{U}$, $\mathbf{W}$, and $\mathbf{V}$. The routine is based on a routine

by Forsythe et al. [2], which is in turn based on the original routine of Golub and Reinsch, found, in various forms, in [4-6] and elsewhere. These references include extensive discussion of the algorithm used. As much as we dislike the use of black-box routines, we need to ask you to accept this one, since it would take us too far afield to cover its necessary background material here. The algorithm is very stable, and it is very unusual for it ever to misbehave. Most of the concepts that enter the algorithm (Householder reduction to bidiagonal form, diagonalization by QR procedure with shifts) will be discussed further in Chapter 11.

As we did for LU decomposition, we encapsulate the singular value decomposition and also the methods that depend on it into an object, `SVD`. We give its declaration here. The rest of this section will give the details on how to use it.

svd.h

```
struct SVD {
```
Object for singular value decomposition of a matrix $\mathbf{A}$, and related functions.
```
    Int m,n;
    MatDoub u,v;                                  The matrices U and V.
    VecDoub w;                                    The diagonal matrix W.
    Doub eps, tsh;
    SVD(MatDoub_I &a) : m(a.nrows()), n(a.ncols()), u(a), v(n,n), w(n) {
```
    Constructor. The single argument is $\mathbf{A}$. The SVD computation is done by `decompose`, and the results are sorted by `reorder`.
```
        eps = numeric_limits<Doub>::epsilon();
        decompose();
        reorder();
        tsh = 0.5*sqrt(m+n+1.)*w[0]*eps;          Default threshold for nonzero singular values.
    }

    void solve(VecDoub_I &b, VecDoub_O &x, Doub thresh);
    void solve(MatDoub_I &b, MatDoub_O &x, Doub thresh);
```
    Solve with (apply the pseudoinverse to) one or more right-hand sides.
```
    Int rank(Doub thresh);                        Quantities associated with the range and
    Int nullity(Doub thresh);                         nullspace of A.
    MatDoub range(Doub thresh);
    MatDoub nullspace(Doub thresh);

    Doub inv_condition() {                        Return reciprocal of the condition num-
        return (w[0] <= 0. || w[n-1] <= 0.) ? 0. : w[n-1]/w[0];     ber of A.
    }

    void decompose();                             Functions used by the constructor.
    void reorder();
    Doub pythag(const Doub a, const Doub b);
};
```

2.6.1 Range, Nullspace, and All That

Consider the familiar set of simultaneous equations

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \tag{2.6.5}$$

where $\mathbf{A}$ is an $M \times N$ matrix, and $\mathbf{x}$ and $\mathbf{b}$ are vectors of dimension N and M respectively. Equation (2.6.5) defines $\mathbf{A}$ as a linear mapping from an N-dimensional vector space to (generally) an M-dimensional one. But the map *might* be able to reach only a lesser-dimensional subspace of the full M-dimensional one. That subspace is called the *range* of $\mathbf{A}$. The dimension of the range is called the *rank* of $\mathbf{A}$. The rank

of **A** is equal to its number of linearly independent columns, and also (perhaps less obviously) to its number of linearly independent rows. If **A** is not identically zero, its rank is at least 1, and at most $\min(M, N)$.

Sometimes there are nonzero vectors **x** that are mapped to zero by **A**, that is, $\mathbf{A} \cdot \mathbf{x} = 0$. The space of such vectors (a subspace of the N-dimensional space that **x** lives in) is called the *nullspace* of **A**, and its dimension is called **A**'s *nullity*. The nullity can have any value from zero to N. The *rank-nullity theorem* states that, for any **A**, the rank plus the nullity is N, the number of columns.

An important special case is $M = N$, so the **A** is square, $N \times N$. If the rank of **A** is N, its maximum possible value, then **A** is nonsingular and invertible: $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ has a unique solution for any **b**, and only the zero vector is mapped to zero. This is a case where LU decomposition (§2.3) is the preferred solution method for **x**. However, if **A** has rank less than N (i.e., has nullity greater than zero), then two things happen: (i) most right-hand side vectors **b** yield no solution, but (ii) some have multiple solutions (in fact a whole subspace of them). We consider this situation further, below.

What has all this to do with singular value decomposition? SVD explicitly constructs orthonormal bases for the nullspace and range of a matrix! Specifically, the columns of **U** whose same-numbered elements w_j are *nonzero* are an orthonormal set of basis vectors that span the range; the columns of **V** whose same-numbered elements w_j are *zero* are an orthonormal basis for the nullspace. Our SVD object has methods that return the rank or nullity (integers), and also the range and nullspace, each of these packaged as a matrix whose columns form an orthonormal basis for the respective subspace.

svd.h

```
Int SVD::rank(Doub thresh = -1.) {
```

Return the rank of **A**, after zeroing any singular values smaller than `thresh`. If `thresh` is negative, a default value based on estimated roundoff is used.

```
    Int j,nr=0;
    tsh = (thresh >= 0. ? thresh : 0.5*sqrt(m+n+1.)*w[0]*eps);
    for (j=0;j<n;j++) if (w[j] > tsh) nr++;
    return nr;
}

Int SVD::nullity(Doub thresh = -1.) {
```

Return the nullity of **A**, after zeroing any singular values smaller than `thresh`. Default value as above.

```
    Int j,nn=0;
    tsh = (thresh >= 0. ? thresh : 0.5*sqrt(m+n+1.)*w[0]*eps);
    for (j=0;j<n;j++) if (w[j] <= tsh) nn++;
    return nn;
}

MatDoub SVD::range(Doub thresh = -1.){
```

Give an orthonormal basis for the range of **A** as the columns of a returned matrix. `thresh` as above.

```
    Int i,j,nr=0;
    MatDoub rnge(m,rank(thresh));
    for (j=0;j<n;j++) {
        if (w[j] > tsh) {
            for (i=0;i<m;i++) rnge[i][nr] = u[i][j];
            nr++;
        }
    }
    return rnge;
}
```

```
MatDoub SVD::nullspace(Doub thresh = -1.){
```
Give an orthonormal basis for the nullspace of **A** as the columns of a returned matrix. `thresh` as above.
```
    Int j,jj,nn=0;
    MatDoub nullsp(n,nullity(thresh));
    for (j=0;j<n;j++) {
        if (w[j] <= tsh) {
            for (jj=0;jj<n;jj++) nullsp[jj][nn] = v[jj][j];
            nn++;
        }
    }
    return nullsp;
}
```

The meaning of the optional parameter `thresh` is discussed below.

2.6.2 SVD of a Square Matrix

We return to the case of a square $N \times N$ matrix **A**. **U**, **V**, and **W** are also square matrices of the same size. Their inverses are also trivial to compute: **U** and **V** are orthogonal, so their inverses are equal to their transposes; **W** is diagonal, so its inverse is the diagonal matrix whose elements are the reciprocals of the elements w_j. From (2.6.1) it now follows immediately that the inverse of **A** is

$$\mathbf{A}^{-1} = \mathbf{V} \cdot \left[\text{diag}\,(1/w_j)\right] \cdot \mathbf{U}^T \tag{2.6.6}$$

The only thing that can go wrong with this construction is for one of the w_j's to be zero, or (numerically) for it to be so small that its value is dominated by roundoff error and therefore unknowable. If more than one of the w_j's has this problem, then the matrix is even more singular. So, first of all, SVD gives you a clear diagnosis of the situation.

Formally, the *condition number* of a matrix is defined as the ratio of the largest (in magnitude) of the w_j's to the smallest of the w_j's. A matrix is singular if its condition number is infinite, and it is *ill-conditioned* if its condition number is too large, that is, if its reciprocal approaches the machine's floating-point precision (for example, less than about 10^{-15} for values of type `double`). A function returning the condition number (or, rather, its reciprocal, to avoid overflow) is implemented in `SVD`.

Now let's have another look at solving the set of simultaneous linear equations (2.6.5) in the case that **A** is singular. We already saw that the set of *homogeneous* equations, where $\mathbf{b} = 0$, is solved immediately by SVD. The solution is any linear combination of the columns returned by the `nullspace` method above.

When the vector **b** on the right-hand side is not zero, the important question is whether it lies in the range of **A** or not. If it does, then the singular set of equations *does* have a solution **x**; in fact it has more than one solution, since any vector in the nullspace (any column of **V** with a corresponding zero w_j) can be added to **x** in any linear combination.

If we want to single out one particular member of this solution set of vectors as a representative, we might want to pick the one with the smallest length $|\mathbf{x}|^2$. Here is how to find that vector using SVD: Simply *replace* $1/w_j$ *by zero if* $w_j = 0$. (It is not very often that one gets to set $\infty = 0$!) Then compute, working from right to

left,

$$\mathbf{x} = \mathbf{V} \cdot \left[\text{diag}\,(1/w_j)\right] \cdot (\mathbf{U}^T \cdot \mathbf{b}) \tag{2.6.7}$$

This will be the solution vector of smallest length; the columns of $\mathbf{V}$ that are in the nullspace complete the specification of the solution set.

Proof: Consider $|\mathbf{x} + \mathbf{x}'|$, where $\mathbf{x}'$ lies in the nullspace. Then, if $\mathbf{W}^{-1}$ denotes the modified inverse of $\mathbf{W}$ with some elements zeroed,

$$\begin{aligned} \left|\mathbf{x} + \mathbf{x}'\right| &= \left|\mathbf{V} \cdot \mathbf{W}^{-1} \cdot \mathbf{U}^T \cdot \mathbf{b} + \mathbf{x}'\right| \\ &= \left|\mathbf{V} \cdot (\mathbf{W}^{-1} \cdot \mathbf{U}^T \cdot \mathbf{b} + \mathbf{V}^T \cdot \mathbf{x}')\right| \\ &= \left|\mathbf{W}^{-1} \cdot \mathbf{U}^T \cdot \mathbf{b} + \mathbf{V}^T \cdot \mathbf{x}'\right| \end{aligned} \tag{2.6.8}$$

Here the first equality follows from (2.6.7), and the second and third from the orthonormality of $\mathbf{V}$. If you now examine the two terms that make up the sum on the right-hand side, you will see that the first one has nonzero j components only where $w_j \neq 0$, while the second one, since $\mathbf{x}'$ is in the nullspace, has nonzero j components only where $w_j = 0$. Therefore the minimum length obtains for $\mathbf{x}' = 0$, q.e.d.

If $\mathbf{b}$ is not in the range of the singular matrix $\mathbf{A}$, then the set of equations (2.6.5) has no solution. But here is some good news: If $\mathbf{b}$ is not in the range of $\mathbf{A}$, then equation (2.6.7) can still be used to construct a "solution" vector $\mathbf{x}$. This vector $\mathbf{x}$ will not exactly solve $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. But, among all possible vectors $\mathbf{x}$, it will do the closest possible job in the least-squares sense. In other words, (2.6.7) finds

$$\mathbf{x} \quad \text{which minimizes} \quad r \equiv |\mathbf{A} \cdot \mathbf{x} - \mathbf{b}| \tag{2.6.9}$$

The number r is called the *residual* of the solution.

The proof is similar to (2.6.8): Suppose we modify $\mathbf{x}$ by adding some arbitrary $\mathbf{x}'$. Then $\mathbf{A} \cdot \mathbf{x} - \mathbf{b}$ is modified by adding some $\mathbf{b}' \equiv \mathbf{A} \cdot \mathbf{x}'$. Obviously $\mathbf{b}'$ is in the range of $\mathbf{A}$. We then have

$$\begin{aligned} \left|\mathbf{A} \cdot \mathbf{x} - \mathbf{b} + \mathbf{b}'\right| &= \left|(\mathbf{U} \cdot \mathbf{W} \cdot \mathbf{V}^T) \cdot (\mathbf{V} \cdot \mathbf{W}^{-1} \cdot \mathbf{U}^T \cdot \mathbf{b}) - \mathbf{b} + \mathbf{b}'\right| \\ &= \left|(\mathbf{U} \cdot \mathbf{W} \cdot \mathbf{W}^{-1} \cdot \mathbf{U}^T - 1) \cdot \mathbf{b} + \mathbf{b}'\right| \\ &= \left|\mathbf{U} \cdot \left[(\mathbf{W} \cdot \mathbf{W}^{-1} - 1) \cdot \mathbf{U}^T \cdot \mathbf{b} + \mathbf{U}^T \cdot \mathbf{b}'\right]\right| \\ &= \left|(\mathbf{W} \cdot \mathbf{W}^{-1} - 1) \cdot \mathbf{U}^T \cdot \mathbf{b} + \mathbf{U}^T \cdot \mathbf{b}'\right| \end{aligned} \tag{2.6.10}$$

Now, $(\mathbf{W} \cdot \mathbf{W}^{-1} - 1)$ is a diagonal matrix that has nonzero j components only for $w_j = 0$, while $\mathbf{U}^T\mathbf{b}'$ has nonzero j components only for $w_j \neq 0$, since $\mathbf{b}'$ lies in the range of $\mathbf{A}$. Therefore the minimum obtains for $\mathbf{b}' = 0$, q.e.d.

Equation (2.6.7), which is also equation (2.6.6) applied associatively to $\mathbf{b}$, is thus very general. If no w_j's are zero, it solves a nonsingular system of linear equations. If some w_j's are zero, and their reciprocals are made zero, then it gives a "best" solution, either the one of shortest length among many, or the one of minimum residual when no exact solution exists. Equation (2.6.6), with the singular $1/w_j$'s zeroized, is called the *Moore-Penrose inverse* or *pseudoinverse* of $\mathbf{A}$.

Equation (2.6.7) is implemented in the SVD object as the method solve. (As in LUdcmp, we also include an overloaded form that solves for multiple right-hand sides simultaneously.) The argument thresh inputs a value below which w_j's are to be considered as being zero; if you omit this argument, or set it to a negative value, then the program uses a default value based on expected roundoff error.

```
void SVD::solve(VecDoub_I &b, VecDoub_O &x, Doub thresh = -1.) {                    svd.h
```

Solve $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ for a vector $\mathbf{x}$ using the pseudoinverse of $\mathbf{A}$ as obtained by SVD. If positive, thresh is the threshold value below which singular values are considered as zero. If thresh is negative, a default based on expected roundoff error is used.

```
    Int i,j,jj;
    Doub s;
    if (b.size() != m || x.size() != n) throw("SVD::solve bad sizes");
    VecDoub tmp(n);
    tsh = (thresh >= 0. ? thresh : 0.5*sqrt(m+n+1.)*w[0]*eps);
    for (j=0;j<n;j++) {                 Calculate U^T B.
        s=0.0;
        if (w[j] > tsh) {               Nonzero result only if w_j is nonzero.
            for (i=0;i<m;i++) s += u[i][j]*b[i];
            s /= w[j];                  This is the divide by w_j.
        }
        tmp[j]=s;
    }
    for (j=0;j<n;j++) {                 Matrix multiply by V to get answer.
        s=0.0;
        for (jj=0;jj<n;jj++) s += v[j][jj]*tmp[jj];
        x[j]=s;
    }
}
```

```
void SVD::solve(MatDoub_I &b, MatDoub_O &x, Doub thresh = -1.)
```

Solves m sets of n equations $\mathbf{A} \cdot \mathbf{X} = \mathbf{B}$ using the pseudoinverse of $\mathbf{A}$. The right-hand sides are input as b[0..n-1][0..m-1], while x[0..n-1][0..m-1] returns the solutions. thresh as above.

```
{
    int i,j,m=b.ncols();
    if (b.nrows() != n || x.nrows() != n || b.ncols() != x.ncols())
        throw("SVD::solve bad sizes");
    VecDoub xx(n);
    for (j=0;j<m;j++) {                 Copy and solve each column in turn.
        for (i=0;i<n;i++) xx[i] = b[i][j];
        solve(xx,xx,thresh);
        for (i=0;i<n;i++) x[i][j] = xx[i];
    }
}
```

Figure 2.6.1 summarizes the situation for the SVD of square matrices.

There are cases in which you may want to set the value of thresh to larger than its default. (You can retrieve the default as the member value tsh.) In the discussion since equation (2.6.5), we have been pretending that a matrix either is singular or else isn't. Numerically, however, the more common situation is that some of the w_j's are very small but nonzero, so that the matrix is ill-conditioned. In that case, the direct solution methods of LU decomposition or Gaussian elimination may actually give a formal solution to the set of equations (that is, a zero pivot may not be encountered); but the solution vector may have wildly large components whose algebraic cancellation, when multiplying by the matrix $\mathbf{A}$, may give a very poor approximation to the right-hand vector $\mathbf{b}$. In such cases, the solution vector $\mathbf{x}$ obtained by *zeroing* the small w_j's and then using equation (2.6.7) is very often better (in the sense of the residual $|\mathbf{A} \cdot \mathbf{x} - \mathbf{b}|$ being smaller) than *both* the direct-method

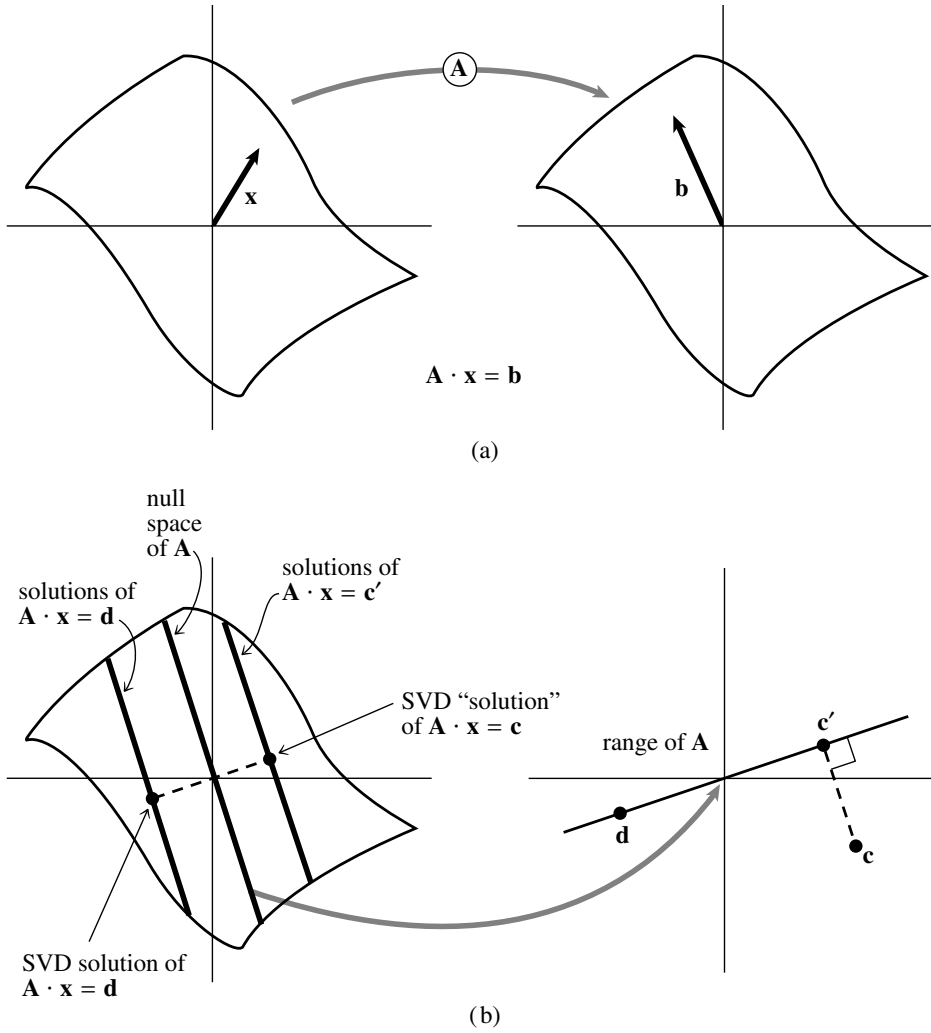


Figure 2.6.1. (a) A nonsingular matrix $\mathbf{A}$ maps a vector space into one of the same dimension. The vector $\mathbf{x}$ is mapped into $\mathbf{b}$, so that $\mathbf{x}$ satisfies the equation $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. (b) A singular matrix $\mathbf{A}$ maps a vector space into one of lower dimensionality, here a plane into a line, called the "range" of $\mathbf{A}$. The "nullspace" of $\mathbf{A}$ is mapped to zero. The solutions of $\mathbf{A} \cdot \mathbf{x} = \mathbf{d}$ consist of any one particular solution plus any vector in the nullspace, here forming a line parallel to the nullspace. Singular value decomposition (SVD) selects the particular solution closest to zero, as shown. The point $\mathbf{c}$ lies outside of the range of $\mathbf{A}$, so $\mathbf{A} \cdot \mathbf{x} = \mathbf{c}$ has no solution. SVD finds the least-squares best compromise solution, namely a solution of $\mathbf{A} \cdot \mathbf{x} = \mathbf{c}'$, as shown.

solution *and* the SVD solution where the small w_j's are left nonzero.

It may seem paradoxical that this can be so, since zeroing a singular value corresponds to throwing away one linear combination of the set of equations that we are trying to solve. The resolution of the paradox is that we are throwing away precisely a combination of equations that is so corrupted by roundoff error as to be at best useless; usually it is worse than useless since it "pulls" the solution vector way off toward infinity along some direction that is almost a nullspace vector. In doing this,

it compounds the roundoff problem and makes the residual $|\mathbf{A} \cdot \mathbf{x} - \mathbf{b}|$ larger.

You therefore have the opportunity of deciding at what threshold `thresh` to zero the small w_j's, based on some idea of what size of computed residual $|\mathbf{A} \cdot \mathbf{x} - \mathbf{b}|$ is acceptable.

For discussion of how the singular value decomposition of a matrix is related to its eigenvalues and eigenvectors, see §11.0.6.

2.6.3 SVD for Fewer Equations than Unknowns

If you have fewer linear equations M than unknowns N, then you are not expecting a unique solution. Usually there will be an $N - M$ dimensional family of solutions (which is the nullity, absent any other degeneracies), but the number could be larger. If you want to find this whole solution space, then `SVD` can readily do the job: Use `solve` to get one (the shortest) solution, then use `nullspace` to get a set of basis vectors for the nullspace. Your solutions are the former plus any linear combination of the latter.

2.6.4 SVD for More Equations than Unknowns

This situation will occur in Chapter 15, when we wish to find the least-squares solution to an overdetermined set of linear equations. In tableau, the equations to be solved are

$$\begin{pmatrix} \\ \mathbf{A} \\ \\ \end{pmatrix} \cdot \begin{pmatrix} \\ \mathbf{x} \\ \\ \end{pmatrix} = \begin{pmatrix} \\ \mathbf{b} \\ \\ \end{pmatrix} \tag{2.6.11}$$

The proofs that we gave above for the square case apply without modification to the case of more equations than unknowns. The least-squares solution vector $\mathbf{x}$ is given by applying the pseudoinverse (2.6.7), which, with nonsquare matrices, looks like this,

$$\begin{pmatrix} \\ \mathbf{x} \\ \\ \end{pmatrix} = \begin{pmatrix} \\ \mathbf{V} \\ \\ \end{pmatrix} \cdot \begin{pmatrix} \\ \mathrm{diag}(1/w_j) \\ \\ \end{pmatrix} \cdot \begin{pmatrix} \\ \mathbf{U}^T \\ \\ \end{pmatrix} \cdot \begin{pmatrix} \\ \mathbf{b} \\ \\ \end{pmatrix} \tag{2.6.12}$$

In general, the matrix $\mathbf{W}$ will not be singular, and no w_j's will need to be set to zero. Occasionally, however, there might be column degeneracies in $\mathbf{A}$. In this case you will need to zero some small w_j values after all. The corresponding column in $\mathbf{V}$ gives the linear combination of $\mathbf{x}$'s that is then ill-determined even by the supposedly overdetermined set.

Sometimes, although you do not need to zero any w_j's for *computational* reasons, you may nevertheless want to take note of any that are unusually small: Their corresponding columns in $\mathbf{V}$ are linear combinations of $\mathbf{x}$'s that are insensitive to your data. In fact, you may then wish to zero these w_j's, by increasing the value of `thresh`, to reduce the number of free parameters in the fit. These matters are discussed more fully in Chapter 15.

2.6.5 Constructing an Orthonormal Basis

Suppose that you have N vectors in an M-dimensional vector space, with $N \leq M$. Then the N vectors span some subspace of the full vector space. Often you want to construct an orthonormal set of N vectors that span the same subspace. The elementary textbook way to do this is by Gram-Schmidt orthogonalization, starting with one vector and then expanding the subspace one dimension at a time. Numerically, however, because of the build-up of roundoff errors, naive Gram-Schmidt orthogonalization is *terrible*.

The right way to construct an orthonormal basis for a subspace is by SVD: Form an $M \times N$ matrix $\mathbf{A}$ whose N columns are your vectors. Construct an `SVD` object from the matrix. The columns of the matrix $\mathbf{U}$ are your desired orthonormal basis vectors.

You might also want to check the w_j's for zero values. If any occur, then the spanned subspace was not, in fact, N-dimensional; the columns of $\mathbf{U}$ corresponding to zero w_j's should be discarded from the orthonormal basis set. The method `range` does this.

QR factorization, discussed in §2.10, also constructs an orthonormal basis; see [3].

2.6.6 Approximation of Matrices

Note that equation (2.6.1) can be rewritten to express any matrix A_{ij} as a sum of outer products of columns of $\mathbf{U}$ and rows of $\mathbf{V}^T$, with the "weighting factors" being the singular values w_j,

$$A_{ij} = \sum_{k=0}^{N-1} w_k \, U_{ik} V_{jk} \tag{2.6.13}$$

If you ever encounter a situation where *most* of the singular values w_j of a matrix $\mathbf{A}$ are very small, then $\mathbf{A}$ will be well-approximated by only a few terms in the sum (2.6.13). This means that you have to store only a few columns of $\mathbf{U}$ and $\mathbf{V}$ (the same k ones) and you will be able to recover, with good accuracy, the whole matrix.

Note also that it is very efficient to multiply such an approximated matrix by a vector $\mathbf{x}$: You just dot $\mathbf{x}$ with each of the stored columns of $\mathbf{V}$, multiply the resulting scalar by the corresponding w_k, and accumulate that multiple of the corresponding column of $\mathbf{U}$. If your matrix is approximated by a small number K of singular values, then this computation of $\mathbf{A} \cdot \mathbf{x}$ takes only about $K(M + N)$ multiplications, instead of MN for the full matrix.

2.6.7 Newer Algorithms

Analogous to the newer methods for eigenvalues of symmetric tridiagonal matrices mentioned in §11.4.4, there are newer methods for SVD. There is a divide-and-conquer algorithm, implemented in LAPACK as `dgesdd`, which is typically faster by a factor of about 5 for large matrices than the algorithm we give (which is similar to the LAPACK routine `dgesvd`). Another routine based on the MRRR algorithm (see §11.4.4) promises to be even better, but it is not available in LAPACK as of 2006. It will appear as routine `dbdscr`.

CITED REFERENCES AND FURTHER READING:

Numerical Recipes Software 2007, "SVD Implementation Code," *Numerical Recipes Webnote No. 2*, at `http://www.nr.com/webnotes?2` [1]

Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), Chapter 9.[2]

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §8.6 and Chapter 12 (SVD). QR decomposition is discussed in §5.2.6.[3]

Lawson, C.L., and Hanson, R. 1974, *Solving Least Squares Problems* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 1995 (Philadelphia: S.I.A.M.), Chapter 18.

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I.10 by G.H. Golub and C. Reinsch.[4]

Anderson, E., et al. 1999, LAPACK User's Guide, 3rd ed. (Philadelphia: S.I.A.M.). Online with software at 2007+, `http://www.netlib.org/lapack`.[5]

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §6.7.[6]

2.7 Sparse Linear Systems

A system of linear equations is called *sparse* if only a relatively small number of its matrix elements a_{ij} are nonzero. It is wasteful to use general methods of linear algebra on such problems, because most of the $O(N^3)$ arithmetic operations devoted to solving the set of equations or inverting the matrix involve zero operands. Furthermore, you might wish to work problems so large as to tax your available memory space, and it is wasteful to reserve storage for unfruitful zero elements. Note that there are two distinct (and not always compatible) goals for any sparse matrix method: saving time and/or saving space.

We considered one archetypal sparse form in §2.4, the band-diagonal matrix. In the tridiagonal case, e.g., we saw that it was possible to save both time (order N instead of N^3) and space (order N instead of N^2). The method of solution was not different in principle from the general method of LU decomposition; it was just applied cleverly, and with due attention to the bookkeeping of zero elements. Many practical schemes for dealing with sparse problems have this same character. They are fundamentally decomposition schemes, or else elimination schemes akin to Gauss-Jordan, but carefully optimized so as to minimize the number of so-called

fill-ins, initially zero elements that must become nonzero during the solution process, and for which storage must be reserved.

Direct methods for solving sparse equations, then, depend crucially on the precise pattern of sparsity of the matrix. Patterns that occur frequently, or that are useful as way stations in the reduction of more general forms, already have special names and special methods of solution. We do not have space here for any detailed review of these. References listed at the end of this section will furnish you with an "in" to the specialized literature, and the following list of buzz words (and Figure 2.7.1) will at least let you hold your own at cocktail parties:

- tridiagonal
- band-diagonal (or banded) with bandwidth M
- band triangular
- block diagonal
- block tridiagonal
- block triangular
- cyclic banded
- singly (or doubly) bordered block diagonal
- singly (or doubly) bordered block triangular
- singly (or doubly) bordered band-diagonal
- singly (or doubly) bordered band triangular
- other (!)

You should also be aware of some of the special sparse forms that occur in the solution of partial differential equations in two or more dimensions. See Chapter 20.

If your particular pattern of sparsity is not a simple one, then you may wish to try an *analyze/factorize/operate* package, which automates the procedure of figuring out how fill-ins are to be minimized. The *analyze* stage is done once only for each pattern of sparsity. The *factorize* stage is done once for each particular matrix that fits the pattern. The *operate* stage is performed once for each right-hand side to be used with the particular matrix. Consult [2,3] for references on this. The NAG library [4] has an analyze/factorize/operate capability. A substantial collection of routines for sparse matrix calculation is also available from IMSL [5] as the *Yale Sparse Matrix Package* [6].

You should be aware that the special order of interchanges and eliminations, prescribed by a sparse matrix method so as to minimize fill-ins and arithmetic operations, generally acts to decrease the method's numerical stability as compared to, e.g., regular LU decomposition with pivoting. Scaling your problem so as to make its nonzero matrix elements have comparable magnitudes (if you can do it) will sometimes ameliorate this problem.

In the remainder of this section, we present some concepts that are applicable to some general classes of sparse matrices, and which do not necessarily depend on details of the pattern of sparsity.

2.7.1 Sherman-Morrison Formula

Suppose that you have already obtained, by herculean effort, the inverse matrix $\mathbf{A}^{-1}$ of a square matrix $\mathbf{A}$. Now you want to make a "small" change in $\mathbf{A}$, for example change one element a_{ij}, or a few elements, or one row, or one column. Is there any

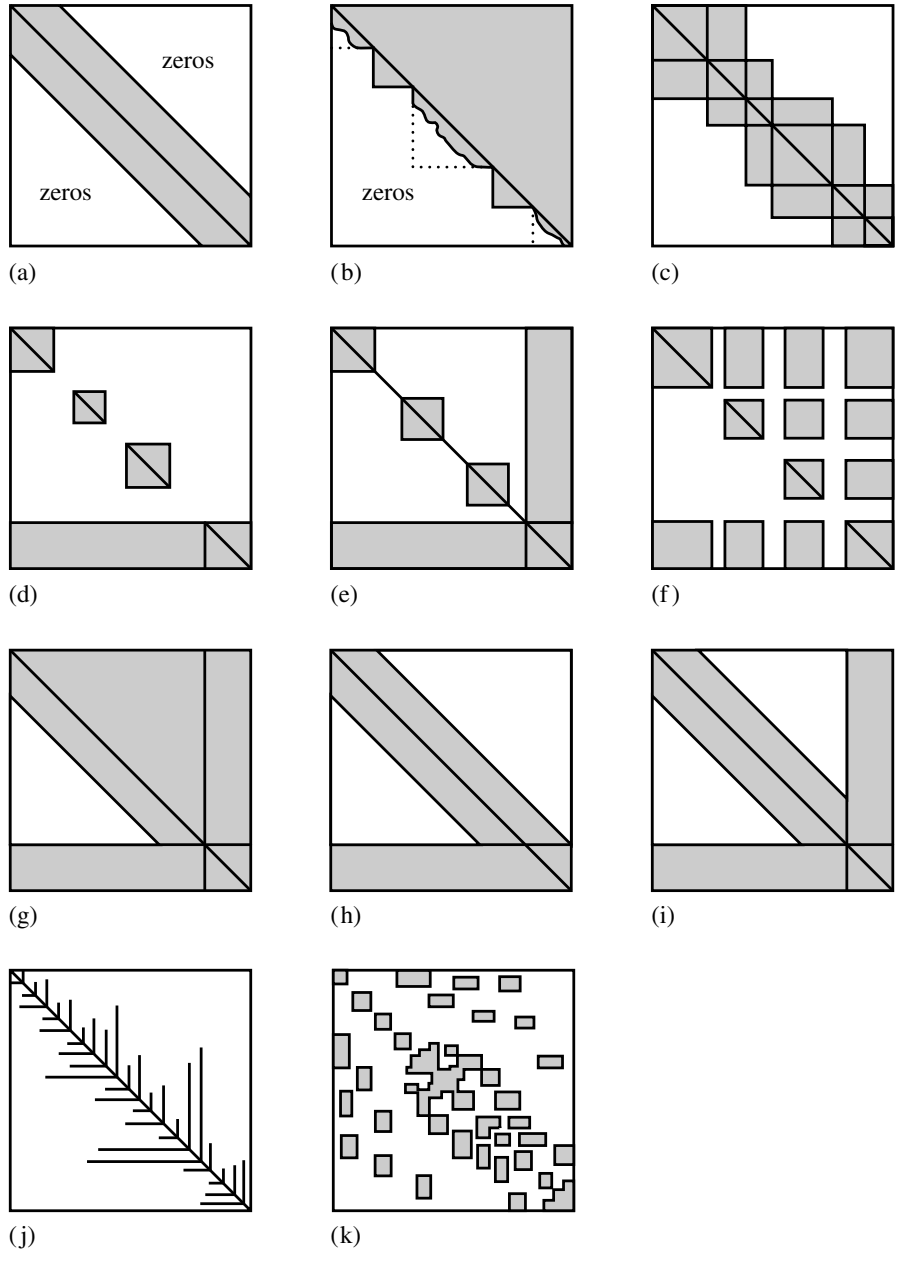


Figure 2.7.1. Some standard forms for sparse matrices. (a) Band-diagonal; (b) block triangular; (c) block tridiagonal; (d) singly bordered block diagonal; (e) doubly bordered block diagonal; (f) singly bordered block triangular; (g) bordered band-triangular; (h) and (i) singly and doubly bordered band-diagonal; (j) and (k) other! (after Tewarson) [1].

way of calculating the corresponding change in $\mathbf{A}^{-1}$ without repeating your difficult labors? Yes, if your change is of the form

$$\mathbf{A} \rightarrow (\mathbf{A} + \mathbf{u} \otimes \mathbf{v}) \tag{2.7.1}$$

for some vectors $\mathbf{u}$ and $\mathbf{v}$. If $\mathbf{u}$ is a unit vector $\mathbf{e}_i$, then (2.7.1) adds the components of $\mathbf{v}$ to the ith row. (Recall that $\mathbf{u} \otimes \mathbf{v}$ is a matrix whose i,jth element is the product of the ith component of $\mathbf{u}$ and the jth component of $\mathbf{v}$.) If $\mathbf{v}$ is a unit vector $\mathbf{e}_j$, then (2.7.1) adds the components of $\mathbf{u}$ to the jth column. If both $\mathbf{u}$ and $\mathbf{v}$ are proportional to unit vectors $\mathbf{e}_i$ and $\mathbf{e}_j$, respectively, then a term is added only to the element a_{ij}.

The *Sherman-Morrison* formula gives the inverse $(\mathbf{A} + \mathbf{u} \otimes \mathbf{v})^{-1}$ and is derived briefly as follows:

$$\begin{aligned}(\mathbf{A} + \mathbf{u} \otimes \mathbf{v})^{-1} &= (\mathbf{1} + \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v})^{-1} \cdot \mathbf{A}^{-1} \\ &= (\mathbf{1} - \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v} + \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v} \cdot \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v} - \ldots) \cdot \mathbf{A}^{-1} \\ &= \mathbf{A}^{-1} - \mathbf{A}^{-1} \cdot \mathbf{u} \otimes \mathbf{v} \cdot \mathbf{A}^{-1} \, (1 - \lambda + \lambda^2 - \ldots) \\ &= \mathbf{A}^{-1} - \frac{(\mathbf{A}^{-1} \cdot \mathbf{u}) \otimes (\mathbf{v} \cdot \mathbf{A}^{-1})}{1 + \lambda} \end{aligned} \tag{2.7.2}$$

where

$$\lambda \equiv \mathbf{v} \cdot \mathbf{A}^{-1} \cdot \mathbf{u} \tag{2.7.3}$$

The second line of (2.7.2) is a formal power series expansion. In the third line, the associativity of outer and inner products is used to factor out the scalars λ.

The use of (2.7.2) is this: Given $\mathbf{A}^{-1}$ and the vectors $\mathbf{u}$ and $\mathbf{v}$, we need only perform two matrix multiplications and a vector dot product,

$$\mathbf{z} \equiv \mathbf{A}^{-1} \cdot \mathbf{u} \qquad \mathbf{w} \equiv (\mathbf{A}^{-1})^T \cdot \mathbf{v} \qquad \lambda = \mathbf{v} \cdot \mathbf{z} \tag{2.7.4}$$

to get the desired change in the inverse

$$\mathbf{A}^{-1} \quad \rightarrow \quad \mathbf{A}^{-1} - \frac{\mathbf{z} \otimes \mathbf{w}}{1 + \lambda} \tag{2.7.5}$$

The whole procedure requires only $3N^2$ multiplies and a like number of adds (an even smaller number if $\mathbf{u}$ or $\mathbf{v}$ is a unit vector).

The Sherman-Morrison formula can be directly applied to a class of sparse problems. If you already have a fast way of calculating the inverse of $\mathbf{A}$ (e.g., a tridiagonal matrix or some other standard sparse form), then (2.7.4) – (2.7.5) allow you to build up to your related but more complicated form, adding for example a row or column at a time. Notice that you can apply the Sherman-Morrison formula more than once successively, using at each stage the most recent update of $\mathbf{A}^{-1}$ (equation 2.7.5). Of course, if you have to modify *every* row, then you are back to an N^3 method. The constant in front of the N^3 is only a few times worse than the better direct methods, but you have deprived yourself of the stabilizing advantages of pivoting — so be careful.

For some other sparse problems, the Sherman-Morrison formula cannot be directly applied for the simple reason that storage of the whole inverse matrix $\mathbf{A}^{-1}$ is not feasible. If you want to add only a single correction of the form $\mathbf{u} \otimes \mathbf{v}$ and solve the linear system

$$(\mathbf{A} + \mathbf{u} \otimes \mathbf{v}) \cdot \mathbf{x} = \mathbf{b} \tag{2.7.6}$$

then you proceed as follows. Using the fast method that is presumed available for the matrix $\mathbf{A}$, solve the two auxiliary problems

$$\mathbf{A} \cdot \mathbf{y} = \mathbf{b} \qquad \mathbf{A} \cdot \mathbf{z} = \mathbf{u} \tag{2.7.7}$$

for the vectors $\mathbf{y}$ and $\mathbf{z}$. In terms of these,

$$\mathbf{x} = \mathbf{y} - \left[\frac{\mathbf{v}\cdot\mathbf{y}}{1+(\mathbf{v}\cdot\mathbf{z})}\right]\mathbf{z} \tag{2.7.8}$$

as we see by multiplying (2.7.2) on the right by $\mathbf{b}$.

2.7.2 Cyclic Tridiagonal Systems

So-called *cyclic tridiagonal systems* occur quite frequently and are a good example of how to use the Sherman-Morrison formula in the manner just described. The equations have the form

$$\begin{bmatrix} b_0 & c_0 & 0 & \cdots & & & \beta \\ a_1 & b_1 & c_1 & \cdots & & & \\ & & & \cdots & & & \\ & & & \cdots & a_{N-2} & b_{N-2} & c_{N-2} \\ \alpha & & & \cdots & 0 & a_{N-1} & b_{N-1} \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ x_1 \\ \cdots \\ x_{N-2} \\ x_{N-1} \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ \cdots \\ r_{N-2} \\ r_{N-1} \end{bmatrix} \tag{2.7.9}$$

This is a tridiagonal system, except for the matrix elements α and β in the corners. Forms like this are typically generated by finite differencing differential equations with periodic boundary conditions (§20.4).

We use the Sherman-Morrison formula, treating the system as tridiagonal plus a correction. In the notation of equation (2.7.6), define vectors $\mathbf{u}$ and $\mathbf{v}$ to be

$$\mathbf{u} = \begin{bmatrix} \gamma \\ 0 \\ \vdots \\ 0 \\ \alpha \end{bmatrix} \qquad \mathbf{v} = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \\ \beta/\gamma \end{bmatrix} \tag{2.7.10}$$

Here γ is arbitrary for the moment. Then the matrix $\mathbf{A}$ is the tridiagonal part of the matrix in (2.7.9), with two terms modified:

$$b_0' = b_0 - \gamma, \qquad b_{N-1}' = b_{N-1} - \alpha\beta/\gamma \tag{2.7.11}$$

We now solve equations (2.7.7) with the standard tridiagonal algorithm and then get the solution from equation (2.7.8).

The routine `cyclic` below implements this algorithm. We choose the arbitrary parameter $\gamma = -b_0$ to avoid loss of precision by subtraction in the first of equations (2.7.11). In the unlikely event that this causes loss of precision in the second of these equations, you can make a different choice.

tridag.h

```
void cyclic(VecDoub_I &a, VecDoub_I &b, VecDoub_I &c, const Doub alpha,
    const Doub beta, VecDoub_I &r, VecDoub_O &x)
```

Solves for a vector `x[0..n-1]` the "cyclic" set of linear equations given by equation (2.7.9). `a`, `b`, `c`, and `r` are input vectors, all dimensioned as `[0..n-1]`, while `alpha` and `beta` are the corner entries in the matrix. The input is not modified.

```
{
    Int i,n=a.size();
    Doub fact,gamma;
    if (n <= 2) throw("n too small in cyclic");
```

```
    VecDoub bb(n),u(n),z(n);
    gamma = -b[0];                          Avoid subtraction error in forming bb[0].
    bb[0]=b[0]-gamma;                       Set up the diagonal of the modified tridi-
    bb[n-1]=b[n-1]-alpha*beta/gamma;            agonal system.
    for (i=1;i<n-1;i++) bb[i]=b[i];
    tridag(a,bb,c,r,x);                     Solve A · x = r.
    u[0]=gamma;                             Set up the vector u.
    u[n-1]=alpha;
    for (i=1;i<n-1;i++) u[i]=0.0;
    tridag(a,bb,c,u,z);                     Solve A · z = u.
    fact=(x[0]+beta*x[n-1]/gamma)/          Form v · x/(1 + v · z).
        (1.0+z[0]+beta*z[n-1]/gamma);
    for (i=0;i<n;i++) x[i] -= fact*z[i];    Now get the solution vector x.
}
```

2.7.3 Woodbury Formula

If you want to add more than a single correction term, then you cannot use (2.7.8) repeatedly, since without storing a new $\mathbf{A}^{-1}$ you will not be able to solve the auxiliary problems (2.7.7) efficiently after the first step. Instead, you need the *Woodbury formula*, which is the block-matrix version of the Sherman-Morrison formula,

$$(\mathbf{A} + \mathbf{U} \cdot \mathbf{V}^T)^{-1} = \mathbf{A}^{-1} - \left[\mathbf{A}^{-1} \cdot \mathbf{U} \cdot (\mathbf{1} + \mathbf{V}^T \cdot \mathbf{A}^{-1} \cdot \mathbf{U})^{-1} \cdot \mathbf{V}^T \cdot \mathbf{A}^{-1}\right] \tag{2.7.12}$$

Here A is, as usual, an $N \times N$ matrix, while $\mathbf{U}$ and $\mathbf{V}$ are $N \times P$ matrices with $P < N$ and usually $P \ll N$. The inner piece of the correction term may become clearer if written as the tableau,

$$\begin{bmatrix} & \\ \mathbf{U} & \\ & \end{bmatrix} \cdot \left[\mathbf{1} + \mathbf{V}^T \cdot \mathbf{A}^{-1} \cdot \mathbf{U}\right]^{-1} \cdot \begin{bmatrix} & \\ \mathbf{V}^T & \\ & \end{bmatrix} \tag{2.7.13}$$

where you can see that the matrix whose inverse is needed is only $P \times P$ rather than $N \times N$.

The relation between the Woodbury formula and successive applications of the Sherman-Morrison formula is now clarified by noting that, if $\mathbf{U}$ is the matrix formed by columns out of the P vectors $\mathbf{u}_0, \ldots, \mathbf{u}_{P-1}$, and $\mathbf{V}$ is the matrix formed by columns out of the P vectors $\mathbf{v}_0, \ldots, \mathbf{v}_{P-1}$,

$$\mathbf{U} \equiv \begin{bmatrix} & & \\ \mathbf{u}_0 & \cdots & \mathbf{u}_{P-1} \\ & & \end{bmatrix} \qquad \mathbf{V} \equiv \begin{bmatrix} & & \\ \mathbf{v}_0 & \cdots & \mathbf{v}_{P-1} \\ & & \end{bmatrix} \tag{2.7.14}$$

then two ways of expressing the same correction to $\mathbf{A}$ are

$$\left(\mathbf{A} + \sum_{k=0}^{P-1} \mathbf{u}_k \otimes \mathbf{v}_k\right) = (\mathbf{A} + \mathbf{U} \cdot \mathbf{V}^T) \tag{2.7.15}$$

(Note that the subscripts on $\mathbf{u}$ and $\mathbf{v}$ do *not* denote components, but rather distinguish the different column vectors.)

Equation (2.7.15) reveals that, if you have $\mathbf{A}^{-1}$ in storage, then you can either make the P corrections in one fell swoop by using (2.7.12) and inverting a $P \times P$ matrix, or else make them by applying (2.7.5) P successive times.

If you don't have storage for $\mathbf{A}^{-1}$, then you *must* use (2.7.12) in the following way: To solve the linear equation

$$\left(\mathbf{A} + \sum_{k=0}^{P-1} \mathbf{u}_k \otimes \mathbf{v}_k\right) \cdot \mathbf{x} = \mathbf{b} \tag{2.7.16}$$

first solve the P auxiliary problems

$$\begin{aligned} \mathbf{A} \cdot \mathbf{z}_0 &= \mathbf{u}_0 \\ \mathbf{A} \cdot \mathbf{z}_1 &= \mathbf{u}_1 \\ &\cdots \\ \mathbf{A} \cdot \mathbf{z}_{P-1} &= \mathbf{u}_{P-1} \end{aligned} \tag{2.7.17}$$

and construct the matrix $\mathbf{Z}$ by columns from the $\mathbf{z}$'s obtained,

$$\mathbf{Z} \equiv \begin{bmatrix} \\ \mathbf{z}_0 \\ \\ \end{bmatrix} \cdots \begin{bmatrix} \\ \mathbf{z}_{P-1} \\ \\ \end{bmatrix} \tag{2.7.18}$$

Next, do the $P \times P$ matrix inversion

$$\mathbf{H} \equiv (\mathbf{1} + \mathbf{V}^T \cdot \mathbf{Z})^{-1} \tag{2.7.19}$$

Finally, solve the one further auxiliary problem

$$\mathbf{A} \cdot \mathbf{y} = \mathbf{b} \tag{2.7.20}$$

In terms of these quantities, the solution is given by

$$\mathbf{x} = \mathbf{y} - \mathbf{Z} \cdot \left[\mathbf{H} \cdot (\mathbf{V}^T \cdot \mathbf{y})\right] \tag{2.7.21}$$

2.7.4 Inversion by Partitioning

Once in a while, you will encounter a matrix (not even necessarily sparse) that can be inverted efficiently by partitioning. Suppose that the $N \times N$ matrix $\mathbf{A}$ is partitioned into

$$\mathbf{A} = \begin{bmatrix} \mathbf{P} & \mathbf{Q} \\ \mathbf{R} & \mathbf{S} \end{bmatrix} \tag{2.7.22}$$

where $\mathbf{P}$ and $\mathbf{S}$ are square matrices of size $p \times p$ and $s \times s$, respectively ($p + s = N$). The matrices $\mathbf{Q}$ and $\mathbf{R}$ are not necessarily square and have sizes $p \times s$ and $s \times p$, respectively.

If the inverse of $\mathbf{A}$ is partitioned in the same manner,

$$\mathbf{A}^{-1} = \begin{bmatrix} \tilde{\mathbf{P}} & \tilde{\mathbf{Q}} \\ \tilde{\mathbf{R}} & \tilde{\mathbf{S}} \end{bmatrix} \tag{2.7.23}$$

then $\tilde{\mathbf{P}}$, $\tilde{\mathbf{Q}}$, $\tilde{\mathbf{R}}$, $\tilde{\mathbf{S}}$, which have the same sizes as $\mathbf{P}$, $\mathbf{Q}$, $\mathbf{R}$, $\mathbf{S}$, respectively, can be found by either the formulas

$$\begin{aligned}
\tilde{\mathbf{P}} &= (\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1} \\
\tilde{\mathbf{Q}} &= -(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1}\cdot(\mathbf{Q}\cdot\mathbf{S}^{-1}) \\
\tilde{\mathbf{R}} &= -(\mathbf{S}^{-1}\cdot\mathbf{R})\cdot(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1} \\
\tilde{\mathbf{S}} &= \mathbf{S}^{-1} + (\mathbf{S}^{-1}\cdot\mathbf{R})\cdot(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1}\cdot(\mathbf{Q}\cdot\mathbf{S}^{-1})
\end{aligned} \tag{2.7.24}$$

or else by the equivalent formulas

$$\begin{aligned}
\tilde{\mathbf{P}} &= \mathbf{P}^{-1} + (\mathbf{P}^{-1}\cdot\mathbf{Q})\cdot(\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1}\cdot(\mathbf{R}\cdot\mathbf{P}^{-1}) \\
\tilde{\mathbf{Q}} &= -(\mathbf{P}^{-1}\cdot\mathbf{Q})\cdot(\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1} \\
\tilde{\mathbf{R}} &= -(\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1}\cdot(\mathbf{R}\cdot\mathbf{P}^{-1}) \\
\tilde{\mathbf{S}} &= (\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1}
\end{aligned} \tag{2.7.25}$$

The parentheses in equations (2.7.24) and (2.7.25) highlight repeated factors that you may wish to compute only once. (Of course, by associativity, you can instead do the matrix multiplications in any order you like.) The choice between using equations (2.7.24) and (2.7.25) depends on whether you want $\tilde{\mathbf{P}}$ or $\tilde{\mathbf{S}}$ to have the simpler formula; or on whether the repeated expression $(\mathbf{S}-\mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1}$ is easier to calculate than the expression $(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1}$; or on the relative sizes of $\mathbf{P}$ and $\mathbf{S}$; or on whether $\mathbf{P}^{-1}$ or $\mathbf{S}^{-1}$ is already known.

Another sometimes useful formula is for the determinant of the partitioned matrix,

$$\det\mathbf{A} = \det\mathbf{P}\,\det(\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q}) = \det\mathbf{S}\,\det(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R}) \tag{2.7.26}$$

2.7.5 Indexed Storage of Sparse Matrices

We have already seen (§2.4) that tri- or band-diagonal matrices can be stored in a compact format that allocates storage only to elements that can be nonzero, plus perhaps a few wasted locations to make the bookkeeping easier. What about more general sparse matrices? When a sparse matrix of dimension $M \times N$ contains only a few times M or N nonzero elements (a typical case), it is surely inefficient — and often physically impossible — to allocate storage for all MN elements. Even if one did allocate such storage, it would be inefficient or prohibitive in machine time to loop over all of it in search of nonzero elements.

Obviously some kind of indexed storage scheme is required, one that stores only nonzero matrix elements, along with sufficient auxiliary information to determine where an element logically belongs and how the various elements can be looped over in common matrix operations. Unfortunately, there is no one standard scheme in general use. Each scheme has its own pluses and minuses, depending on the application.

Before we look at sparse matrices, let's consider the simpler problem of a *sparse vector*. The obvious data structure is a list of the nonzero values and another list of the corresponding locations:

sparse.h

```
struct NRsparseCol
Sparse vector data structure.
{
    Int nrows;                  Number of rows.
    Int nvals;                  Maximum number of nonzeros.
```

```
    VecInt row_ind;                                  Row indices of nonzeros.
    VecDoub val;                                     Array of nonzero values.

    NRsparseCol(Int m,Int nnvals) : nrows(m), nvals(nnvals),
    row_ind(nnvals,0),val(nnvals,0.0) {}             Constructor. Initializes vector to zero.

    NRsparseCol() : nrows(0),nvals(0),row_ind(),val() {}     Default constructor.

    void resize(Int m, Int nnvals) {
        nrows = m;
        nvals = nnvals;
        row_ind.assign(nnvals,0);
        val.assign(nnvals,0.0);
    }

};
```

While we think of this as defining a column vector, you can use exactly the same data structure for a row vector — just mentally interchange the meaning of row and column for the variables. For matrices, however, we have to decide ahead of time whether to use row-oriented or column-oriented storage.

One simple scheme is to use a vector of sparse columns:

```
NRvector<NRsparseCol *> a;
for (i=0;i<n;i++) {
    nvals=...
    a[i]=new NRsparseCol(m,nvals);
}
```

Each column is filled with statements like

```
count=0;
for (j=...) {
    a[i]->row_ind[count]=...
    a[i]->val[count]=...
    count++;
}
```

This data structure is good for an algorithm that primarily works with columns of the matrix, but it is not very efficient when one needs to loop over all elements of the matrix.

A good general storage scheme is the *compressed column storage* format. It is sometimes called the Harwell-Boeing format, after the two large organizations that first systematically provided a standard collection of sparse matrices for research purposes. In this scheme, three vectors are used: `val` for the nonzero values as they are traversed column by column, `row_ind` for the corresponding row indices of each value, and `col_ptr` for the locations in the other two arrays that start a column. In other words, if `val[k]=a[i][j]`, then `row_ind[k]=i`. The first nonzero in column `j` is at `col_ptr[j]`. The last is at `col_ptr[j+1]-1`. Note that `col_ptr[0]` is always 0, and by convention we define `col_ptr[n]` equal to the number of nonzeros. Note also that the dimension of the `col_ptr` array is $N + 1$, not N. The advantage of this scheme is that it requires storage of only about two times the number of nonzero matrix elements. (Other methods can require as much as three or five times.)

As an example, consider the matrix

$$\begin{bmatrix} 3.0 & 0.0 & 1.0 & 2.0 & 0.0 \\ 0.0 & 4.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 7.0 & 5.0 & 9.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 6.0 & 5.0 \end{bmatrix} \tag{2.7.27}$$

In compressed column storage mode, matrix (2.7.27) is represented by two arrays of length 9 and an array of length 6, as follows

index k	0	1	2	3	4	5	6	7	8
val[k]	3.0	4.0	7.0	1.0	5.0	2.0	9.0	6.0	5.0
row_ind[k]	0	1	2	0	2	0	2	4	4

(2.7.28)

index i	0	1	2	3	4	5
col_ptr[i]	0	1	3	5	8	9

Notice that, according to the storage rules, the value of N (namely 5) is the maximum valid index in `col_ptr`. The value of `col_ptr[5]` is 9, the length of the other two arrays. The elements 1.0 and 5.0 in column number 2, for example, are located in positions `col_ptr[2]` $\le k <$ `col_ptr[3]`.

Here is a data structure to handle this storage scheme:

sparse.h
```
struct NRsparseMat
Sparse matrix data structure for compressed column storage.
{
    Int nrows;                          Number of rows.
    Int ncols;                          Number of columns.
    Int nvals;                          Maximum number of nonzeros.
    VecInt col_ptr;                     Pointers to start of columns. Length is ncols+1.
    VecInt row_ind;                     Row indices of nonzeros.
    VecDoub val;                        Array of nonzero values.

    NRsparseMat();                          Default constructor.
    NRsparseMat(Int m,Int n,Int nnvals);    Constructor. Initializes vector to zero.
    VecDoub ax(const VecDoub &x) const;     Multiply A by a vector x[0..ncols-1].
    VecDoub atx(const VecDoub &x) const;    Multiply A^T by a vector x[0..nrows-1].
    NRsparseMat transpose() const;          Form A^T.
};
```

The code for the constructors is standard:

sparse.h
```
NRsparseMat::NRsparseMat() : nrows(0),ncols(0),nvals(0),col_ptr(),
    row_ind(),val() {}
NRsparseMat::NRsparseMat(Int m,Int n,Int nnvals) : nrows(m),ncols(n),
    nvals(nnvals),col_ptr(n+1,0),row_ind(nnvals,0),val(nnvals,0.0) {}
```

The single most important use of a matrix in compressed column storage mode is to multiply a vector to its right. Don't implement this by traversing the rows of $\mathbf{A}$, which is extremely inefficient in this storage mode. Here's the right way to do it:

sparse.h
```
VecDoub NRsparseMat::ax(const VecDoub &x) const {
    VecDoub y(nrows,0.0);
    for (Int j=0;j<ncols;j++) {
        for (Int i=col_ptr[j];i<col_ptr[j+1];i++)
            y[row_ind[i]] += val[i]*x[j];
    }
    return y;
}
```

Some inefficiency occurs because of the indirect addressing. While there are other storage modes that minimize this, they have their own drawbacks.

It is also simple to multiply the *transpose* of a matrix by a vector to its right, since we just traverse the columns directly. (Indirect addressing is still required.) Note that the transpose matrix is not actually constructed.

```
VecDoub NRsparseMat::atx(const VecDoub &x) const {                    sparse.h
    VecDoub y(ncols);
    for (Int i=0;i<ncols;i++) {
        y[i]=0.0;
        for (Int j=col_ptr[i];j<col_ptr[i+1];j++)
            y[i] += val[j]*x[row_ind[j]];
    }
    return y;
}
```

Because the choice of compressed column storage treats rows and columns quite differently, it is rather an involved operation to construct the transpose of a matrix, given the matrix itself in compressed column storage mode. When the operation cannot be avoided, it is

```
NRsparseMat NRsparseMat::transpose() const {                          sparse.h
    Int i,j,k,index,m=nrows,n=ncols;
    NRsparseMat at(n,m,nvals);              Initialized to zero.
    First find the column lengths for A^T, i.e. the row lengths of A.
    VecInt count(m,0);                      Temporary counters for each row of A.
    for (i=0;i<n;i++)
        for (j=col_ptr[i];j<col_ptr[i+1];j++) {
            k=row_ind[j];
            count[k]++;
        }
    for (j=0;j<m;j++)                       Now set at.col_ptr. 0th entry stays 0.
        at.col_ptr[j+1]=at.col_ptr[j]+count[j];
    for(j=0;j<m;j++)                        Reset counters to zero.
        count[j]=0;
    for (i=0;i<n;i++)                       Main loop.
        for (j=col_ptr[i];j<col_ptr[i+1];j++) {
            k=row_ind[j];
            index=at.col_ptr[k]+count[k];   Element's position in column of A^T.
            at.row_ind[index]=i;
            at.val[index]=val[j];
            count[k]++;                     Increment counter for next element in that
        }                                       column.
    return at;
}
```

The only sparse matrix-matrix multiply routine we give is to form the product $\mathbf{ADA}^T$, where $\mathbf{D}$ is a diagonal matrix. This particular product is used to form the so-called normal equations in the interior-point method for linear programming (§10.11). We encapsulate the algorithm in its own structure, `ADAT`:

```
struct ADAT {                                                         sparse.h
    const NRsparseMat &a,&at;               Store references to A and A^T.
    NRsparseMat *adat;                      This will hold ADA^T.

    ADAT(const NRsparseMat &A,const NRsparseMat &AT);
    Allocates compressed column storage for AA^T, where A and A^T are input in compressed
    column format, and fills in values of col_ptr and row_ind. Each column must be in sorted
    order in input matrices. Matrix is output with each column sorted.
    void updateD(const VecDoub &D);
    Computes ADA^T, where D is a diagonal matrix. This function can be called repeatedly
    to update ADA^T for fixed A.
    NRsparseMat &ref();
    Returns reference to adat, which holds ADA^T.
    ~ADAT();
};
```

The algorithm proceeds in two steps. First, the nonzero pattern of $\mathbf{AA}^T$ is found by a call to the constructor. Since $\mathbf{D}$ is diagonal, $\mathbf{AA}^T$ and $\mathbf{ADA}^T$ have the same nonzero structure. Algorithms using `ADAT` will typically have both $\mathbf{A}$ and $\mathbf{A}^T$ available, so we pass them both to the constructor rather than recompute $\mathbf{A}^T$ from $\mathbf{A}$. The constructor allocates storage and assigns values to `col_ptr` and `row_ind`. The structure of `ADAT` is returned with columns in sorted order because routines like the AMD ordering algorithm used in §10.11 require it.

sparse.h

```
ADAT::ADAT(const NRsparseMat &A,const NRsparseMat &AT) : a(A), at(AT) {
    Int h,i,j,k,l,nvals,m=AT.ncols;
    VecInt done(m);
    for (i=0;i<m;i++)                       Initialize to not done.
        done[i]=-1;
    nvals=0;                                First pass determines number of nonzeros.
    for (j=0;j<m;j++) {                     Outer loop over columns of A^T in AA^T.
        for (i=AT.col_ptr[j];i<AT.col_ptr[j+1];i++) {
            k=AT.row_ind[i];                A^T_{kj} ≠ 0. Find column k in first matrix, A.
            for (l=A.col_ptr[k];l<A.col_ptr[k+1];l++) {
                h=A.row_ind[l];             A_{hl} ≠ 0.
                if (done[h] != j) {         Test if contribution already included.
                    done[h]=j;
                    nvals++;
                }
            }
        }
    }
    adat = new NRsparseMat(m,m,nvals);      Allocate storage for ADAT.
    for (i=0;i<m;i++)                       Re-initialize.
        done[i]=-1;
    nvals=0;
    Second pass: Determine columns of adat. Code is identical to first pass except adat->col_ptr
    and adat->row_ind get assigned at appropriate places.
    for (j=0;j<m;j++) {
        adat->col_ptr[j]=nvals;
        for (i=AT.col_ptr[j];i<AT.col_ptr[j+1];i++) {
            k=AT.row_ind[i];
            for (l=A.col_ptr[k];l<A.col_ptr[k+1];l++) {
                h=A.row_ind[l];
                if (done[h] != j) {
                    done[h]=j;
                    adat->row_ind[nvals]=h;
                    nvals++;
                }
            }
        }
    }
    adat->col_ptr[m]=nvals;                 Set last value.
    for (j=0;j<m;j++) {                     Sort columns
        i=adat->col_ptr[j];
        Int size=adat->col_ptr[j+1]-i;
        if (size > 1) {
            VecInt col(size,&adat->row_ind[i]);
            sort(col);
            for (k=0;k<size;k++)
                adat->row_ind[i+k]=col[k];
        }
    }
}
```

The next routine, `updateD`, actually fills in the values in the `val` array. It can be called repeatedly to update $\mathbf{ADA}^T$ for fixed $\mathbf{A}$.

sparse.h
```
void ADAT::updateD(const VecDoub &D) {
    Int h,i,j,k,l,m=a.nrows,n=a.ncols;
    VecDoub temp(n),temp2(m,0.0);
    for (i=0;i<m;i++) {                    Outer loop over columns of A^T.
        for (j=at.col_ptr[i];j< at.col_ptr[i+1];j++) {
            k=at.row_ptr[j];               Scale elements of each column with D and
            temp[k]=at.val[j]*D[k];            store in temp.
        }
        for (j=at.col_ptr[i];j<at.col_ptr[i+1];j++) {        Go down column again.
            k=at.row_ptr[j];
            for (l=a.col_ptr[k];l<a.col_ptr[k+1];l++) {      Go down column k in
                h=a.row_ptr[l];                                  A.
                temp2[h] += temp[k]*a.val[l];        All terms from temp[k] used here.
            }
        }
        for (j=adat->col_ptr[i];j<adat->col_ptr[i+1];j++) {
        Store temp2 in column of answer.
            k=adat->row_ptr[j];
            adat->val[j]=temp2[k];
            temp2[k]=0.0;                  Restore temp2.
        }
    }
}
```

The final two functions are simple. The `ref` routine returns a reference to the matrix $\mathbf{A}\mathbf{D}\mathbf{A}^T$ stored in the structure for other routines that may need to work with it. And the destructor releases the storage.

sparse.h
```
NRsparseMat & ADAT::ref() {
    return *adat;
}

ADAT::~ADAT() {
    delete adat;
}
```

By the way, if you invoke `ADAT` with *different* matrices $\mathbf{A}$ and $\mathbf{B}^T$, everything will work fine as long as $\mathbf{A}$ and $\mathbf{B}$ have the same nonzero pattern.

In *Numerical Recipes* second edition, we gave a related sparse matrix storage mode in which the diagonal of the matrix is stored first, followed by the off-diagonal elements. We now feel that the added complexity of that scheme is not worthwhile for any of the uses in this book. For a discussion of this and other storage schemes, see [7,8]. To see how to work with the diagonal in the compressed column mode, look at the code for `asolve` at the end of this section.

2.7.6 Conjugate Gradient Method for a Sparse System

So-called *conjugate gradient methods* provide a quite general means for solving the $N \times N$ linear system

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \tag{2.7.29}$$

The attractiveness of these methods for large sparse systems is that they reference $\mathbf{A}$ only through its multiplication of a vector, or the multiplication of its transpose and a vector. As we have seen, these operations can be very efficient for a properly stored sparse matrix. You, the "owner" of the matrix $\mathbf{A}$, can be asked to provide functions that perform these sparse matrix multiplications as efficiently as possible. We, the "grand strategists," supply an abstract base class, `Linbcg` below, that contains the method for solving the set of linear equations, (2.7.29), using your functions.

The simplest, "ordinary" conjugate gradient algorithm [9-11] solves (2.7.29) only in the case that $\mathbf{A}$ is symmetric and positive-definite. It is based on the idea of minimizing the function

$$f(\mathbf{x}) = \tfrac{1}{2}\, \mathbf{x} \cdot \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \cdot \mathbf{x} \tag{2.7.30}$$

This function is minimized when its gradient

$$\nabla f = \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \tag{2.7.31}$$

is zero, which is equivalent to (2.7.29). The minimization is carried out by generating a succession of search directions $\mathbf{p}_k$ and improved minimizers $\mathbf{x}_k$. At each stage a quantity α_k is found that minimizes $f(\mathbf{x}_k + \alpha_k \mathbf{p}_k)$, and $\mathbf{x}_{k+1}$ is set equal to the new point $\mathbf{x}_k + \alpha_k \mathbf{p}_k$. The $\mathbf{p}_k$ and $\mathbf{x}_k$ are built up in such a way that $\mathbf{x}_{k+1}$ is also the minimizer of f over the whole vector space of directions already taken, $\{\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_{k-1}\}$. After N iterations you arrive at the minimizer over the entire vector space, i.e., the solution to (2.7.29).

Later, in §10.8, we will generalize this "ordinary" conjugate gradient algorithm to the minimization of arbitrary nonlinear functions. Here, where our interest is in solving linear, but not necessarily positive-definite or symmetric, equations, a different generalization is important, the *biconjugate gradient method*. This method does not, in general, have a simple connection with function minimization. It constructs four sequences of vectors, $\mathbf{r}_k$, $\bar{\mathbf{r}}_k$, $\mathbf{p}_k$, $\bar{\mathbf{p}}_k$, $k = 0, 1, \ldots$. You supply the initial vectors $\mathbf{r}_0$ and $\bar{\mathbf{r}}_0$, and set $\mathbf{p}_0 = \mathbf{r}_0$, $\bar{\mathbf{p}}_0 = \bar{\mathbf{r}}_0$. Then you carry out the following recurrence:

$$\begin{aligned}
\alpha_k &= \frac{\bar{\mathbf{r}}_k \cdot \mathbf{r}_k}{\bar{\mathbf{p}}_k \cdot \mathbf{A} \cdot \mathbf{p}_k} \\
\mathbf{r}_{k+1} &= \mathbf{r}_k - \alpha_k \mathbf{A} \cdot \mathbf{p}_k \\
\bar{\mathbf{r}}_{k+1} &= \bar{\mathbf{r}}_k - \alpha_k \mathbf{A}^T \cdot \bar{\mathbf{p}}_k \\
\beta_k &= \frac{\bar{\mathbf{r}}_{k+1} \cdot \mathbf{r}_{k+1}}{\bar{\mathbf{r}}_k \cdot \mathbf{r}_k} \\
\mathbf{p}_{k+1} &= \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k \\
\bar{\mathbf{p}}_{k+1} &= \bar{\mathbf{r}}_{k+1} + \beta_k \bar{\mathbf{p}}_k
\end{aligned} \tag{2.7.32}$$

This sequence of vectors satisfies the *biorthogonality* condition

$$\bar{\mathbf{r}}_i \cdot \mathbf{r}_j = \mathbf{r}_i \cdot \bar{\mathbf{r}}_j = 0, \qquad j < i \tag{2.7.33}$$

and the *biconjugacy* condition

$$\bar{\mathbf{p}}_i \cdot \mathbf{A} \cdot \mathbf{p}_j = \mathbf{p}_i \cdot \mathbf{A}^T \cdot \bar{\mathbf{p}}_j = 0, \qquad j < i \tag{2.7.34}$$

There is also a mutual orthogonality,

$$\bar{\mathbf{r}}_i \cdot \mathbf{p}_j = \mathbf{r}_i \cdot \bar{\mathbf{p}}_j = 0, \qquad j < i \tag{2.7.35}$$

The proof of these properties proceeds by straightforward induction [12]. As long as the recurrence does not break down earlier because one of the denominators is zero, it must terminate after $m \le N$ steps with $\mathbf{r}_m = \bar{\mathbf{r}}_m = 0$. This is basically because after at most N steps you run out of new orthogonal directions to the vectors you've already constructed.

To use the algorithm to solve the system (2.7.29), make an initial guess $\mathbf{x}_0$ for the solution. Choose $\mathbf{r}_0$ to be the *residual*

$$\mathbf{r}_0 = \mathbf{b} - \mathbf{A} \cdot \mathbf{x}_0 \tag{2.7.36}$$

and choose $\bar{\mathbf{r}}_0 = \mathbf{r}_0$. Then form the sequence of improved estimates

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k \tag{2.7.37}$$

while carrying out the recurrence (2.7.32). Equation (2.7.37) guarantees that $\mathbf{r}_{k+1}$ from the recurrence is in fact the residual $\mathbf{b} - \mathbf{A} \cdot \mathbf{x}_{k+1}$ corresponding to $\mathbf{x}_{k+1}$. Since $\mathbf{r}_m = 0$, $\mathbf{x}_m$ is the solution to equation (2.7.29).

While there is no guarantee that this whole procedure will not break down or become unstable for general $\mathbf{A}$, in practice this is rare. More importantly, the exact termination in at most N iterations occurs only with exact arithmetic. Roundoff error means that you should

regard the process as a genuinely iterative procedure, to be halted when some appropriate error criterion is met.

The ordinary conjugate gradient algorithm is the special case of the biconjugate gradient algorithm when $\mathbf{A}$ is symmetric, and we choose $\bar{\mathbf{r}}_0 = \mathbf{r}_0$. Then $\bar{\mathbf{r}}_k = \mathbf{r}_k$ and $\bar{\mathbf{p}}_k = \mathbf{p}_k$ for all k; you can omit computing them and halve the work of the algorithm. This conjugate gradient version has the interpretation of minimizing equation (2.7.30). If $\mathbf{A}$ is positive-definite as well as symmetric, the algorithm cannot break down (in theory!). The `solve` routine `Linbcg` below indeed reduces to the ordinary conjugate gradient method if you input a symmetric $\mathbf{A}$, but it does all the redundant computations.

Another variant of the general algorithm corresponds to a symmetric but non-positive-definite $\mathbf{A}$, with the choice $\bar{\mathbf{r}}_0 = \mathbf{A} \cdot \mathbf{r}_0$ instead of $\bar{\mathbf{r}}_0 = \mathbf{r}_0$. In this case $\bar{\mathbf{r}}_k = \mathbf{A} \cdot \mathbf{r}_k$ and $\bar{\mathbf{p}}_k = \mathbf{A} \cdot \mathbf{p}_k$ for all k. This algorithm is thus equivalent to the ordinary conjugate gradient algorithm, but with all dot products $\mathbf{a} \cdot \mathbf{b}$ replaced by $\mathbf{a} \cdot \mathbf{A} \cdot \mathbf{b}$. It is called the *minimum residual* algorithm, because it corresponds to successive minimizations of the function

$$\Phi(\mathbf{x}) = \tfrac{1}{2}\, \mathbf{r} \cdot \mathbf{r} = \tfrac{1}{2}\, |\mathbf{A} \cdot \mathbf{x} - \mathbf{b}|^2 \tag{2.7.38}$$

where the successive iterates $\mathbf{x}_k$ minimize Φ over the same set of search directions $\mathbf{p}_k$ generated in the conjugate gradient method. This algorithm has been generalized in various ways for unsymmetric matrices. The *generalized minimum residual* method (GMRES; see [13,14]) is probably the most robust of these methods.

Note that equation (2.7.38) gives

$$\nabla \Phi(\mathbf{x}) = \mathbf{A}^T \cdot (\mathbf{A} \cdot \mathbf{x} - \mathbf{b}) \tag{2.7.39}$$

For any nonsingular matrix $\mathbf{A}$, $\mathbf{A}^T \cdot \mathbf{A}$ is symmetric and positive-definite. You might therefore be tempted to solve equation (2.7.29) by applying the ordinary conjugate gradient algorithm to the problem

$$(\mathbf{A}^T \cdot \mathbf{A}) \cdot \mathbf{x} = \mathbf{A}^T \cdot \mathbf{b} \tag{2.7.40}$$

Don't! The condition number of the matrix $\mathbf{A}^T \cdot \mathbf{A}$ is the square of the condition number of $\mathbf{A}$ (see §2.6 for definition of condition number). A large condition number both increases the number of iterations required and limits the accuracy to which a solution can be obtained. It is almost always better to apply the biconjugate gradient method to the original matrix $\mathbf{A}$.

So far we have said nothing about the *rate* of convergence of these methods. The ordinary conjugate gradient method works well for matrices that are well-conditioned, i.e., "close" to the identity matrix. This suggests applying these methods to the *preconditioned* form of equation (2.7.29),

$$(\tilde{\mathbf{A}}^{-1} \cdot \mathbf{A}) \cdot \mathbf{x} = \tilde{\mathbf{A}}^{-1} \cdot \mathbf{b} \tag{2.7.41}$$

The idea is that you might already be able to solve your linear system easily for some $\tilde{\mathbf{A}}$ close to $\mathbf{A}$, in which case $\tilde{\mathbf{A}}^{-1} \cdot \mathbf{A} \approx \mathbf{1}$, allowing the algorithm to converge in fewer steps. The matrix $\tilde{\mathbf{A}}$ is called a *preconditioner* [9], and the overall scheme given here is known as the *preconditioned biconjugate gradient method* or *PBCG*. In the code below, the user-supplied routine `atimes` does sparse matrix multiplication by $\mathbf{A}$, while the user-supplied routine `asolve` effects matrix multiplication by the inverse of the preconditioner $\tilde{\mathbf{A}}^{-1}$.

For efficient implementation, the PBCG algorithm introduces an additional set of vectors $\mathbf{z}_k$ and $\bar{\mathbf{z}}_k$ defined by

$$\tilde{\mathbf{A}} \cdot \mathbf{z}_k = \mathbf{r}_k \qquad \text{and} \qquad \tilde{\mathbf{A}}^T \cdot \bar{\mathbf{z}}_k = \bar{\mathbf{r}}_k \tag{2.7.42}$$

and modifies the definitions of α_k, β_k, $\mathbf{p}_k$, and $\bar{\mathbf{p}}_k$ in equation (2.7.32):

$$\begin{aligned} \alpha_k &= \frac{\bar{\mathbf{r}}_k \cdot \mathbf{z}_k}{\bar{\mathbf{p}}_k \cdot \mathbf{A} \cdot \mathbf{p}_k} \\ \beta_k &= \frac{\bar{\mathbf{r}}_{k+1} \cdot \mathbf{z}_{k+1}}{\bar{\mathbf{r}}_k \cdot \mathbf{z}_k} \\ \mathbf{p}_{k+1} &= \mathbf{z}_{k+1} + \beta_k \mathbf{p}_k \\ \bar{\mathbf{p}}_{k+1} &= \bar{\mathbf{z}}_{k+1} + \beta_k \bar{\mathbf{p}}_k \end{aligned} \tag{2.7.43}$$

To use Linbcg, below, you will need to supply routines that solve the auxiliary linear systems (2.7.42). If you have no idea what to use for the preconditioner $\tilde{\mathbf{A}}$, then use the diagonal part of $\mathbf{A}$, or even the identity matrix, in which case the burden of convergence will be entirely on the biconjugate gradient method itself.

Linbcg's routine solve, below, is based on a program originally written by Anne Greenbaum. (See [11] for a different, less sophisticated, implementation.) There are a few wrinkles you should know about.

What constitutes "good" convergence is rather application-dependent. The routine solve therefore provides for four possibilities, selected by setting the flag itol on input. If itol=1, iteration stops when the quantity $|\mathbf{A}\cdot\mathbf{x}-\mathbf{b}|/|\mathbf{b}|$ is less than the input quantity tol. If itol=2, the required criterion is

$$|\tilde{\mathbf{A}}^{-1}\cdot(\mathbf{A}\cdot\mathbf{x}-\mathbf{b})|/|\tilde{\mathbf{A}}^{-1}\cdot\mathbf{b}| < \texttt{tol} \tag{2.7.44}$$

If itol=3, the routine uses its own estimate of the error in $\mathbf{x}$ and requires its magnitude, divided by the magnitude of $\mathbf{x}$, to be less than tol. The setting itol=4 is the same as itol=3, except that the largest (in absolute value) component of the error and largest component of $\mathbf{x}$ are used instead of the vector magnitude (that is, the L_∞ norm instead of the L_2 norm). You may need to experiment to find which of these convergence criteria is best for your problem.

On output, err is the tolerance actually achieved. If the returned count iter does not indicate that the maximum number of allowed iterations itmax was exceeded, then err should be less than tol. If you want to do further iterations, leave all returned quantities as they are and call the routine again. The routine loses its memory of the spanned conjugate gradient subspace between calls, however, so you should not force it to return more often than about every N iterations.

linbcg.h

```
struct Linbcg {
```

Abstract base class for solving sparse linear equations by the preconditioned biconjugate gradient method. To use, declare a derived class in which the methods atimes and asolve are defined for your problem, along with any data that they need. Then call the solve method.

```
    virtual void asolve(VecDoub_I &b, VecDoub_O &x, const Int itrnsp) = 0;
    virtual void atimes(VecDoub_I &x, VecDoub_O &r, const Int itrnsp) = 0;
    void solve(VecDoub_I &b, VecDoub_IO &x, const Int itol, const Doub tol,
        const Int itmax, Int &iter, Doub &err);
    Doub snrm(VecDoub_I &sx, const Int itol);          Utility used by solve.
};

void Linbcg::solve(VecDoub_I &b, VecDoub_IO &x, const Int itol, const Doub tol,
    const Int itmax, Int &iter, Doub &err)
```

Solves $\mathbf{A}\cdot\mathbf{x}=\mathbf{b}$ for x[0..n-1], given b[0..n-1], by the iterative biconjugate gradient method. On input x[0..n-1] should be set to an initial guess of the solution (or all zeros); itol is 1,2,3, or 4, specifying which convergence test is applied (see text); itmax is the maximum number of allowed iterations; and tol is the desired convergence tolerance. On output, x[0..n-1] is reset to the improved solution, iter is the number of iterations actually taken, and err is the estimated error. The matrix $\mathbf{A}$ is referenced only through the user-supplied routines atimes, which computes the product of either $\mathbf{A}$ or its transpose on a vector, and asolve, which solves $\tilde{\mathbf{A}}\cdot\mathbf{x}=\mathbf{b}$ or $\tilde{\mathbf{A}}^T\cdot\mathbf{x}=\mathbf{b}$ for some preconditioner matrix $\tilde{\mathbf{A}}$ (possibly the trivial diagonal part of $\mathbf{A}$). This routine can be called repeatedly, with itmax$\lesssim$n, to monitor how err decreases; or it can be called once with a sufficiently large value of itmax so that convergence to tol is achieved.

```
{
    Doub ak,akden,bk,bkden=1.0,bknum,bnrm,dxnrm,xnrm,zm1nrm,znrm;
    const Doub EPS=1.0e-14;
    Int j,n=b.size();
    VecDoub p(n),pp(n),r(n),rr(n),z(n),zz(n);
    iter=0;                                  Calculate initial residual.
    atimes(x,r,0);                           Input to atimes is x[0..n-1], output is r[0..n-1];
    for (j=0;j<n;j++) {                          the final 0 indicates that the matrix (not its
        r[j]=b[j]-r[j];                          transpose) is to be used.
        rr[j]=r[j];
    }
```

```
//atimes(r,rr,0);
if (itol == 1) {
    bnrm=snrm(b,itol);
    asolve(r,z,0);
}
else if (itol == 2) {
    asolve(b,z,0);
    bnrm=snrm(z,itol);
    asolve(r,z,0);
}
else if (itol == 3 || itol == 4) {
    asolve(b,z,0);
    bnrm=snrm(z,itol);
    asolve(r,z,0);
    znrm=snrm(z,itol);
} else throw("illegal itol in linbcg");
while (iter < itmax) {
    ++iter;
    asolve(rr,zz,1);
    for (bknum=0.0,j=0;j<n;j++) bknum += z[j]*rr[j];
    Calculate coefficient bk and direction vectors p and pp.
    if (iter == 1) {
        for (j=0;j<n;j++) {
            p[j]=z[j];
            pp[j]=zz[j];
        }
    } else {
        bk=bknum/bkden;
        for (j=0;j<n;j++) {
            p[j]=bk*p[j]+z[j];
            pp[j]=bk*pp[j]+zz[j];
        }
    }
    bkden=bknum;
    atimes(p,z,0);
    for (akden=0.0,j=0;j<n;j++) akden += z[j]*pp[j];
    ak=bknum/akden;
    atimes(pp,zz,1);
    for (j=0;j<n;j++) {
        x[j] += ak*p[j];
        r[j] -= ak*z[j];
        rr[j] -= ak*zz[j];
    }
    asolve(r,z,0);
    if (itol == 1)
        err=snrm(r,itol)/bnrm;
    else if (itol == 2)
        err=snrm(z,itol)/bnrm;
    else if (itol == 3 || itol == 4) {
        zm1nrm=znrm;
        znrm=snrm(z,itol);
        if (abs(zm1nrm-znrm) > EPS*znrm) {
            dxnrm=abs(ak)*snrm(p,itol);
            err=znrm/abs(zm1nrm-znrm)*dxnrm;
        } else {
            err=znrm/bnrm;
            continue;
        }
        xnrm=snrm(x,itol);
        if (err <= 0.5*xnrm) err /= xnrm;
        else {
            err=znrm/bnrm;
            continue;
        }
```

Uncomment this line to get the "minimum residual" variant of the algorithm.

Input to asolve is r[0..n-1], output is z[0..n-1]; the final 0 indicates that the matrix $\widetilde{\mathbf{A}}$ (not its transpose) is to be used.

Main loop.

Final 1 indicates use of transpose matrix $\widetilde{\mathbf{A}}^T$.

Calculate coefficient ak, new iterate x, and new residuals r and rr.

Solve $\widetilde{\mathbf{A}} \cdot \mathbf{z} = \mathbf{r}$ and check stopping criterion.

Error may not be accurate, so loop again.

Error may not be accurate, so loop again.

```
        }
        if (err <= tol) break;
    }
}
```

The `solve` routine uses this short utility for computing vector norms:

linbcg.h

```
Doub Linbcg::snrm(VecDoub_I &sx, const Int itol)
```

Compute one of two norms for a vector `sx[0..n-1]`, as signaled by `itol`. Used by `solve`.

```
{
    Int i,isamax,n=sx.size();
    Doub ans;
    if (itol <= 3) {
        ans = 0.0;
        for (i=0;i<n;i++) ans += SQR(sx[i]);        Vector magnitude norm.
        return sqrt(ans);
    } else {
        isamax=0;
        for (i=0;i<n;i++) {                         Largest component norm.
            if (abs(sx[i]) > abs(sx[isamax])) isamax=i;
        }
        return abs(sx[isamax]);
    }
}
```

Here is an example of a derived class that solves $\mathbf{A}\cdot\mathbf{x} = \mathbf{b}$ for a matrix $\mathbf{A}$ in `NRsparseMat`'s compressed column sparse format. A naive diagonal preconditioner is used.

asolve.h

```
struct NRsparseLinbcg : Linbcg {
    NRsparseMat &mat;
    Int n;
    NRsparseLinbcg(NRsparseMat &matrix) : mat(matrix), n(mat.nrows) {}
```

The constructor just binds a reference to your sparse matrix, making it available to `asolve` and `atimes`. To solve for a right-hand side, you call this object's `solve` method, as defined in the base class.

```
    void atimes(VecDoub_I &x, VecDoub_O &r, const Int itrnsp) {
        if (itrnsp) r=mat.atx(x);
        else r=mat.ax(x);
    }
    void asolve(VecDoub_I &b, VecDoub_O &x, const Int itrnsp) {
        Int i,j;
        Doub diag;
        for (i=0;i<n;i++) {
            diag=0.0;
            for (j=mat.col_ptr[i];j<mat.col_ptr[i+1];j++)
                if (mat.row_ind[j] == i) {
                    diag=mat.val[j];
                    break;
                }
            x[i]=(diag != 0.0 ? b[i]/diag : b[i]);
```

The matrix $\tilde{\mathbf{A}}$ is the diagonal part of $\mathbf{A}$. Since the transpose matrix has the same diagonal, the flag `itrnsp` is not used in this example.

```
        }
    }
};
```

For another example of using a class derived from `Linbcg` to solve a sparse matrix problem, see §3.8.

CITED REFERENCES AND FURTHER READING:

Tewarson, R.P. 1973, *Sparse Matrices* (New York: Academic Press).[1]

Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter I.3 (by J.K. Reid).[2]

George, A., and Liu, J.W.H. 1981, *Computer Solution of Large Sparse Positive Definite Systems* (Englewood Cliffs, NJ: Prentice-Hall).[3]

NAG Fortran Library (Oxford, UK: Numerical Algorithms Group), see 2007+, `http://www.nag.co.uk`.[4]

IMSL Math/Library Users Manual (Houston: IMSL Inc.), see 2007+, `http://www.vni.com/products/imsl`.[5]

Eisenstat, S.C., Gursky, M.C., Schultz, M.H., and Sherman, A.H. 1977, *Yale Sparse Matrix Package*, Technical Reports 112 and 114 (Yale University Department of Computer Science).[6]

Bai, Z., Demmel, J., Dongarra, J. Ruhe, A., and van der Vorst, H. (eds.) 2000, *Templates for the Solution of Algebraic Eigenvalue Problems: A Practical Guide* Ch. 10 (Philadelphia: S.I.A.M.). Online at URL in `http://www.cs.ucdavis.edu/~bai/ET/contents.html`.[7]

SPARSKIT, 2007+, at `http://www-users.cs.umn.edu/~saad/software/SPARSKIT/sparskit.html`.[8]

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), Chapters 4 and 10, particularly §10.2–§10.3.[9]

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 8.[10]

Baker, L. 1991, *More C Tools for Scientists and Engineers* (New York: McGraw-Hill).[11]

Fletcher, R. 1976, in *Numerical Analysis Dundee 1975*, Lecture Notes in Mathematics, vol. 506, A. Dold and B Eckmann, eds. (Berlin: Springer), pp. 73–89.[12]

PCGPAK User's Guide (New Haven: Scientific Computing Associates, Inc.).[13]

Saad, Y., and Schulz, M. 1986, *SIAM Journal on Scientific and Statistical Computing*, vol. 7, pp. 856–869.[14]

Ueberhuber, C.W. 1997, *Numerical Computation: Methods, Software, and Analysis*, 2 vols. (Berlin: Springer), Chapter 13.

Bunch, J.R., and Rose, D.J. (eds.) 1976, *Sparse Matrix Computations* (New York: Academic Press).

Duff, I.S., and Stewart, G.W. (eds.) 1979, *Sparse Matrix Proceedings 1978* (Philadelphia: S.I.A.M.).

2.8 Vandermonde Matrices and Toeplitz Matrices

In §2.4 the case of a tridiagonal matrix was treated specially, because that particular type of linear system admits a solution in only of order N operations, rather than of order N^3 for the general linear problem. When such particular types exist, it is important to know about them. Your computational savings, should you ever happen to be working on a problem that involves the right kind of particular type, can be enormous.

This section treats two special types of matrices that can be solved in of order N^2 operations, not as good as tridiagonal, but a lot better than the general case. (Other than the operations count, these two types having nothing in common.) Matrices of the first type, termed *Vandermonde matrices*, occur in some problems having to do with the fitting of polynomials, the reconstruction of distributions from their moments, and also other contexts. In this book, for example, a Vandermonde problem crops up in §3.5. Matrices of the second type, termed *Toeplitz matrices*, tend

to occur in problems involving deconvolution and signal processing. In this book, a Toeplitz problem is encountered in §13.7.

These are not the only special types of matrices worth knowing about. The *Hilbert matrices*, whose components are of the form $a_{ij} = 1/(i + j + 1)$, $i, j = 0, \ldots, N-1$, can be inverted by an exact integer algorithm and are very difficult to invert in any other way, since they are notoriously ill-conditioned (see [1] for details). The Sherman-Morrison and Woodbury formulas, discussed in §2.7, can sometimes be used to convert new special forms into old ones. Reference [2] gives some other special forms. We have not found these additional forms to arise as frequently as the two that we now discuss.

2.8.1 Vandermonde Matrices

A Vandermonde matrix of size $N \times N$ is completely determined by N arbitrary numbers $x_0, x_1, \ldots, x_{N-1}$, in terms of which its N^2 components are the integer powers x_i^j, $i, j = 0, \ldots, N-1$. Evidently there are two possible such forms, depending on whether we view the i's as rows and j's as columns, or vice versa. In the former case, we get a linear system of equations that looks like this,

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{N-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{N-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{N-1} & x_{N-1}^2 & \cdots & x_{N-1}^{N-1} \end{bmatrix} \cdot \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{N-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{bmatrix} \tag{2.8.1}$$

Performing the matrix multiplication, you will see that this equation solves for the unknown coefficients c_i that fit a polynomial to the N pairs of abscissas and ordinates (x_j, y_j). Precisely this problem will arise in §3.5, and the routine given there will solve (2.8.1) by the method that we are about to describe.

The alternative identification of rows and columns leads to the set of equations

$$\begin{bmatrix} 1 & 1 & \cdots & 1 \\ x_0 & x_1 & \cdots & x_{N-1} \\ x_0^2 & x_1^2 & \cdots & x_{N-1}^2 \\ & & \cdots & \\ x_0^{N-1} & x_1^{N-1} & \cdots & x_{N-1}^{N-1} \end{bmatrix} \cdot \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \cdots \\ w_{N-1} \end{bmatrix} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ \cdots \\ q_{N-1} \end{bmatrix} \tag{2.8.2}$$

Write this out and you will see that it relates to the *problem of moments*: Given the values of N points x_i, find the unknown weights w_i, assigned so as to match the given values q_j of the first N moments. (For more on this problem, consult [3].) The routine given in this section solves (2.8.2).

The method of solution of both (2.8.1) and (2.8.2) is closely related to Lagrange's polynomial interpolation formula, which we will not formally meet until §3.2. Notwithstanding, the following derivation should be comprehensible:

Let $P_j(x)$ be the polynomial of degree $N-1$ defined by

$$P_j(x) = \prod_{\substack{n=0 \\ n \neq j}}^{N-1} \frac{x - x_n}{x_j - x_n} = \sum_{k=0}^{N-1} A_{jk} x^k \tag{2.8.3}$$

Here the meaning of the last equality is to define the components of the matrix A_{ij} as the coefficients that arise when the product is multiplied out and like terms collected.

The polynomial $P_j(x)$ is a function of x generally. But you will notice that it is specifically designed so that it takes on a value of zero at all x_i with $i \neq j$ and has a value of unity at $x = x_j$. In other words,

$$P_j(x_i) = \delta_{ij} = \sum_{k=0}^{N-1} A_{jk} x_i^k \tag{2.8.4}$$

But (2.8.4) says that A_{jk} is exactly the inverse of the matrix of components x_i^k, which appears in (2.8.2), with the subscript as the column index. Therefore the solution of (2.8.2) is just that matrix inverse times the right-hand side,

$$w_j = \sum_{k=0}^{N-1} A_{jk} q_k \tag{2.8.5}$$

As for the transpose problem (2.8.1), we can use the fact that the inverse of the transpose is the transpose of the inverse, so

$$c_j = \sum_{k=0}^{N-1} A_{kj} y_k \tag{2.8.6}$$

The routine in §3.5 implements this.

It remains to find a good way of multiplying out the monomial terms in (2.8.3), in order to get the components of A_{jk}. This is essentially a bookkeeping problem, and we will let you read the routine itself to see how it can be solved. One trick is to define a master $P(x)$ by

$$P(x) \equiv \prod_{n=0}^{N-1} (x - x_n) \tag{2.8.7}$$

work out its coefficients, and then obtain the numerators and denominators of the specific P_j's via synthetic division by the one supernumerary term. (See §5.1 for more on synthetic division.) Since each such division is only a process of order N, the total procedure is of order N^2.

You should be warned that Vandermonde systems are notoriously ill-conditioned, by their very nature. (As an aside anticipating §5.8, the reason is the same as that which makes Chebyshev fitting so impressively accurate: There exist high-order polynomials that are very good uniform fits to zero. Hence roundoff error can introduce rather substantial coefficients of the leading terms of these polynomials.) It is a good idea always to compute Vandermonde problems in double precision or higher.

The routine for (2.8.2) that follows is due to G.B. Rybicki.

```
void vander(VecDoub_I &x, VecDoub_O &w, VecDoub_I &q)                    vander.h
Solves the Vandermonde linear system sum_{i=0}^{N-1} x_i^k w_i = q_k (k = 0,...,N-1). Input consists
of the vectors x[0..n-1] and q[0..n-1]; the vector w[0..n-1] is output.
{
    Int i,j,k,n=q.size();
    Doub b,s,t,xx;
    VecDoub c(n);
    if (n == 1) w[0]=q[0];
    else {
        for (i=0;i<n;i++) c[i]=0.0;          Initialize array.
        c[n-1] = -x[0];                      Coefficients of the master polynomial are found
        for (i=1;i<n;i++) {                      by recursion.
            xx = -x[i];
            for (j=(n-1-i);j<(n-1);j++) c[j] += xx*c[j+1];
            c[n-1] += xx;
        }
        for (i=0;i<n;i++) {                  Each subfactor in turn
```

```
            xx=x[i];
            t=b=1.0;
            s=q[n-1];
            for (k=n-1;k>0;k--) {          is synthetically divided,
                b=c[k]+xx*b;
                s += q[k-1]*b;             matrix-multiplied by the right-hand side,
                t=xx*t+b;
            }
            w[i]=s/t;                      and supplied with a denominator.
        }
    }
}
```

2.8.2 Toeplitz Matrices

An $N \times N$ Toeplitz matrix is specified by giving $2N-1$ numbers R_k, where the index k ranges over $k = -N+1, \ldots, -1, 0, 1, \ldots, N-1$. Those numbers are then emplaced as matrix elements constant along the (upper-left to lower-right) diagonals of the matrix:

$$\begin{bmatrix} R_0 & R_{-1} & R_{-2} & \cdots & R_{-(N-2)} & R_{-(N-1)} \\ R_1 & R_0 & R_{-1} & \cdots & R_{-(N-3)} & R_{-(N-2)} \\ R_2 & R_1 & R_0 & \cdots & R_{-(N-4)} & R_{-(N-3)} \\ \cdots & & & \cdots & & \\ R_{N-2} & R_{N-3} & R_{N-4} & \cdots & R_0 & R_{-1} \\ R_{N-1} & R_{N-2} & R_{N-3} & \cdots & R_1 & R_0 \end{bmatrix} \tag{2.8.8}$$

The linear Toeplitz problem can thus be written as

$$\sum_{j=0}^{N-1} R_{i-j} x_j = y_i \qquad (i = 0, \ldots, N-1) \tag{2.8.9}$$

where the x_j's, $j = 0, \ldots, N-1$, are the unknowns to be solved for.

The Toeplitz matrix is symmetric if $R_k = R_{-k}$ for all k. Levinson [4] developed an algorithm for fast solution of the symmetric Toeplitz problem, by a *bordering method*, that is, a recursive procedure that solves the $(M+1)$-dimensional Toeplitz problem

$$\sum_{j=0}^{M} R_{i-j} x_j^{(M)} = y_i \qquad (i = 0, \ldots, M) \tag{2.8.10}$$

in turn for $M = 0, 1, \ldots$ until $M = N-1$, the desired result, is finally reached. The vector $x_j^{(M)}$ is the result at the Mth stage and becomes the desired answer only when $N-1$ is reached.

Levinson's method is well documented in standard texts (e.g., [5]). The useful fact that the method generalizes to the *nonsymmetric* case seems to be less well known. At some risk of excessive detail, we therefore give a derivation here, due to G.B. Rybicki.

In following a recursion from step M to step $M+1$ we find that our developing solution $x^{(M)}$ changes in this way:

$$\sum_{j=0}^{M} R_{i-j} x_j^{(M)} = y_i \qquad i = 0, \ldots, M \tag{2.8.11}$$

becomes

$$\sum_{j=0}^{M} R_{i-j} x_j^{(M+1)} + R_{i-(M+1)} x_{M+1}^{(M+1)} = y_i \qquad i = 0, \ldots, M+1 \tag{2.8.12}$$

By eliminating y_i we find

$$\sum_{j=0}^{M} R_{i-j}\left(\frac{x_j^{(M)} - x_j^{(M+1)}}{x_{M+1}^{(M+1)}}\right) = R_{i-(M+1)} \qquad i = 0, \ldots, M \tag{2.8.13}$$

or by letting $i \to M - i$ and $j \to M - j$,

$$\sum_{j=0}^{M} R_{j-i} G_j^{(M)} = R_{-(i+1)} \tag{2.8.14}$$

where

$$G_j^{(M)} \equiv \frac{x_{M-j}^{(M)} - x_{M-j}^{(M+1)}}{x_{M+1}^{(M+1)}} \tag{2.8.15}$$

To put this another way,

$$x_{M-j}^{(M+1)} = x_{M-j}^{(M)} - x_{M+1}^{(M+1)} G_j^{(M)} \qquad j = 0, \ldots, M \tag{2.8.16}$$

Thus, if we can use recursion to find the order M quantities $x^{(M)}$ and $G^{(M)}$ *and* the single order $M + 1$ quantity $x_{M+1}^{(M+1)}$, then all of the other $x_j^{(M+1)}$'s will follow. Fortunately, the quantity $x_{M+1}^{(M+1)}$ follows from equation (2.8.12) with $i = M + 1$,

$$\sum_{j=0}^{M} R_{M+1-j} x_j^{(M+1)} + R_0 x_{M+1}^{(M+1)} = y_{M+1} \tag{2.8.17}$$

For the unknown order $M + 1$ quantities $x_j^{(M+1)}$ we can substitute the previous order quantities in G since

$$G_{M-j}^{(M)} = \frac{x_j^{(M)} - x_j^{(M+1)}}{x_{M+1}^{(M+1)}} \tag{2.8.18}$$

The result of this operation is

$$x_{M+1}^{(M+1)} = \frac{\sum_{j=0}^{M} R_{M+1-j} x_j^{(M)} - y_{M+1}}{\sum_{j=0}^{M} R_{M+1-j} G_{M-j}^{(M)} - R_0} \tag{2.8.19}$$

The only remaining problem is to develop a recursion relation for G. Before we do that, however, we should point out that there are actually two distinct sets of solutions to the original linear problem for a nonsymmetric matrix, namely right-hand solutions (which we have been discussing) and left-hand solutions z_i. The formalism for the left-hand solutions differs only in that we deal with the equations

$$\sum_{j=0}^{M} R_{j-i} z_j^{(M)} = y_i \qquad i = 0, \ldots, M \tag{2.8.20}$$

Then, the same sequence of operations on this set leads to

$$\sum_{j=0}^{M} R_{i-j} H_j^{(M)} = R_{i+1} \tag{2.8.21}$$

where

$$H_j^{(M)} \equiv \frac{z_{M-j}^{(M)} - z_{M-j}^{(M+1)}}{z_{M+1}^{(M+1)}} \tag{2.8.22}$$

(compare with 2.8.14 – 2.8.15). The reason for mentioning the left-hand solutions now is that, by equation (2.8.21), the H_j's satisfy exactly the same equation as the x_j's except for the substitution $y_i \to R_{i+1}$ on the right-hand side. Therefore we can quickly deduce from equation (2.8.19) that

$$H_{M+1}^{(M+1)} = \frac{\sum_{j=0}^{M} R_{M+1-j} H_j^{(M)} - R_{M+2}}{\sum_{j=0}^{M} R_{M+1-j} G_{M-j}^{(M)} - R_0} \tag{2.8.23}$$

By the same token, G satisfies the same equation as z, except for the substitution $y_i \to R_{-(i+1)}$. This gives

$$G_{M+1}^{(M+1)} = \frac{\sum_{j=0}^{M} R_{j-M-1} G_j^{(M)} - R_{-M-2}}{\sum_{j=0}^{M} R_{j-M-1} H_{M-j}^{(M)} - R_0} \tag{2.8.24}$$

The same "morphism" also turns equation (2.8.16), and its partner for z, into the final equations

$$\begin{aligned} G_j^{(M+1)} &= G_j^{(M)} - G_{M+1}^{(M+1)} H_{M-j}^{(M)} \\ H_j^{(M+1)} &= H_j^{(M)} - H_{M+1}^{(M+1)} G_{M-j}^{(M)} \end{aligned} \tag{2.8.25}$$

Now, starting with the initial values

$$x_0^{(0)} = y_0/R_0 \qquad G_0^{(0)} = R_{-1}/R_0 \qquad H_0^{(0)} = R_1/R_0 \tag{2.8.26}$$

we can recurse away. At each stage M we use equations (2.8.23) and (2.8.24) to find $H_{M+1}^{(M+1)}$, $G_{M+1}^{(M+1)}$, and then equation (2.8.25) to find the other components of $H^{(M+1)}, G^{(M+1)}$. From there the vectors $x^{(M+1)}$ and/or $z^{(M+1)}$ are easily calculated.

The program below does this. It incorporates the second equation in (2.8.25) in the form

$$H_{M-j}^{(M+1)} = H_{M-j}^{(M)} - H_{M+1}^{(M+1)} G_j^{(M)} \tag{2.8.27}$$

so that the computation can be done "in place."

Notice that the above algorithm fails if $R_0 = 0$. In fact, because the bordering method does not allow pivoting, the algorithm will fail if any of the diagonal principal minors of the original Toeplitz matrix vanish. (Compare with discussion of the tridiagonal algorithm in §2.4.) If the algorithm fails, your matrix is not necessarily singular — you might just have to solve your problem by a slower and more general algorithm such as LU decomposition with pivoting.

The routine that implements equations (2.8.23) – (2.8.27) is also due to Rybicki. Note that the routine's `r[n-1+j]` is equal to R_j above, so that subscripts on the `r` array vary from 0 to $2N - 2$.

toeplz.h

```
void toeplz(VecDoub_I &r, VecDoub_O &x, VecDoub_I &y)
```

Solves the Toeplitz system $\sum_{j=0}^{N-1} R_{(N-1+i-j)} x_j = y_i$ $(i = 0, \ldots, N-1)$. The Toeplitz matrix need not be symmetric. `y[0..n-1]` and `r[0..2*n-2]` are input arrays; `x[0..n-1]` is the output array.

```
{
    Int j,k,m,m1,m2,n1,n=y.size();
    Doub pp,pt1,pt2,qq,qt1,qt2,sd,sgd,sgn,shn,sxn;
    n1=n-1;
```

```
    if (r[n1] == 0.0) throw("toeplz-1 singular principal minor");
    x[0]=y[0]/r[n1];                    Initialize for the recursion.
    if (n1 == 0) return;
    VecDoub g(n1),h(n1);
    g[0]=r[n1-1]/r[n1];
    h[0]=r[n1+1]/r[n1];
    for (m=0;m<n;m++) {                 Main loop over the recursion.
        m1=m+1;
        sxn = -y[m1];                   Compute numerator and denominator for x from eq.
        sd = -r[n1];                        (2.8.19),
        for (j=0;j<m+1;j++) {
            sxn += r[n1+m1-j]*x[j];
            sd += r[n1+m1-j]*g[m-j];
        }
        if (sd == 0.0) throw("toeplz-2 singular principal minor");
        x[m1]=sxn/sd;                   whence x.
        for (j=0;j<m+1;j++)             Eq. (2.8.16).
            x[j] -= x[m1]*g[m-j];
        if (m1 == n1) return;
        sgn = -r[n1-m1-1];              Compute numerator and denominator for G and H,
        shn = -r[n1+m1+1];                  eqs. (2.8.24) and (2.8.23),
        sgd = -r[n1];
        for (j=0;j<m+1;j++) {
            sgn += r[n1+j-m1]*g[j];
            shn += r[n1+m1-j]*h[j];
            sgd += r[n1+j-m1]*h[m-j];
        }
        if (sgd == 0.0) throw("toeplz-3 singular principal minor");
        g[m1]=sgn/sgd;                  whence G and H.
        h[m1]=shn/sd;
        k=m;
        m2=(m+2) >> 1;
        pp=g[m1];
        qq=h[m1];
        for (j=0;j<m2;j++) {
            pt1=g[j];
            pt2=g[k];
            qt1=h[j];
            qt2=h[k];
            g[j]=pt1-pp*qt2;
            g[k]=pt2-pp*qt1;
            h[j]=qt1-qq*pt2;
            h[k--]=qt2-qq*pt1;
        }
    }                                   Back for another recurrence.
    throw("toeplz - should not arrive here!");
}
```

If you are in the business of solving *very* large Toeplitz systems, you should find out about so-called "new, fast" algorithms, which require only on the order of $N(\log N)^2$ operations, compared to N^2 for Levinson's method. These methods are too complicated to include here. Papers by Bunch [6] and de Hoog [7] will give entry to the literature.

CITED REFERENCES AND FURTHER READING:

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), Chapter 5 [also treats some other special forms].

Forsythe, G.E., and Moler, C.B. 1967, *Computer Solution of Linear Algebraic Systems* (Englewood Cliffs, NJ: Prentice-Hall), §19.[1]

Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).[2]

von Mises, R. 1964, *Mathematical Theory of Probability and Statistics* (New York: Academic Press), pp. 394ff.[3]

Levinson, N., Appendix B of N. Wiener, 1949, *Extrapolation, Interpolation and Smoothing of Stationary Time Series* (New York: Wiley).[4]

Robinson, E.A., and Treitel, S. 1980, *Geophysical Signal Analysis* (Englewood Cliffs, NJ: Prentice-Hall), pp. 163ff.[5]

Bunch, J.R. 1985, "Stability of Methods for Solving Toeplitz Systems of Equations," *SIAM Journal on Scientific and Statistical Computing*, vol. 6, pp. 349–364.[6]

de Hoog, F. 1987, "A New Algorithm for Solving Toeplitz Systems of Equations," *Linear Algebra and Its Applications*, vol. 88/89, pp. 123–138.[7]

2.9 Cholesky Decomposition

If a square matrix $\mathbf{A}$ happens to be symmetric and positive-definite, then it has a special, more efficient, triangular decomposition. *Symmetric* means that $a_{ij} = a_{ji}$ for $i, j = 0, \ldots, N-1$, while *positive-definite* means that

$$\mathbf{v} \cdot \mathbf{A} \cdot \mathbf{v} > 0 \quad \text{for all vectors } \mathbf{v} \tag{2.9.1}$$

(In Chapter 11 we will see that positive-definite has the equivalent interpretation that $\mathbf{A}$ has all positive eigenvalues.) While symmetric, positive-definite matrices are rather special, they occur quite frequently in some applications, so their special factorization, called *Cholesky decomposition*, is good to know about. When you can use it, Cholesky decomposition is about a factor of two faster than alternative methods for solving linear equations.

Instead of seeking arbitrary lower and upper triangular factors $\mathbf{L}$ and $\mathbf{U}$, Cholesky decomposition constructs a lower triangular matrix $\mathbf{L}$ whose transpose $\mathbf{L}^T$ can itself serve as the upper triangular part. In other words we replace equation (2.3.1) by

$$\mathbf{L} \cdot \mathbf{L}^T = \mathbf{A} \tag{2.9.2}$$

This factorization is sometimes referred to as "taking the square root" of the matrix $\mathbf{A}$, though, because of the transpose, it is not literally that. The components of $\mathbf{L}^T$ are of course related to those of $\mathbf{L}$ by

$$L^T_{ij} = L_{ji} \tag{2.9.3}$$

Writing out equation (2.9.2) in components, one readily obtains the analogs of equations (2.3.12) – (2.3.13),

$$L_{ii} = \left(a_{ii} - \sum_{k=0}^{i-1} L_{ik}^2 \right)^{1/2} \tag{2.9.4}$$

and

$$L_{ji} = \frac{1}{L_{ii}} \left(a_{ij} - \sum_{k=0}^{i-1} L_{ik} L_{jk} \right) \qquad j = i+1, i+2, \ldots, N-1 \tag{2.9.5}$$

If you apply equations (2.9.4) and (2.9.5) in the order $i = 0, 1, \ldots, N-1$, you will see that the L's that occur on the right-hand side are already determined by the time they are needed. Also, only components a_{ij} with $j \geq i$ are referenced. (Since $\mathbf{A}$ is symmetric, these have complete information.) If storage is at a premium, it is possible to have the factor $\mathbf{L}$ overwrite the subdiagonal (lower triangular but not including the diagonal) part of $\mathbf{A}$, preserving the input upper triangular values of $\mathbf{A}$; one extra vector of length N is then needed to store the diagonal part of $\mathbf{L}$. The operations count is $N^3/6$ executions of the inner loop (consisting of one multiply and one subtract), with also N square roots. As already mentioned, this is about a factor 2 better than LU decomposition of $\mathbf{A}$ (where its symmetry would be ignored).

You might wonder about pivoting. The pleasant answer is that Cholesky decomposition is extremely stable numerically, without any pivoting at all. Failure of the decomposition simply indicates that the matrix $\mathbf{A}$ (or, with roundoff error, another very nearby matrix) is not positive-definite. In fact, this is an efficient way to test *whether* a symmetric matrix is positive-definite. (In this application, you may want to replace the `throw` in the code below with some less drastic signaling method.)

By now you should be familiar with, if not bored by, our conventions for objects implementing decomposition methods, so we list the object `Cholesky` as a single big mouthful. The methods `elmult` and `elsolve` perform manipulations using the matrix $\mathbf{L}$. The first multiplies $\mathbf{L} \cdot \mathbf{y} = \mathbf{c}$ for a given $\mathbf{y}$, returning $\mathbf{c}$. The second solves this same equation, given $\mathbf{c}$ and returning $\mathbf{y}$. These manipulations are useful in contexts such as multivariate Gaussians (§7.4 and §16.5) and in the analysis of covariance matrices (§15.6).

cholesky.h

```
struct Cholesky{
```

Object for Cholesky decomposition of a matrix $\mathbf{A}$, and related functions.

```
    Int n;
    MatDoub el;                    Stores the decomposition.
    Cholesky(MatDoub_I &a) : n(a.nrows()), el(a) {
```

Constructor. Given a positive-definite symmetric matrix `a[0..n-1][0..n-1]`, construct and store its Cholesky decomposition, $\mathbf{A} = \mathbf{L} \cdot \mathbf{L}^T$.

```
        Int i,j,k;
        VecDoub tmp;
        Doub sum;
        if (el.ncols() != n) throw("need square matrix");
        for (i=0;i<n;i++) {
            for (j=i;j<n;j++) {
                for (sum=el[i][j],k=i-1;k>=0;k--) sum -= el[i][k]*el[j][k];
                if (i == j) {
                    if (sum <= 0.0)          A, with rounding errors, is not positive-definite.
                        throw("Cholesky failed");
                    el[i][i]=sqrt(sum);
                } else el[j][i]=sum/el[i][i];
            }
        }
        for (i=0;i<n;i++) for (j=0;j<i;j++) el[j][i] = 0.;
    }
    void solve(VecDoub_I &b, VecDoub_O &x) {
```

Solve the set of `n` linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$, where `a` is a positive-definite symmetric matrix whose Cholesky decomposition has been stored. `b[0..n-1]` is input as the right-hand side vector. The solution vector is returned in `x[0..n-1]`.

```
        Int i,k;
        Doub sum;
        if (b.size() != n || x.size() != n) throw("bad lengths in Cholesky");
        for (i=0;i<n;i++) {              Solve L · y = b, storing y in x.
            for (sum=b[i],k=i-1;k>=0;k--) sum -= el[i][k]*x[k];
            x[i]=sum/el[i][i];
        }
        for (i=n-1;i>=0;i--) {           Solve L^T · x = y.
            for (sum=x[i],k=i+1;k<n;k++) sum -= el[k][i]*x[k];
            x[i]=sum/el[i][i];
        }
    }
    void elmult(VecDoub_I &y, VecDoub_O &b) {
```

Multiply $\mathbf{L} \cdot \mathbf{y} = \mathbf{b}$, where $\mathbf{L}$ is the lower triangular matrix in the stored Cholesky decomposition. `y[0..n-1]` is input. The result is returned in `b[0..n-1]`.

```
        Int i,j;
        if (b.size() != n || y.size() != n) throw("bad lengths");
        for (i=0;i<n;i++) {
            b[i] = 0.;
            for (j=0;j<=i;j++) b[i] += el[i][j]*y[j];
        }
    }
```

```
    void elsolve(VecDoub_I &b, VecDoub_O &y) {
```
Solve $\mathbf{L} \cdot \mathbf{y} = \mathbf{b}$, where $\mathbf{L}$ is the lower triangular matrix in the stored Cholesky decomposition. `b[0..n-1]` is input as the right-hand side vector. The solution vector is returned in `y[0..n-1]`.
```
        Int i,j;
        Doub sum;
        if (b.size() != n || y.size() != n) throw("bad lengths");
        for (i=0;i<n;i++) {
            for (sum=b[i],j=0; j<i; j++) sum -= el[i][j]*y[j];
            y[i] = sum/el[i][i];
        }
    }
    void inverse(MatDoub_O &ainv) {
```
Set `ainv[0..n-1][0..n-1]` to the matrix inverse of $\mathbf{A}$, the matrix whose Cholesky decomposition has been stored.
```
        Int i,j,k;
        Doub sum;
        ainv.resize(n,n);
        for (i=0;i<n;i++) for (j=0;j<=i;j++){
            sum = (i==j? 1. : 0.);
            for (k=i-1;k>=j;k--) sum -= el[i][k]*ainv[j][k];
            ainv[j][i]= sum/el[i][i];
        }
        for (i=n-1;i>=0;i--) for (j=0;j<=i;j++){
            sum = (i<j? 0. : ainv[j][i]);
            for (k=i+1;k<n;k++) sum -= el[k][i]*ainv[j][k];
            ainv[i][j] = ainv[j][i] = sum/el[i][i];
        }
    }
    Doub logdet() {
```
Return the logarithm of the determinant of $\mathbf{A}$, the matrix whose Cholesky decomposition has been stored.
```
        Doub sum = 0.;
        for (Int i=0; i<n; i++) sum += log(el[i][i]);
        return 2.*sum;
    }
};
```

CITED REFERENCES AND FURTHER READING:

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I/1.

Gill, P.E., Murray, W., and Wright, M.H. 1991, *Numerical Linear Algebra and Optimization*, vol. 1 (Redwood City, CA: Addison-Wesley), §4.9.2.

Dahlquist, G., and Bjorck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §5.3.5.

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §4.2.

2.10 QR Decomposition

There is another matrix factorization that is sometimes very useful, the so-called QR *decomposition*,

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{R} \tag{2.10.1}$$

Here $\mathbf{R}$ is upper triangular, while $\mathbf{Q}$ is orthogonal, that is,

$$\mathbf{Q}^T \cdot \mathbf{Q} = \mathbf{1} \tag{2.10.2}$$

where $\mathbf{Q}^T$ is the transpose matrix of $\mathbf{Q}$. Although the decomposition exists for a general rectangular matrix, we shall restrict our treatment to the case when all the matrices are square, with dimensions $N \times N$.

Like the other matrix factorizations we have met (LU, SVD, Cholesky), QR decomposition can be used to solve systems of linear equations. To solve

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \tag{2.10.3}$$

first form $\mathbf{Q}^T \cdot \mathbf{b}$ and then solve

$$\mathbf{R} \cdot \mathbf{x} = \mathbf{Q}^T \cdot \mathbf{b} \tag{2.10.4}$$

by backsubstitution. Since QR decomposition involves about twice as many operations as LU decomposition, it is not used for typical systems of linear equations. However, we will meet special cases where QR is the method of choice.

The standard algorithm for the QR decomposition involves successive Householder transformations (to be discussed later in §11.3). We write a Householder matrix in the form $\mathbf{1} - \mathbf{u} \otimes \mathbf{u}/c$, where $c = \frac{1}{2}\mathbf{u} \cdot \mathbf{u}$. An appropriate Householder matrix applied to a given matrix can zero all elements in a column of the matrix situated below a chosen element. Thus we arrange for the first Householder matrix $\mathbf{Q}_0$ to zero all elements in column 0 of $\mathbf{A}$ below the zeroth element. Similarly, $\mathbf{Q}_1$ zeroes all elements in column 1 below element 1, and so on up to $\mathbf{Q}_{n-2}$. Thus

$$\mathbf{R} = \mathbf{Q}_{n-2} \cdots \mathbf{Q}_0 \cdot \mathbf{A} \tag{2.10.5}$$

Since the Householder matrices are orthogonal,

$$\mathbf{Q} = (\mathbf{Q}_{n-2} \cdots \mathbf{Q}_0)^{-1} = \mathbf{Q}_0 \cdots \mathbf{Q}_{n-2} \tag{2.10.6}$$

In many applications $\mathbf{Q}$ is not needed explicitly, and it is sufficient to store only the factored form (2.10.6). (We do, however, store $\mathbf{Q}$, or rather its transpose, in the code below.) Pivoting is not usually necessary unless the matrix $\mathbf{A}$ is very close to singular. A general QR algorithm for rectangular matrices including pivoting is given in [1]. For square matrices and without pivoting, an implementation is as follows:

qrdcmp.h

```
struct QRdcmp {
```

Object for QR decomposition of a matrix $\mathbf{A}$, and related functions.

```
    Int n;
    MatDoub qt, r;                                  Stored Q^T and R.
    Bool sing;                                      Indicates whether A is singular.
    QRdcmp(MatDoub_I &a);                           Constructor from A.
    void solve(VecDoub_I &b, VecDoub_O &x);         Solve A·x = b for x.
    void qtmult(VecDoub_I &b, VecDoub_O &x);        Multiply Q^T·b = x.
    void rsolve(VecDoub_I &b, VecDoub_O &x);        Solve R·x = b for x.
    void update(VecDoub_I &u, VecDoub_I &v);        See next subsection.
    void rotate(const Int i, const Doub a, const Doub b);      Used by update.
};
```

As usual, the constructor performs the actual decomposition:

qrdcmp.h

```
QRdcmp::QRdcmp(MatDoub_I &a)
    : n(a.nrows()), qt(n,n), r(a), sing(false) {
```

Construct the QR decomposition of `a[0..n-1][0..n-1]`. The upper triangular matrix $\mathbf{R}$ and the transpose of the orthogonal matrix $\mathbf{Q}$ are stored. `sing` is set to `true` if a singularity is encountered during the decomposition, but the decomposition is still completed in this case; otherwise it is set to `false`.

```
    Int i,j,k;
    VecDoub c(n), d(n);
    Doub scale,sigma,sum,tau;
```

```
    for (k=0;k<n-1;k++) {
        scale=0.0;
        for (i=k;i<n;i++) scale=MAX(scale,abs(r[i][k]));
        if (scale == 0.0) {                    Singular case.
            sing=true;
            c[k]=d[k]=0.0;
        } else {                               Form Q_k and Q_k · A.
            for (i=k;i<n;i++) r[i][k] /= scale;
            for (sum=0.0,i=k;i<n;i++) sum += SQR(r[i][k]);
            sigma=SIGN(sqrt(sum),r[k][k]);
            r[k][k] += sigma;
            c[k]=sigma*r[k][k];
            d[k] = -scale*sigma;
            for (j=k+1;j<n;j++) {
                for (sum=0.0,i=k;i<n;i++) sum += r[i][k]*r[i][j];
                tau=sum/c[k];
                for (i=k;i<n;i++) r[i][j] -= tau*r[i][k];
            }
        }
    }
    d[n-1]=r[n-1][n-1];
    if (d[n-1] == 0.0) sing=true;
    for (i=0;i<n;i++) {                        Form Q^T explicitly.
        for (j=0;j<n;j++) qt[i][j]=0.0;
        qt[i][i]=1.0;
    }
    for (k=0;k<n-1;k++) {
        if (c[k] != 0.0) {
            for (j=0;j<n;j++) {
                sum=0.0;
                for (i=k;i<n;i++)
                    sum += r[i][k]*qt[i][j];
                sum /= c[k];
                for (i=k;i<n;i++)
                    qt[i][j] -= sum*r[i][k];
            }
        }
    }
    for (i=0;i<n;i++) {                        Form R explicitly.
        r[i][i]=d[i];
        for (j=0;j<i;j++) r[i][j]=0.0;
    }
}
```

The next set of member functions is used to solve linear systems. In many applications only the part (2.10.4) of the algorithm is needed, so we put in separate routines the multiplication $\mathbf{Q}^T \cdot \mathbf{b}$ and the backsubstitution on $\mathbf{R}$.

qrdcmp.h

```
void QRdcmp::solve(VecDoub_I &b, VecDoub_O &x) {
```

Solve the set of `n` linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. `b[0..n-1]` is input as the right-hand side vector, and `x[0..n-1]` is returned as the solution vector.

```
    qtmult(b,x);                   Form Q^T · b.
    rsolve(x,x);                   Solve R · x = Q^T · b.
}

void QRdcmp::qtmult(VecDoub_I &b, VecDoub_O &x) {
```

Multiply $\mathbf{Q}^T \cdot \mathbf{b}$ and put the result in $\mathbf{x}$. Since $\mathbf{Q}$ is orthogonal, this is equivalent to solving $\mathbf{Q} \cdot \mathbf{x} = \mathbf{b}$ for $\mathbf{x}$.

```
    Int i,j;
    Doub sum;
    for (i=0;i<n;i++) {
        sum = 0.;
```

```
        for (j=0;j<n;j++) sum += qt[i][j]*b[j];
        x[i] = sum;
    }
}

void QRdcmp::rsolve(VecDoub_I &b, VecDoub_O &x) {
```

Solve the triangular set of `n` linear equations $\mathbf{R} \cdot \mathbf{x} = \mathbf{b}$. `b[0..n-1]` is input as the right-hand side vector, and `x[0..n-1]` is returned as the solution vector.

```
    Int i,j;
    Doub sum;
    if (sing) throw("attempting solve in a singular QR");
    for (i=n-1;i>=0;i--) {
        sum=b[i];
        for (j=i+1;j<n;j++) sum -= r[i][j]*x[j];
        x[i]=sum/r[i][i];
    }
}
```

See [2] for details on how to use QR decomposition for constructing orthogonal bases, and for solving least-squares problems. (We prefer to use SVD, §2.6, for these purposes, because of its greater diagnostic capability in pathological cases.)

2.10.1 Updating a QR decomposition

Some numerical algorithms involve solving a succession of linear systems each of which differs only slightly from its predecessor. Instead of doing $O(N^3)$ operations each time to solve the equations from scratch, one can often update a matrix factorization in $O(N^2)$ operations and use the new factorization to solve the next set of linear equations. The LU decomposition is complicated to update because of pivoting. However, QR turns out to be quite simple for a very common kind of update,

$$\mathbf{A} \to \mathbf{A} + \mathbf{s} \otimes \mathbf{t} \tag{2.10.7}$$

(compare equation 2.7.1). In practice it is more convenient to work with the equivalent form

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{R} \quad \to \quad \mathbf{A}' = \mathbf{Q}' \cdot \mathbf{R}' = \mathbf{Q} \cdot (\mathbf{R} + \mathbf{u} \otimes \mathbf{v}) \tag{2.10.8}$$

One can go back and forth between equations (2.10.7) and (2.10.8) using the fact that $\mathbf{Q}$ is orthogonal, giving

$$\mathbf{t} = \mathbf{v} \quad \text{and either} \quad \mathbf{s} = \mathbf{Q} \cdot \mathbf{u} \quad \text{or} \quad \mathbf{u} = \mathbf{Q}^T \cdot \mathbf{s} \tag{2.10.9}$$

The algorithm [2] has two phases. In the first we apply $N-1$ Jacobi rotations (§11.1) to reduce $\mathbf{R} + \mathbf{u} \otimes \mathbf{v}$ to upper Hessenberg form. Another $N-1$ Jacobi rotations transform this upper Hessenberg matrix to the new upper triangular matrix $\mathbf{R}'$. The matrix $\mathbf{Q}'$ is simply the product of $\mathbf{Q}$ with the $2(N-1)$ Jacobi rotations. In applications we usually want $\mathbf{Q}^T$, so the algorithm is arranged to work with this matrix (which is stored in the `QRdcmp` object) instead of with $\mathbf{Q}$.

```
void QRdcmp::update(VecDoub_I &u, VecDoub_I &v) {
```

qrdcmp.h

Starting from the stored QR decomposition $\mathbf{A} = \mathbf{Q} \cdot \mathbf{R}$, update it to be the QR decomposition of the matrix $\mathbf{Q} \cdot (\mathbf{R} + \mathbf{u} \otimes \mathbf{v})$. Input quantities are `u[0..n-1]`, and `v[0..n-1]`.

```
    Int i,k;
    VecDoub w(u);
    for (k=n-1;k>=0;k--)
        if (w[k] != 0.0) break;
    if (k < 0) k=0;
    for (i=k-1;i>=0;i--) {
        rotate(i,w[i],-w[i+1]);
        if (w[i] == 0.0)
            w[i]=abs(w[i+1]);
```

Find largest `k` such that `u[k]` $\neq 0$.

Transform $\mathbf{R} + \mathbf{u} \otimes \mathbf{v}$ to upper Hessenberg.

```
        else if (abs(w[i]) > abs(w[i+1]))
            w[i]=abs(w[i])*sqrt(1.0+SQR(w[i+1]/w[i]));
        else w[i]=abs(w[i+1])*sqrt(1.0+SQR(w[i]/w[i+1]));
    }
    for (i=0;i<n;i++) r[0][i] += w[0]*v[i];
    for (i=0;i<k;i++)                    Transform upper Hessenberg matrix to upper tri-
        rotate(i,r[i][i],-r[i+1][i]);      angular.
    for (i=0;i<n;i++)
        if (r[i][i] == 0.0) sing=true;
}

void QRdcmp::rotate(const Int i, const Doub a, const Doub b)
```

Utility used by `update`. Given matrices `r[0..n-1][0..n-1]` and `qt[0..n-1][0..n-1]`, carry out a Jacobi rotation on rows `i` and `i + 1` of each matrix. `a` and `b` are the parameters of the rotation: $\cos\theta = a/\sqrt{a^2+b^2}$, $\sin\theta = b/\sqrt{a^2+b^2}$.

```
{
    Int j;
    Doub c,fact,s,w,y;
    if (a == 0.0) {                  Avoid unnecessary overflow or underflow.
        c=0.0;
        s=(b >= 0.0 ? 1.0 : -1.0);
    } else if (abs(a) > abs(b)) {
        fact=b/a;
        c=SIGN(1.0/sqrt(1.0+(fact*fact)),a);
        s=fact*c;
    } else {
        fact=a/b;
        s=SIGN(1.0/sqrt(1.0+(fact*fact)),b);
        c=fact*s;
    }
    for (j=i;j<n;j++) {              Premultiply r by Jacobi rotation.
        y=r[i][j];
        w=r[i+1][j];
        r[i][j]=c*y-s*w;
        r[i+1][j]=s*y+c*w;
    }
    for (j=0;j<n;j++) {              Premultiply qt by Jacobi rotation.
        y=qt[i][j];
        w=qt[i+1][j];
        qt[i][j]=c*y-s*w;
        qt[i+1][j]=s*y+c*w;
    }
}
```

We will make use of QR decomposition, and its updating, in §9.7.

CITED REFERENCES AND FURTHER READING:

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I/8.[1]

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §5.2, §5.3, §12.5.[2]

2.11 Is Matrix Inversion an N^3 Process?

We close this chapter with a little entertainment, a bit of algorithmic prestidigitation that probes more deeply into the subject of matrix inversion. We start with a seemingly simple question:

How many individual multiplications does it take to perform the matrix multiplication of two 2×2 matrices,

$$\begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix} \cdot \begin{pmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{pmatrix} = \begin{pmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{pmatrix} \tag{2.11.1}$$

Eight, right? Here they are written explicitly:

$$\begin{aligned} c_{00} &= a_{00} \times b_{00} + a_{01} \times b_{10} \\ c_{01} &= a_{00} \times b_{01} + a_{01} \times b_{11} \\ c_{10} &= a_{10} \times b_{00} + a_{11} \times b_{10} \\ c_{11} &= a_{10} \times b_{01} + a_{11} \times b_{11} \end{aligned} \tag{2.11.2}$$

Do you think that one can write formulas for the c's that involve only *seven* multiplications? (Try it yourself, before reading on.)

Such a set of formulas was, in fact, discovered by Strassen [1]. The formulas are

$$\begin{aligned} Q_0 &\equiv (a_{00} + a_{11}) \times (b_{00} + b_{11}) \\ Q_1 &\equiv (a_{10} + a_{11}) \times b_{00} \\ Q_2 &\equiv a_{00} \times (b_{01} - b_{11}) \\ Q_3 &\equiv a_{11} \times (-b_{00} + b_{10}) \\ Q_4 &\equiv (a_{00} + a_{01}) \times b_{11} \\ Q_5 &\equiv (-a_{00} + a_{10}) \times (b_{00} + b_{01}) \\ Q_6 &\equiv (a_{01} - a_{11}) \times (b_{10} + b_{11}) \end{aligned} \tag{2.11.3}$$

in terms of which

$$\begin{aligned} c_{00} &= Q_0 + Q_3 - Q_4 + Q_6 \\ c_{10} &= Q_1 + Q_3 \\ c_{01} &= Q_2 + Q_4 \\ c_{11} &= Q_0 + Q_2 - Q_1 + Q_5 \end{aligned} \tag{2.11.4}$$

What's the use of this? There is one fewer multiplication than in equation (2.11.2), but *many more* additions and subtractions. It is not clear that anything has been gained. But notice that in (2.11.3) the a's and b's are never commuted. Therefore (2.11.3) and (2.11.4) are valid when the a's and b's are themselves matrices. The problem of multiplying two very large matrices (of order $N = 2^m$ for some integer m) can now be broken down recursively by partitioning the matrices into quarters, sixteenths, etc. And note the key point: The savings is not just a factor "7/8"; it is that factor at *each* hierarchical level of the recursion. In total it reduces the process of matrix multiplication to order $N^{\log_2 7}$ instead of N^3.

What about all the extra additions in (2.11.3) – (2.11.4)? Don't they outweigh the advantage of the fewer multiplications? For large N, it turns out that there are six times as many additions as multiplications implied by (2.11.3) – (2.11.4). But, if N is very large, this constant factor is no match for the change in the *exponent* from N^3 to $N^{\log_2 7}$.

With this "fast" matrix multiplication, Strassen also obtained a surprising result for matrix inversion [1]. Suppose that the matrices

$$\begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix} \quad \text{and} \quad \begin{pmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{pmatrix} \tag{2.11.5}$$

are inverses of each other. Then the c's can be obtained from the a's by the following operations (compare equations 2.7.11 and 2.7.25):

$$\begin{aligned} R_0 &= \text{Inverse}(a_{00}) \\ R_1 &= a_{10} \times R_0 \\ R_2 &= R_0 \times a_{01} \\ R_3 &= a_{10} \times R_2 \\ R_4 &= R_3 - a_{11} \\ R_5 &= \text{Inverse}(R_4) \\ c_{01} &= R_2 \times R_5 \\ c_{10} &= R_5 \times R_1 \\ R_6 &= R_2 \times c_{10} \\ c_{00} &= R_0 - R_6 \\ c_{11} &= -R_5 \end{aligned} \tag{2.11.6}$$

In (2.11.6) the "inverse" operator occurs just twice. It is to be interpreted as the reciprocal if the a's and c's are scalars, but as matrix inversion if the a's and c's are themselves submatrices. Imagine doing the inversion of a very large matrix, of order $N = 2^m$, recursively by partitions in half. At each step, halving the order *doubles* the number of inverse operations. But this means that there are only N divisions in all! So divisions don't dominate in the recursive use of (2.11.6). Equation (2.11.6) is dominated, in fact, by its 6 multiplications. Since these can be done by an $N^{\log_2 7}$ algorithm, so can the matrix inversion!

This is fun, but let's look at practicalities: If you estimate how large N has to be before the difference between exponent 3 and exponent $\log_2 7 = 2.807$ is substantial enough to outweigh the bookkeeping overhead, arising from the complicated nature of the recursive Strassen algorithm, you will find that LU decomposition is in no immediate danger of becoming obsolete. However, the fast matrix multiplication routine itself is beginning to appear in libraries like BLAS, where it is typically used for $N \gtrsim 100$.

Strassen's original result for matrix multiplication has been steadily improved. The fastest currently known algorithm [2] has an asymptotic order of $N^{2.376}$, but it is not likely to be practical to implement it.

If you like this kind of fun, then try these: (1) Can you multiply the complex numbers $(a+ib)$ and $(c+id)$ in only *three* real multiplications? [Answer: See §5.5.] (2) Can you evaluate a general fourth-degree polynomial in x for many different values of x with only *three* multiplications per evaluation? [Answer: See §5.1.]

CITED REFERENCES AND FURTHER READING:

Strassen, V. 1969, "Gaussian Elimination Is Not Optimal," *Numerische Mathematik*, vol. 13, pp. 354–356.[1]

Coppersmith, D., and Winograd, S. 1990, "Matrix Multiplications via Arithmetic Progressions," *Journal of Symbolic Computation*, vol. 9, pp. 251–280.[2]

Kronsjö, L. 1987, *Algorithms: Their Complexity and Efficiency*, 2nd ed. (New York: Wiley).

Winograd, S. 1971, "On the Multiplication of 2 by 2 Matrices," *Linear Algebra and Its Applications*, vol. 4, pp. 381–388.

Pan, V. Ya. 1980, "New Fast Algorithms for Matrix Operations," *SIAM Journal on Computing*, vol. 9, pp. 321–342.

Pan, V. 1984, *How to Multiply Matrices Faster*, Lecture Notes in Computer Science, vol. 179 (New York: Springer)

Pan, V. 1984, "How Can We Speed Up Matrix Multiplication?", *SIAM Review*, vol. 26, pp. 393–415.